

★ Overview

Agentic Code QA Instruction Guide

Last updated: May 16, 2026

Overview

Agentic Code QA evaluates whether models can answer realistic software-engineering questions by exploring an actual repository at a fixed commit. The project is designed to test **codebase-grounded reasoning**: understanding implementation details, tracing behavior across files, identifying risks or root causes, and explaining conclusions clearly enough for another engineer to trust. The term “QA” in this context is short for “Question & Answer.”

Your work will contribute to **improving AI capabilities for real-world software engineering**. By creating tasks that reflect how engineers actually investigate unfamiliar codebases, you help identify where AI agents still struggle with long-context repository understanding, technical judgment, and reliable developer-facing explanations.

Experts create tasks that **force the model to use the repository** rather than general programming knowledge. For each assigned repo at a specific commit, language, difficulty, and intent, contributors write a concise developer-style prompt (a question), a human-authored golden answer grounded in repository exploration, and a rubric that distinguishes correct, partial, and weak answers.

The rest of this guide contains the concrete working instructions: common mistakes to avoid, LLM usage boundaries, task creation and scope requirements, and reference examples.

Project Expectations

Project Expectations

Writers are expected to produce clear, accurate, and unique tasks while staying active and responsive throughout the project.

Expectation	Details
Quality and Consistency	• Deliver work that is specific to the assigned repository at a specific commit, intent, language, and difficulty. • Keep each task meaningfully distinct from prior submissions; repeated prompt patterns or generic rubrics may be sent back.
Minimum Activity	• Complete at least 5 tasks per week to remain active on the project. • Notify Team Leads in advance if you cannot meet the weekly minimum.
Task Timing	• The client target is for each task to complete the full write, review, and revision cycle within 5 hours. • Aim to write the initial task in 3-4 hours.
Engagement and Communication	• Monitor Slack and task interfaces regularly for reviewer feedback, process updates, and Team Lead messages. • Respond promptly when clarification is needed.



Payments

Payments

Writers are paid per merged task. Reviewers are paid hourly. Promotions may be available for top performers and will be communicated in Slack.

① Base Writer Pay

Base pay depends on how many revision cycles the task needs before it is merged.

- 0-3 revision cycles: \$120 per merged task.
- 4 or more revision cycles: \$0 per merged task. You may still submit the task if you choose, but the payout amount is zero.

② One-Shot Bonuses

One-shot bonuses apply only to tasks that merge with 0 revisions. Bonus amount depends on assigned task difficulty.

- Easy or medium difficulty: +\$30.
- Hard difficulty: +\$50.

③ LLM Tool Reimbursement

Mercor offers a \$20 monthly reimbursement for basic LLM tools, such as Cursor, used to examine repositories. This reimbursement is available after your first task is merged.

- Submit your receipt through the [reimbursement form](#). (Feel free to redact PII)
- Use only tools allowed by the [LLM Usage Policy](#) and project instructions.
- Submit one form per calendar month.

④ Payment Schedule

Payments are calculated on Saturdays (Pacific time), for the previous 7-day Friday -> Friday window. Merged tasks that achieved Merged status *within the previous window*

are included in that week's calculation. Payments typically appear for experts within the following week, per [Mercor payment policies](#).

Change Log

Change Log

Use this tab to see recent updates to the instruction guide. Review new entries before starting or revising tasks.

Date	Updates
1 May 2026	- Updated difficulty calibration thresholds. - Updated My First Task to reflect Step 6: New Test Model.
2 May 2026	Added How to Make a Task “Hard” 101.
5 May 2026	Added How to Design Rubrics 101.
7 May 2026	Updated intent language in How to Make a Task “Hard” 101 to more precisely match Intents.
8 May 2026	Added Architecture & System Design example task.
11 May 2026	Added rubric criteria limit guidance.
13-14 May 2026	- Added LLM Usage Policy. - Updated Payments. - Added Writing the Rubric. - Added How Task Claims Work. - Fixed image placement on Overview.
16 May 2026	- Updated formatting throughout - Updated “Step 6” -> “Step 5” throughout to match current workflow - Updated Task Examples
18 May 2026	- Updated Payments to clarify that code exploration tool reimbursements are monthly, not one-time - Added Payment Schedule to Payments tab.



LLM Usage Policy

LLM Usage Policy

Purpose

LLMs may support research, brainstorming, and quality checks. Final task content must reflect the contributor's **judgment, authorship, and verification**. The final submission should reflect your own understanding of the repository, prompt, rubric, and expected answer.

Allowed Uses

Use LLMs to strengthen your work while keeping final content personally written and verified. Acceptable uses include:

- Exploring repository context, task ideas, edge cases, and subtle failure modes.
- Checking grammar, clarity, tone, and formatting before submission.
- Stress-testing prompts, rubrics, and golden answers for missing coverage, vague wording, or compound criteria.
- Asking for technical background or areas that may need additional verification.

Requirements

You remain **fully accountable** for anything you submit. Before submitting, you must:

- Verify claims against the repository, environment, task instructions, or other authoritative sources.
- Rewrite any LLM-assisted wording in your own voice.
- Remove model-like artifacts, including generic phrasing, repetitive structure, stray formatting, excessive em dashes, emoji, or unsupported confidence.
- Ensure the prompt, rubric, and golden answer are grounded in the actual task context.

Prohibited Uses

Do not submit model-written work as your own. This includes:

- Model-written prompt statements, golden answers, or rubric criteria/rationales.
- Unverified model claims about code behavior, expected outputs, dependencies, or project requirements.
- Any task content that reads as generic, detached from the repository (or focused on the wrong commit), or not personally verified.

Our best advice: Do not copy and paste LLM-generated or LLM-formatted golden answers. Instead, use the blank `golden_answer.md` file (a blank stub is part of the task you download) and/or other artifact files as a scratch pad to workshop your ideas and revise as needed. Copy only the final, human-authored content into the task submission form.



Task Creation & Scope

Task Creation & Scope

This phase includes tasks that require one or more of the following:

- repo navigation and multi-file exploration
- architecture and design reasoning across components
- behavioral and control-flow analysis
- root cause identification from symptoms
- repository-scale code comprehension and explanation
- PR and change impact assessment
- code debugging and repair with executable fixes

All tasks have predefined:

- Language
- GitHub repo at a specific commit
- [Difficulty rating](#) (easy/medium/hard)
- [Intent](#)

End-to-End Task Workflow

End-to-End Task Workflow

This tab gives the high-level lifecycle of a task in Agentic Code QA. The detailed instructions for writing each artifact live in the next four tabs.



Workflow

① Writers create a repo-grounded task.

Writers claim an assigned repo, explore the codebase at the assigned commit, choose a realistic engineering question, write the prompt and golden answer, create the rubric, and run difficulty calibration + model evaluation.

② Reviewers inspect the task.

Reviewers check whether the prompt is fair and specific, the golden answer is correct, the rubric is measurable, the repository context is valid, and the difficulty tag matches the real task burden.

③ Writers revise when needed.

If the reviewer sends the task back, writers should address review feedback before claiming or starting new tasks. First tasks may take multiple revision cycles, but long-term quality is measured by low revision count and clean submissions.

④ Approved tasks are finalized.

Once approved, the task is finalized (“merged”) and ready for downstream use. The writer-facing model eval/calibration step already happened before submission as part of task creation.

Required Components Per Task

Each task must include the following core artifacts. Use this table as a quick reference; the detailed writing rules are in the later workflow tabs.

Artifact	Purpose	Key Requirements
Prompt (prompt_statement.md)	The question shown to the agent.	Contains only the question. May reference files, modules, functions, symptoms, patches, or line numbers by name. Must not paste large code snippets, extracted docs, or answer hints.
Golden Answer (golden_answer.md)	The expert reference answer.	Fully answers the prompt from repo evidence. Names relevant files/functions/flows and explains the reasoning behind the conclusion. Should be specific enough to produce atomic rubric criteria.
Rubric (rubric.json)	The grading rubric for model answers.	Usually 15-35 total criteria; up to 40 for hard tasks when needed. Include reasoning + completeness criteria, 5-10 penalties, and at least 2 style criteria. Never mark style as critical or penalty.
pr.patch (only some intents)	PR/change artifact provided for some intents.	Required when the question depends on a prewritten proposed code change.

		Examples: PR triage, architecture/design questions involving a patch, or code onboarding questions about changed behavior.
--	--	--



Task Claiming & Setup

Task Claiming & Setup

Before writing a task, make sure you have claimed the right task and can inspect the exact repository environment at the specific commit that the model will receive. Most setup mistakes create invalid tasks later, so resolve access, metadata, and repo-download issues before drafting.

Claiming Tasks

Claiming Rules

Before working on any task, you have to select and claim one. Follow the guidelines below depending on your situation.

Situation	Guideline	Tip
You are a new writer	Expect a claim cap of 2 inflight tasks until you have enough merged work.	Inflight includes tasks Claimed, Revising, Ready for Review, and Approved.
You have In Progress tasks	Work on those first before claiming a new task.	Revision and review loops should move before new work starts.
Available is blank	You are at claim cap, or no tasks match your selected languages.	Ask in Slack if this seems wrong.
Need another language	Ask the Slack helpdesk or Team Lead to add it.	Writers are not able to edit language choices after onboarding.

Task Dashboard FAQ

Use this section when the Airtable task dashboard shows unexpected results.

Dashboard Question	Answer
What are the dashboard sections?	• In Progress: tasks you already began; work these first. • Available: new tasks sorted by selected languages when tasks exist and you are below the claim cap. • Merged Tasks: previous merged work; use this to track progress toward 5 merged tasks (claim cap lifts) and verify payments.
The Available dashboard is blank. What is happening?	Usually one of two things happened: 1. No tasks are available for your selected language. 2. You reached your claim cap. If this seems wrong, ask in #code-qa-helpdesk on Slack.
How do I claim a task?	1. Open a task in Available, click Claim Task, and confirm it appears in In Progress. 2. Start working soon after claiming; tasks that sit in Claimed or Revising for many hours will be released for another writer.
I forgot to select a language during onboarding. What can I do?	Ask #code-qa-helpdesk or a Team Lead to add the language. Include the exact languages you want added. Writers do not have a self-serve UI for language changes after onboarding.
Which languages are available?	C, C++, C#, Cobol, CSS, Dart, Go, Hcl, HTML, Java, JavaScript, Kotlin, Lua, .NET, PHP, PowerShell, Python, R, React, React Native, Ruby, Rust, Scala, Shell, SQL, Swift, Terraform, TypeScript. Some languages may have no live tasks.

Setting Up Your Task

Setup Procedure

Use this procedure to confirm that your local workspace matches the task. You need enough local access to inspect the code and verify task-specific claims.

① Confirm task metadata.

Open the Airtable task and check the assigned language, repository, commit hash, task category/intent, and target difficulty. If any field conflicts with the actual repo (no repo in Dockerfile, commit doesn't exist, wrong language), flag it before writing. This is done by contacting a Team Lead or posting in the helpdesk channel on Slack.

② Authenticate and download the task.

Confirm GitHub access, run the Airtable clone/download commands, refresh expired S3 links when needed, and verify that the task templates are present locally.

③ Inspect the repo at the assigned commit.

Open the repository, confirm you are on the expected commit, and locate the files or modules that match the assigned intent. For example, if the task is about mini-cart navigation, inspect the mini-cart component, route definitions, and checkout/cart page handlers.

Common Setup Problems

Use this table to decide when to fix locally, refresh, or flag the task. To “Flag” a task, contact the helpdesk in Slack and/or a Team Lead directly. Writers do not have a “flag” button but Team Leads do.

Problem	What To Try	When To Flag
GitHub access fails	Run `gh auth status`. Confirm SSH access.	Flag if access cannot be granted or the task points to a repo you cannot inspect.

	Ask for repo/team access if permission is denied.	
S3 download fails	Click the Airtable button to generate a fresh S3 URL. Rerun the download command.	Flag if refreshed URLs still 404 or the task package is missing expected files.
Wrong commit or repo	Compare with Airtable. Check the remote URL.	Flag if the downloaded repo does not match assigned metadata, or is deleted on GitHub.
Docker/setup fails	Run only necessary scripts to set up the task (follow onscreen instructions during task setup).	Flag if missing files or broken setup (especially a Dockerfile lacking git commands or an explicit commit hash) prevents inspecting the behavior.
Language mismatch	Compare task language to the repo implementation language.	Flag with task number and correct language tag.
No usable repo	Check Dockerfile for a repo URL and checkout commit.	Flag if there is no valid repo or commit to inspect.

Repo Exploration & Task Scope

Repo Exploration & Task Scope

Repo exploration is where the task becomes specific. The goal is to find a realistic engineering question that requires codebase investigation and repo-specific evidence.

Repo Exploration

Exploration Process

Work from broad context toward a narrow, answerable question.

① Map the repo area.

Find the entry points, important modules, tests, configs, and files tied to the assigned intent. Build a short mental model before writing anything.

② Trace the relevant behavior.

Follow call chains, state updates, route transitions, config loading, data flow, PR dependencies, or error paths. Capture the exact files and functions that support the eventual golden answer.

③ Choose a task-shaped question.

The question should be narrow enough to answer definitively, but it should require meaningful exploration. A good task often asks about a non-obvious interaction, indirect dependency, lifecycle ordering, blast radius, or causal chain.

④ Check whether the task is fair.

A reviewer should be able to verify the answer from the repository, patch, or executable evidence. Hidden context or guesswork means the task needs revision before proceeding.

Intent Guide

Use the assigned intent to decide what kind of question to ask.

Intent	Definition	Examples
Architecture & System Design	How components are structured, interact, and why design decisions were made.	"How does authentication flow through this middleware stack?" "We need to add a caching layer to this service... how should we design it?"
Root Cause Analysis	Why behavior is broken, unexpected, or failing, traced through the codebase. These tasks include pre-written <code>pr.patch</code> files.	"This function returns None intermittently but I can't reproduce it locally. What's happening?" "Why does this test pass in isolation but fail in the suite?"
Code Onboarding & Comprehension	Plain-language walkthrough of what existing code, files, or functions do.	"I just joined this project. What does this module actually do?" "What is the entry point for this service, and how does execution flow from there?"
PR Triage & Impact Assessment	Explain a code change, assess blast radius, classify severity/complexity, and flag security risk. These tasks include pre-written <code>pr.patch</code> files.	"What does this PR actually change, and is anything likely to break?" "This diff touches the auth module. What are the security implications?" Real case: a PR adds a tag-parsing helper to an analytics schema. Use <code>pr.patch</code> plus the existing view that references the function to explain what changes, what could break, severity, and security concerns.

Task Scope

Scope Guidelines

These guidelines describe the line between a strong repo-grounded task and a generic or unfair task.

Guideline	Description	Examples / Tips
Ground the task in repo evidence	The question should depend on concrete files, functions, configs, routes, tests, state transitions, or patch context.	Example evidence: mini-cart handlers + route definitions. Example evidence: composition, transform interface, bbox/keypoint utilities. Example evidence: <code>pr.patch</code> + migration code impact.
Make the reasoning non-local	The model should connect multiple pieces of evidence or understand an interaction.	Good task shapes: handler vs route behavior; processor setup vs call-time validation; patch change vs downstream blast radius.
Keep the prompt fair	Give enough starting context for the model to know what problem to investigate.	Good: observed reload symptom + named UI actions. Bad: “Something is wrong with navigation. Find it.”
Avoid lookup tasks	Tasks need more than a single obvious function, docstring, or grep result.	Bad: “What does <code>`parseTags`</code> return?” Better: “How does the parser affect downstream analytics behavior?”
Avoid artificial vagueness	Hard tasks should be hard because repo behavior is subtle. The prompt still needs enough context to be fair.	If a reviewer cannot tell what evidence should answer it, the task is underspecified.
Check uniqueness	The task should be meaningfully distinct from prior submissions.	A unique task needs more than changed repo/file names in a repeated prompt pattern.

Scope Examples

Use examples like these to calibrate how much context belongs in the task idea.

Type	Example	Commentary
✓ Good root cause scope	Mini-cart reload example: “Trace which files and handlers implement Edit shopping bag / Proceed to checkout, what API they use to change screens, and how that differs from SPA navigation.”	Specific observed behavior. Requires multiple files. Does not reveal the <code>`window.location.href`</code> conclusion.
✗ Bad root cause scope	“The bug is caused by <code>`window.location.href`</code> in <code>miniCart.js</code> . Explain why.”	Gives away the answer path. Weakens exploration and difficulty.
✓ Good onboarding scope	<code>`Compose`</code> flow example: Ask how processors, accepted keys, aliases, validation, child transforms, filtering, and postprocess fit together.	A real onboarding question. Requires tracing across several modules.
✗ Bad onboarding scope	“What is <code>`Compose`</code> ?”	Definition-style lookup. Insufficient repo-specific reasoning.
✓ Good PR / architecture scope	PR / architecture example: Ask what ripples from a patch, or where a time-based autoscaling feature should fit in existing architecture.	Requires blast-radius or design reasoning beyond summary.
✗ Bad PR / architecture scope	“Summarize this diff” or “write a PRD for autoscaling.”	Too shallow or too detached from current repo structure.

Prompt & Golden Answer Writing

Prompt & Golden Answer Writing

The prompt and golden answer should be designed together. The prompt should make the agent explore the repo; the golden answer should prove the expert path through the relevant evidence.

Prompt Statement Guidelines

A prompt is the natural user-facing question the model will answer. It should give enough context for fair repo investigation while keeping the rubric and answer path out of view. Follow the guidelines below when writing your prompt.

Guideline	Description	Examples
Ask one clear engineering question	The prompt should have a concrete target behavior, symptom, flow, or change to analyze.	 Good: “Why do mini-cart checkout actions reload the whole page while checkout-step transitions stay in-app?”  Bad: “Explain this repository.”
Name useful starting points	Mention files, modules, actions, patches, or symptoms when needed to make the task fair.	 Good: “Look at the mini-cart actions and where routes are declared.”  Good: “Across `composition.py`, `transforms_interface.py`, bbox/keypoint utils, and label manager...”

Do not paste the answer path	Keep your discovered causal chain and downstream conclusion out of the prompt.	✓ Good: “What downstream behavior could this parser change affect?” ✗ Bad: “This parser feeds the analytics view and breaks SQL filters. Explain why.”
Avoid large copied code blocks	The repo is provided as context. The model should inspect it directly.	✓ Good: Reference `pr.patch` as an artifact. ✗ Bad: Paste the full patch into the prompt.
Match the assigned intent	The framing should match RCA, onboarding, PR triage, or architecture.	✓ Root cause: “What causes this observed reload?” ✓ PR review: “What should I flag before approval?” ✗ Architecture prompt that only asks for a bug fix.
Use realistic developer language	The prompt should sound like a human debugging, reviewing, or onboarding to a real codebase.	✓ Good: “I’m new on this project. I’m trying to understand the normal execution flow…” ✗ Bad: “As a world-class architect, produce a comprehensive treatise.”

Preserve answerability	Include the symptom, patch, flow, or area of uncertainty needed to make the question solvable.	✓ Good: “What we see: full document navigation from those two actions. What we expected: SPA-style transition.” ✗ Bad: “Something is wrong with checkout. Find it.”
-------------------------------	--	---

Golden Answer Guidelines

The golden answer is the reference answer reviewers and rubrics are anchored to. It should be specific enough to support rubric criteria and polished enough to read as a clear expert response.

Guideline	Description	Example / Tip
Start with the answer	Give the direct conclusion before the supporting details.	Example: “The reload happens because the mini-cart handlers assign <code>`window.location.href`</code>, causing browser document navigation.”
Show code evidence	Name the files/functions/modules that prove the answer.	Examples: • <code>`miniCart.js`</code> handlers. • <code>`Compose.__init__`</code> processor creation. • <code>`migrate-source-files.ts`</code> transform-array behavior.
Explain the causal chain	Connect evidence to outcome.	Explain more than “uses href”: React Router never sees the navigation intent, so the browser loads a new document.

Cover relevant alternatives	Mention plausible wrong explanations when they clarify the answer.	Example: GraphQL/useMiniCart explains cart contents only; the navigation mechanism comes from the click handlers.
Support the rubric	Write answer elements that can become atomic criteria.	Separate mechanism, functions, contrast path, affected files, and implications so the rubric can grade them separately.
Avoid LLM-like filler	Use concise, repo-grounded expert prose.	Cut out broad statements lacking repo evidence.

Prompt Examples

Good examples should force repo exploration without overfeeding the answer. Bad examples are either too broad, too leading, or too detached from the actual repository.

Intent	Example	Commentary
✓ Root Cause Analysis	Mini-cart reload example: “What we see: full document navigation from Edit shopping bag / Proceed to checkout. What we expected: SPA routing. Trace which handlers implement those clicks and how their URL change differs from in-app navigation.”	Specific observed symptom. Requires tracing handlers and routes. Does not reveal <code>`window.location.href`</code> .
✗ Root Cause Analysis	“The mini cart uses <code>`window.location.href`</code> ; explain why that reloads the browser.”	Gives away the core mechanism. Too close to the golden answer.
✓ Code Onboarding	<code>`Compose`</code> flow example:	Specific multi-module flow.

	“Walk through how `Compose` with bbox/keypoint params, label fields, aliases, and strict mode validates input, prepares processors, applies child transforms, filters, and postprocesses outputs.”	Still asks for explanation while keeping the answer path hidden.
 Code Onboarding	“What does `Compose.__call__` do?”	Too local and likely answerable from one method.
 PR review	Migration patch example: “What is `pr.patch` doing under the covers, which parts of the codebase ripple from it, and what should I flag before approval?”	Natural reviewer question. Forces blast-radius analysis.
 Architecture	Architecture example: “I want time-of-day autoscaling. Where should this fit architecturally, and how should config be defined?”	Asks for design placement and config mechanism only.



Rubric Writing & Difficulty Calibration

Rubric Writing & Difficulty Calibration

A rubric is the grading contract for a task. It tells us exactly which facts, reasoning steps, and harmful failure modes a model response must include or avoid in order to count as correct.

For Agentic Code QA, a strong rubric translates the golden answer into independent pass/fail checks grounded in repository evidence.

Target Difficulty Bands

- **Too Easy:** pass rate **85-100%** (not valid for this project)
- **Easy:** target pass rate **60–85%**
- **Medium:** target pass rate **35–60%**
- **Hard:** target pass rate **0–35%**
- All target ranges are **+/- 3%**

Rubric Criteria

The independent parts of a rubric are called “criteria”. Each criterion is a true/false statement that defines a good response to the prompt (...or a bad one, in the case of penalty criteria).

Criterion Writing Rules

Start by making each criterion precise enough to grade consistently. A reviewer should be able to read one criterion and decide whether the response earns it without guessing what you meant.

Rule	Description	Examples
Make each criterion atomic	One criterion should test one observable behavior. Split distinct facts, mechanisms, and recommendations into separate criteria.	✓ Good: “States that the Edit shopping bag click triggers a <code>`window.location.href`</code> assignment in <code>`miniCart.js`</code>.” ✗ Bad: “Explains checkout navigation and

		routing and reload behavior.”
Use measurable wording	A reviewer should know whether the response earns the point from the text alone. Avoid words like good, correct, appropriate, robust, and clear unless you anchor them to observable content.	✓ Good: “Explains that <code>`_set_keys`</code> expands the compose key registry before call-time validation.” ✗ Bad: “Correctly explains validation.”
Use concrete action verbs	Start each criterion with a concrete, present-tense action verb so the grader knows what output to look for.	Examples: States, Identifies, Explains, Includes, Distinguishes, Penalizes.
Tie criteria to repo evidence	Criteria should require evidence from the assigned codebase, PR, function, module, or behavior.	✓ Good: “Identifies that <code>`migrateSourceFiles`</code> now accepts a single transform or transform array.” ✗ Bad: “Mentions relevant migration files.”
Allow valid alternatives	Allow alternative phrasing or file sequences when another correct path proves the same answer.	✓ Tip: Credit either function names or equivalent click-action names if the response connects both actions to browser navigation.
Keep style non-critical	Style can improve readability, but factual correctness should carry the task.	✓ Good: “Separates conclusion from evidence and implications.” (weight: bonus) ✗ Bad: “Uses bullet points.” (weight: critical)
Avoid placeholders or	Every field must be final before submission. Placeholder text makes	Reject: TODO, TBD, [description], empty

template residue	the rubric ungradable.	rationale, copied template labels.
Keep the golden answer aligned	The golden answer should be able to earn every positive criterion and avoid every penalty. If the rubric changes, re-check the golden answer.	 Good: After adding a penalty for claiming a view was modified, confirm the golden answer correctly says the diff only introduces a function.

Criterion Examples

Use examples like these to make criteria atomic, measurable, and tied to the codebase.

Good criteria name the specific mechanism, behavior, or claim being evaluated.

Type	Example	Why It Is Good / Bad
 Reasoning	Root cause example: “States that assigning a URL string to <code>window.location.href</code> causes the browser to perform a full document navigation.”	Tests the mechanism that produces the reload. Grounded in the actual root cause example.
 Reasoning	Code flow example: “Explains that <code>Compose.add_targets</code> propagates alias mappings into child transforms.”	Specific processor/alias behavior. Specific to Compose behavior.
 Completeness	PR impact example: “Identifies that <code>updateSourceFile</code> now applies an array of transforms sequentially and preserves the modified source between transforms.”	Concrete PR behavior. Useful for blast-radius assessment.
 Penalty	SQL rubric example: Description: “Claims the view was modified in this PR.” Rationale: The diff only introduces a new function, it doesn’t modify the view	Specific false claim. Prevents inflated blast-radius reasoning.

✓ Bonus	“Formats the explanation using structured hierarchical elements like Markdown headers or numbered lists.”	Rewards proper formatting while not prescribing a single option.
✗ Vague criterion	“Explains the issue correctly.”	Unmeasurable. Fails to identify what the response must say.
✗ Compound criterion	“Names the handler, explains routing, mentions GraphQL, and proposes a fix.”	Too many independent checks in one criterion. Split into separate criteria.

Rubric Metadata

In addition to the rubric criterion, there are several other metadata fields that must be filled out to provide more information about your rubric. Each metadata field is described below.

1. Description

Fill `description` with the text of the pass/fail criterion. Follow all criterion guidelines above.

2. Rationale

Use `rationale` to explain why the criterion matters for grading this task. A good rationale connects the criterion to correctness, answerability, blast radius, or a common failure mode. It is for a human to read, and is not explicitly part of the test itself. **It is also relatively brief!** (We do not want an essay typed in the box.)

Example rationale: This criterion matters because the root-cause answer must distinguish the mini-cart click handler from GraphQL cart loading. Without that

distinction, the model can name nearby cart files while missing the reload mechanism being graded.

3. Tag

Use `tag` to identify the kind of criterion being evaluated. Choose the tag based on what the criterion tests, not where it appears in the answer.

Tag	Use For	Example Criterion
reasoning	Cause, mechanism, tradeoff, code behavior, data flow, control flow, or architectural implication.	Identifies that repeated `VOLATILE` calls make the generated SQL nondeterministic.
completeness	Required factual answer elements that must appear for the answer to be complete.	Includes both mini-cart click actions and the file where the navigation behavior is defined.
style	Organization, clarity, or readability. Style should be a small minority of the rubric.	Provides headers separating conclusion, evidence, and implication.

4. Weight

Use `weight` to set how the criterion affects grading. Most task-specific reasoning and required completeness criteria should be critical; optional nuance should be bonus; harmful false claims should be penalties.

Weight	Use For	Tips
critical	Core correctness. Missing this point would make the answer materially incomplete or wrong.	Use for most task-specific reasoning and required completeness criteria.
bonus	Useful optional nuance that	Use for safe follow-up analysis,

	improves the answer but is not required for correctness.	nonessential edge cases, or an extra implication. Also commonly used for style criteria.
penalty	Harmful false claims, misleading shortcuts, or bad engineering recommendations.	Penalizes claiming a PR changed a UI view when the diff only adds a helper function.

5. Dependencies

Use `dependent\_on` only when a criterion cannot be evaluated fairly unless another criterion is met first. For example, a downstream blast-radius criterion may depend on first identifying the correct modified function. Leave this field empty when the criterion can be graded on its own. Never make a critical criterion dependent on a bonus criterion.

Rubric Criteria Limits

Criterion Type	Required Range
Critical	7-23 (Mandatory to have both reasoning and completeness criteria as critical)
Bonus	4-17
Style	2-6
Penalty	5-10
<i>Total Criteria</i>	~35 target (40 maximum for harder tasks)

Using Qualifiers in Rubric Criteria

When a single requirement covers multiple conditions within the same case, use explicit qualifiers like “all,” “every,” or “each.” In some cases, you may use qualifiers like “one or more of” and then a list of comma-delimited options.

Qualifiers make it clear when the requirement is met if only a subset of conditions is satisfied.

 **Ambiguous:** The API exposes GET, POST, and PATCH methods

 **Clear:** The API exposes all three methods: GET, POST, PATCH

In the ambiguous example, it is a bit unclear whether the requirement is “one or more from the list” or “all of these.” Be explicit. Rubric evaluations tend to flip (change score) between runs when qualifiers are not clearly stated, leading to inaccurate grading.

QC & Calibration

Airtable QC & Submission Checks

We have built in a series of LLM tools within Airtable to help you check over your submission before sending it to a reviewer. Use these LLM checks as supporting signals before submission.

Human judgment remains required: read the outputs and revise only when the finding is valid.

Check	When To Run	How To Use The Result
Prompt QC	After drafting prompt.	Read the output, then use human judgment. Keep prompt wording under your control and avoid answer hints.
Rubric QC	After drafting rubric.	Look for non-atomic criteria, vague wording, missing penalties, or rubric/golden-answer mismatch. Revise the rubric and golden answer together when needed.

Golden Answer QC	When both the golden answer and rubric are populated.	Check whether the answer fully resolves the prompt from repo evidence. Treat the QC result as a signal that still requires human judgment.
Generate and Evaluate Model / New Test Model	After validation steps above pass.	This is the source of truth for difficulty. It may take 5-10 minutes or longer. Use the true score evaluation at the bottom. The run averages 3 evaluation runs but only prints detailed results from the first run.
Submit Your Task	After manual review, QC checks, validation, and difficulty calibration are complete.	Submit in Airtable only when the task is ready for reviewer inspection. If this is your first task, wait for reviewer approval before claiming another. Address revisions before starting new work.

Difficulty Calibration

Each task is associated with a given difficulty level, and you will have to calibrate your task to ensure that the finished task lands at the intended level of model challenge.

You will run a model eval that scores the model answer against the weighted rubric. To pass difficulty calibration, your rubric pass rate must align with the target difficulty.

Difficulty	Target Pass Rate	Typical Task Shape
Too Easy	>85%	Simple lookups or surface questions. Tasks in the Too Easy scoring band are not valid for this project.
Easy	60-85% (+/- 3%)	Localized repo question; usually answerable from 1-2 files without running code.
Medium	35-60% (+/- 3%)	Cross-file reasoning, data/control flow, abstraction behavior, or moderate PR impact analysis.
Hard	0-35% (+/- 3%)	Multi-hop reasoning, subtle edge cases, architecture tradeoffs, executable evidence, or system-level

		impact.
--	--	---------

Evaluation Signal

Use both the scored answer and the model trajectory when judging difficulty. A task can have the right pass rate for the wrong reason if the prompt is ambiguous, the rubric is misaligned, or the model succeeds without meaningful repo exploration.

Signal	What To Check	How To Interpret
Step 5 score	Use the Step 5 eval score before submission. This score includes rubric weights and penalties.	The score must land in the assigned band, with +/- 3% tolerance. Scores above 85% mean the task is too easy.
Golden validation	Confirm Validate Golden passes and the golden answer scores 100%.	A low golden score usually means the rubric and golden answer are misaligned. Fix that, run again. (See Golden Evaluation below.)
Prompt fairness	Check whether the prompt gives enough starting context without revealing the answer path.	Missing context makes the task unfair. Answer breadcrumbs make the task too easy.
Intent fit	Compare the final task to the assigned intent and difficulty target.	If the content no longer matches the assigned intent, retarget the prompt or flag the task rather than forcing calibration.

Calibration Fixes

When calibration misses the target, revise the prompt, golden answer, or rubric based on the reason it missed. Do not force a task into the assigned band by hiding necessary context or accepting generic answers.

Result	Likely Cause	Fixes
Too easy	Prompt gives away files/conclusion, answer is a local lookup, rubric accepts generic answers, or Step 5 passes with minimal exploration.	Remove answer breadcrumbs. Require deeper repo evidence. Scope toward a non-obvious interaction, risk, or causal chain.
Too hard or ambiguous	Prompt lacks needed symptom/file/patch context, rubric demands unsupported detail, or the answer path depends on hidden assumptions.	Clarify starting context. Add necessary references. Relax over-specific criteria that are not required for correctness.
Wrong intent	Task content no longer matches the assigned intent.	Retarget the prompt to the assigned intent or flag the task if the assignment itself is wrong.
Rubric mismatch	Golden answer does not achieve 100%, valid alternatives are penalized, or penalties distort the Step 5 score.	Rewrite criteria from the final golden answer. Add fair alternative paths. Check penalty weights before recalibrating.

Golden Evaluation

Golden answers were graded against the same rubric to verify rubric quality. Expected outcome: all critical criteria pass, all bonuses achievable, no penalties.

Metric	Value	Definition
Pass Rate	100% (100/100)	% of tasks where ALL critical criteria pass AND zero penalties triggered
Excellence Rate	100% (100/100)	% of tasks where ALL critical pass AND ≥ 1 bonus passes AND zero penalties

Violation Rate	0% (0/100)	% of tasks with ≥ 1 penalty triggered (penalty “passes” = bad behavior present)
----------------	------------	--

Tips for Making Hard Tasks

Triggering model failure is one of the hardest parts of this project. Follow the tips below to drive up task difficulty:

- **Start from research before writing the prompt.** Spend the first part of task creation understanding the repo, the assigned intent, relevant files, tests, issues, docs, and common user pain points. Hard tasks usually come from knowing enough context to ask about the non-obvious interaction a model would not naturally prioritize. Devote significant resources to research so that you have the knowledge to ask a hard question.
- **Give fair starting context while preserving discovery.** A long prompt that names every file, function, lifecycle step, and suspected cause can make a hard repo question easy. Provide the symptom, goal, patch, or subsystem the developer would realistically know, then let the model discover the exact evidence chain.
- **Trace indirect effects and downstream dependencies.** For PR triage, move beyond summarizing the diff: apply the patch mentally, trace where changed functions are called, and ask about the maintainability, correctness, security, or compatibility risk that appears after following those dependencies.
- **Make architecture tasks about real constraints.** Strong architecture prompts come from requested features, recurring user needs, or current repo limitations. Ask how the design should fit the existing modules, configuration, interfaces, and deployment constraints, so the answer requires tradeoff reasoning instead of generic design advice.
- **Make onboarding tasks explain a concrete flow or implementation path.** Generic comprehension prompts are easy because the model can summarize nearby code. Harder onboarding asks the model to explain how several pieces

work together, where a change would enter the system, or why a behavior emerges from aliases, processors, config, routes, state, or validation.

- **Make root-cause tasks distinguish plausible wrong explanations.** A hard root-cause prompt gives a realistic failing behavior and enough context to investigate, then requires the answer to trace the exact mechanism and rule out tempting but incorrect causes. Keep the file, line, and bug source out of the prompt.
- **Recalibrate after hardening.** If the task becomes hard because context is missing, add the missing context. If it remains easy because the model succeeds with shallow exploration, remove answer breadcrumbs or scope toward a deeper interaction. If it becomes unfair, repair the prompt before rerunning Step 5.



Rubric Design 101

How to Design Rubrics 101

Written by one of our talented Super Reviewers, this is a beginner-friendly guide to rubrics for this project! We encourage all new writers to read this section.

Before we dive into how to design rubrics for Agentic Code QA, let us step back to basics. Please note that **the first three sections apply to any AI training dataset** and not this project alone. It's a universal guide.

What are Rubrics

Rubrics are **evaluation parameters**. If you have ever done an assignment that has multiple right answers, **think of Rubrics as the evaluation parameters for the best possible right answer (our golden answer)**. That said, the other answers are not wrong, they are just not the best. **Agentic Code QA is looking for rubrics for the best possible answer.**

How are Rubrics Used

When a model is evaluated for correctness and other parameters, the model is expected to pass the rubric evaluation with a score of >85% (the threshold for “too easy”). If the model fails to do so, then it requires further training. In the AI world, this is called Reinforcement Learning. **You are teaching the model by rewarding it for correctness and accuracy and penalizing it for wrongdoing.** Just like how we teach children.

How to Write Rubrics

You have to first **stop thinking like a developer** and **start thinking like a parent/teacher**. Imagine you are coaching a child to provide your Golden Answer and have to evaluate them. What would you do? Ask them questions, give assessments to test their ability to recollect the answers, etc. Rubrics do the same thing. If something is marked `critical` in a rubric and the model does not do that, your model is not performing well. As discussed in the [section on how to make a task Hard](#), your answer has to be the best and most efficient answer without following any shortcuts. That makes rubrics vital to evaluate how good your answer is and helps in rewarding a model that produces it. AI training companies then use this to retrain their models by updating their weights accordingly.

Rubrics in Agentic Code QA

Rubrics are divided into 3 tags and 3 weights. We shall go through each one of them one by one and understand how and when to use them. By the end, I will teach you how to frame the rubrics in your mind while thinking about the golden answer.

Tags

1. **Reasoning:** Every question needs thinking. **You cannot answer a question without thinking.** Even if someone asks you 1×1 and you answer 1, your brain has thought about it, although for a split second. This thinking is what goes into Reasoning. All the thinking parts your golden answer is going to have, like thinking about using a piece of code, identifying something in the code, etc., are all part of Reasoning rubrics. All the statements that lead to the final answer are also Reasoning. **You are giving the model a step-by-step thinking methodology to arrive at your golden answer.** You are telling the model how to get there properly.
2. **Completeness:** Suppose your Golden Answer said something that completes the answer, say it has found **something that is important for understanding but it was discovered as part of the thinking process, that is completeness.** You can always use this to differentiate between Reasoning and Completeness. If it is important for thinking, it is tagged Reasoning. If it was discovered *as part of the thinking but needed*, it is tagged Completeness.
3. **Style:** This is the easiest of all. Note that Style *cannot be critical*. That is because you cannot penalize a model for not following one style. If I answer “ 1×1 is 1” or “1 is the answer for 1×1 ”, both are correct. This is style. **The presentation of the answer, how the parts and parcels of the code are used, how code is displayed, all these come under style.**

Weight

1. **Critical:** This is a **must-have** in the model's answer. **The model fails terribly if it does not generate this.** The model must be retrained if this is not answered. For example, imagine teaching a child about how humans breathe and they forget to mention or think about the nose. Would you accept that? Would you retrain the child by telling them that they also have to think about the nose?
2. **Bonus:** This is good to have. **The model does not fail, but it is good to have.** For example, the child says that we breathe through our mouth when the nose is blocked. This is a good thing to have in the answer. The answer is **not wrong without it, but is better with it.**

3. **Penalty:** This should *not* be in the model's answer. **You are giving negative criteria so the model stops doing this.** For example, if the child says that we can breathe through our ears, this is absolutely wrong and not possible, but perhaps the child is very persistent. You correct them, but first you penalize them for saying this *because you know it is incorrect* and would be a bad way for the child to continue thinking.

With this in mind, first try to get to the Critical rubric criteria required in the answer. Forget about bonus or penalty for a minute and just focus on Critical. **Note down all the rubric criteria that are required for the model to succeed while writing the golden answer itself.** If you use an LLM, then you would be disqualified. **Do not use an LLM to write your rubrics.**

If you write two lines in the golden answer that can form one rubric criterion, write that in parallel. You also evaluate your answer at the same time.

After all your Critical rubric criteria are covered, then check your Golden Answer. Did you mention something that is a Bonus? Include that. Then and only then, go ahead and write the Style criteria. See how your answer was framed, what was shown, etc. That should give you some basics.

Then go to the Penalties. Check rules for penalties below, but think of it like: *the model must not do this*. You frame these as if the model did this bad thing that it was not supposed to do. To write this, **imagine you are an evil developer who is asking a model to solve a prompt. Yup, evil. Apply your evil skills.**

Your rubric tells how to answer your prompt. If your prompt is already good enough and your answer has extra points/does not follow any shortcuts, you should have the right number of rubric criteria, no matter what.

Rules to Write Rubrics

I won't go into the basics here (basic rubric writing is covered elsewhere), and will focus on Hard tasks only.

1. Your rubric must award at least one critical point for an alternative answer that is correct. **The model thought a different way than you? No problem. That is not wrong.** At least give a mark for its efforts.

2. Rubric criteria must not overlap. **One rubric criterion must cover one part of the Golden Answer only.** That being said, you can combine a part or two of the golden answer into one rubric criterion, but you must follow Point 5 below, so be careful on what you are combining.
3. **Rubric criteria must be specific, but not overly specific.** Do not mention in detail the exact file, line, name, etc. Mentioning a function name in so-and-so file is OK, but going in depth is not acceptable. Why not? *You would force the model to answer your way.* This is not correct, because it closes off the ability to answer (or think) differently than you.
4. Rubrics **must not be vague or use vague/non-quantified terms. No words like “correctly,” “incorrectly,” “easy,” “clear,” etc.** Every rubric criterion is, by definition, a test of correctness. Avoid any subjective word like “good” or “clear” because it has no universal definition.
5. **Rubrics must be atomic.** One rubric criterion must cover one and only one part of the golden answer. Anything more will get rejected. There are exceptions, but those will be discussed on a case by case basis. Try to avoid “and” and “or” in your rubrics as much as you can. (The exceptions are usually when two things are inextricably linked, you cannot *ever* have one without the other, so it is OK to test them together in one rubric criterion. This is rare, but it does happen. A good case for using “or” is when there is a clearly scoped set of right answers, and *any one of them is OK*. In that case you should enumerate all of them and write something like: the model must do a, b, or c.)
6. **Penalties cannot be direct reversals of critical and bonus rubrics.** That would not make sense because you have already awarded points for doing the positive behavior. If you also took away points for *not doing the positive behavior*, you would be “double counting,” meaning penalizing the same behavior twice.

So the main points to remember are: **think like you are teaching a child how to answer a question** and **rubric criteria are used to evaluate the answer**. Get into the boots of a parent or teacher and writing rubrics will get very easy.



Task Examples

Use the left navigation to view one task example from each intent.

1 Example Task 5368 Root Case Analysis

Example Task 5368

Name: liam-hq-liam-1408-rca
Intent: Root Case Analysis
Difficulty: Medium
Language: TypeScript

Repo

Repository: <https://github.com/liam-hq/liam>

Checkout (commit): 21113d02f61b166a8a0d2316ff7b1566fc84c551

Prompt

Some UNIQUE constraints from my pg_dump file aren't reflected in the SQL parser's column metadata. The parser's own one-to-one test passes, so the feature works for some patterns. What determines which UNIQUE constraints get picked up and which don't?

LLM Prompt Review

Prompt QC Feedback

****Grading Item 1**** (1/1)

The question seeks nuanced insight into why certain UNIQUE constraints are missed, indicating more than a superficial query.

****Grading Item 2**** (1/1)

It reads as a natural developer-to-developer question without role-play language.

****Grading Item 3**** (1/1)

It avoids basic definitions and instead probes behavior of a specific feature.

****Grading Item 4**** (1/1)

Answering requires understanding how the parser processes repo-specific inputs, implying real exploration.

****Grading Item 5**** (1/1)

The prompt is a question only and does not include pasted code or logs.

****Grading Item 6**** (1/1)

It is specific and asks a concrete question about repo behavior with a verifiable answer.

****Grading Item 7**** (1/1)

The prompt aligns with the intent by demanding reasoning about repository-driven functionality.

****Grading Item 8**** (1/1)

It calls for technical judgment about parser behavior rather than a trivial lookup.

****Grading Item 9**** (1/1)

No meta-instructions or formatting directives are present.

****Grading Item 10**** (1/1)

Contains only ASCII characters and standard punctuation.

FINAL_SCORE: 10

Rubric

id	tag	description	weight	rationale	dependent_on
48552	reasoning	Identifies <code>handleOneToOneRelationships</code> as the shared utility both parsers call, which checks <code>column.unique</code> to determine cardinality.	bonus	The shared utility is the pivot point. The discrepancy must be upstream of it.	
48553	reasoning	Explains the <code>schema.rb</code> parser sets <code>column.unique</code> through <code>extractColumnOptions</code> when the column definition includes <code>unique: true</code> .	critical	Understanding WHY <code>schema.rb</code> works is half of explaining the discrepancy.	

48554	reasoning	Explains the SQL parser only sets column.unique from inline CONSTR_UNIQUE in ColumnDef elements within handleCreateStmt.	critical	Understanding what the SQL parser DOES handle narrows the gap.	
48555	reasoning	Identifies ALTER TABLE ADD CONSTRAINT UNIQUE is dropped by the SQL converter without updating column.unique.	critical	The most impactful gap since pg_dump uses this pattern.	
48556	reasoning	Explains pg_dump separates constraints into ALTER TABLE statements instead of inlining them in column definitions.	bonus	The root cause is a format mismatch.	
48557	reasoning	Notes the SQL parser's one-to-one test uses inline UNIQUE on the column definition.	critical	The prompt explicitly asks why the test suite doesn't catch this.	48554
48558	reasoning	Notes CREATE UNIQUE INDEX does not result in column.unique being set to true.	critical	A second distinct gap in the SQL parser's uniqueness handling.	
48559	reasoning	Notes constraintToRelationship checks contype against CONSTR_FOREIGN and returns undefined for all other constraint types.	bonus	Traces the exact code that discards UNIQUE constraints.	

48560	reasoning	Notes handleCreateStmt only processes elements where ColumnDef in elt, skipping table-level Constraint nodes entirely.	bonus	A third gap.	
48561	reasoning	Notes the ALTER TABLE variant of the one-to-one test also places UNIQUE inline on the column in CREATE TABLE, not in the ALTER TABLE statement.	critical	Shows both SQL one-to-one tests exercise the same path.	48557
48562	reasoning	Notes schema.rb one-to-one detection depends on inline column-level `unique: true`; unique index declarations are stored as indexes and do not by themselves set `column.unique`.	critical	Explains the architectural reason the schema.rb parser works.	
48563	reasoning	Claims both parsers have the same uniqueness handling bug.	penalty	The schema.rb parser handles inline unique: true correctly.	
48564	reasoning	Claims handleOneToOneRelationships is the root cause rather than the upstream uniqueness handling in the SQL converter.	penalty	handleOneToOneRelationships correctly reads column.unique.	

48565	reasoning	Attributes the issue to pr.patch changes rather than pre-existing converter behavior.	penalty	The SQL converter code at the base commit already has these gaps.	
48566	completeness	Locates the SQL converter at frontend/packages/db-structure/src/parser/sql/postgresql/converter.ts.	critical	File path grounds the SQL parser analysis.	
48567	completeness	Locates the schema.rb parser at frontend/packages/db-structure/src/parser/schemarb/parser.ts.	critical	File path grounds the schema.rb parser analysis.	
48568	completeness	Locates handleOneToOneRelationships at frontend/packages/db-structure/src/parser/utls/handleOneToOneRelationships.ts.	critical	File path grounds the shared utility.	
48569	completeness	Identifies extractColumnOptions as the schema.rb parser function that reads the unique keyword from column definitions.	critical	Names the specific function in the schema.rb parser responsible for the working path.	
48570	completeness	Identifies constraintToRelationship as the SQL converter function that filters out non-FK constraints.	critical	Names the specific function where UNIQUE constraints are discarded.	
48571	completeness	Notes no SQL parser test exercises the scenario where uniqueness is added via ALTER TABLE rather than inline on the column.	critical	The prompt explicitly asks why the test suite doesn't catch this.	

48572	completeness	Notes the FIXME comment and skip'd test in the SQL parser test file about CONSTRAINT statement support.	critical	Documentary evidence the parser creators knew about the limitation.	
48573	completeness	Locates the SQL parser test file at frontend/packages/db-structure/src/parser/sql/postgresql/index.test.ts.	bonus	File path grounding.	
48574	completeness	Provides concrete SQL examples showing the inline pattern that works versus the ALTER TABLE pattern that fails.	bonus	Examples make the gap tangible.	
48575	completeness	Notes table-level UNIQUE constraints in CREATE TABLE are also skipped because handleCreateStmt only processes ColumnDef elements.	bonus	A third distinct gap.	
48576	completeness	Notes handleIndexStmt stores index data in table.indexes without updating column.unique.	bonus	Clarifies index data exists but is disconnected from column metadata.	
48577	completeness	Claims the schema.rb parser also fails to detect column uniqueness from inline unique: true.	penalty	The schema.rb parser handles this correctly.	

48578	completeness	Claims inline UNIQUE in CREATE TABLE SQL is also broken.	penalty	Inline column-level UNIQUE is the one path the SQL parser handles correctly.	
48579	style	Traces both parsers systematically, comparing their uniqueness handling paths.	bonus	A cross-parser comparison is clearer when both sides are traced explicitly.	
48580	style	References specific file paths, function names, and code patterns from the repository.	bonus	Repo-grounded answers are more verifiable.	
48581	style	Avoids proposing a complete code patch, staying focused on diagnosis.	bonus	RCA intent is diagnosis, not implementation.	

LLM Rubric Review

Rubric QC Feedback

****Grading Item 1:** The rubric is organized into exactly three top-level evaluation categories: reasoning, completeness, and style. All three categories are present. (2/2)**

All three required categories are present as separate top-level arrays in the rubric JSON.

****Grading Item 2:** The rubric contains a reasonable number of grading items for the task complexity and covers the task holistically. (2/2)**

There are 30 total items (14 reasoning, 13 completeness, 3 style), which is within the suggested range and offers broad coverage for the task.

****Grading Item 3:** Every grading item is atomic and self-contained. (2/2)**

Each item focuses on a single, distinct expectation; none bundle multiple independent checks.

****Grading Item 4: Grading item descriptions are clear, specific, and measurable. (2/2)****

The descriptions cite concrete behaviors, file paths, or statements that can be objectively verified.

****Grading Item 5: Every grading item is derivable from the provided golden_answer. (1/2)****

Nearly all items trace directly to details in the golden answer, but the penalty about attributing the issue to PR changes lacks support from the golden answer, so full credit isn't warranted.

****Grading Item 6: The rubric is comprehensive relative to the golden_answer. (2/2)****

All major points from the golden answer—shared utility, schema.rb handling, SQL converter gaps, pg_dump patterns, and test coverage—are represented.

****Grading Item 7: The rubric would award full marks to the provided golden_answer. (2/2)****

The golden answer satisfies every critical requirement and avoids all penalties described, so it would earn full credit.

****Grading Item 8: The rubric is fair to valid alternative reasoning paths. (2/2)****

Although detailed, the items allow any repo-grounded explanation matching the documented behaviors, not just the golden answer's wording.

****Grading Item 9:** The reasoning category evaluates correctness of reasoning, technical judgment, conceptual understanding, and justification. (2/2)**

Reasoning items focus on how uniqueness is set or missed, why tests pass, and relevant parser mechanics—true reasoning content.

****Grading Item 10:** The completeness category evaluates coverage of all major question parts and necessary supporting details. (2/2)**

Completeness items emphasize referencing key files, tests, and evidence, aligning with coverage requirements rather than correctness.

****Grading Item 11:** The style category evaluates clarity, organization, readability, and professionalism of the response. (2/2)**

Style items focus on presentation aspects such as staying diagnostic, citing repo paths, and tracing both parsers coherently.

****Grading Item 12:** Grading items are placed in appropriate categories. (2/2)**

Each item's placement matches its focus; there are no misplaced reasoning/completeness/style checks.

****Grading Item 13:** The rubric includes the task-wide expectations: reasoning quality, completeness, and code style. (2/2)**

Each of the three categories contains items covering these broad expectations.

****Grading Item 14: The rubric includes intent-specific grading items when required. (2/2)****

Given the RCA intent, items about root causes, parser gaps, and test explanations satisfy the intent-specific needs.

****Grading Item 15: Weights use only valid values and are well calibrated. (2/2)****

All items use `critical`, `bonus`, or `penalty`, and their assignment reflects the importance of each expectation.

****Grading Item 16: Penalty items target genuinely harmful or misleading behavior. (2/2)****

Penalties address incorrect attributions or conflating parser behaviors, each representing distinct harmful misunderstandings.

****Grading Item 17: Dependency usage is logically sound. (2/2)****

Dependencies exist only where logical prerequisites are needed, and their directions make sense.

****Grading Item 18: The rubric is not overprescriptive. (2/2)****

While detailed, it allows alternative repo-grounded explanations and doesn't force an exact narrative sequence.

Grading Item 19: Each grading item contributes meaningfully to evaluating answer quality. (2/2)

Items are non-redundant and each adds evaluative value.

FINAL_SCORE: 37

Golden Answer

The shared cardinality logic

Both the schema.rb and PostgreSQL SQL parsers call the same `handleOneToOneRelationships` function from `frontend/packages/db-structure/src/parser/utils/handleOneToOneRelationships.ts`. This function iterates every relationship and promotes it from `ONE_TO_MANY` to `ONE_TO_ONE` when the foreign key column has `column.unique === true`. No other code path touches cardinality. So the discrepancy comes down to whether each parser sets `column.unique` before this function runs.

How the schema.rb parser sets column.unique

In the schema.rb parser (`frontend/packages/db-structure/src/parser/schemarb/parser.ts`), `extractColumnDetails` delegates to `extractColumnOptions`, which reads keyword arguments from the column definition. When a column is declared with `unique: true` in the schema.rb DSL:

```
```ruby
t.bigint "user_id", unique: true
```
```

The `case 'unique'` branch sets `column.unique = value instanceof TrueNode`. This is a direct, inline property on the column definition. For this inline schema.rb pattern, the uniqueness appears right where the parser expects it; separate `t.index ..., unique: true` declarations are stored as indexes, not copied into `column.unique`.

How the SQL parser fails to set column.unique

The PostgreSQL SQL parser

(`frontend/packages/db-structure/src/parser/sql/postgresql/converter.ts`) handles column uniqueness inside `handleCreateStmt`. It checks each `ColumnDef` element for inline `CONSTR\_UNIQUE` or `CONSTR\_PRIMARY` constraints:

```
```typescript
unique:
 colDef.constraints
 ?.filter(isConstraintNode)
 .some((c) =>
 ['CONSTR_UNIQUE', 'CONSTR_PRIMARY'].includes(
 c.Constraint.contype ?? "",
),
) || false,
...
```
```

This works for `CREATE TABLE posts (user\_id INT UNIQUE)` where the UNIQUE constraint is part of the column definition. But `pg\_dump` does not emit uniqueness this way. A standard `pg\_dump` output separates constraints from table definitions:

```
```sql
CREATE TABLE posts (
 user_id integer NOT NULL
);
```
```

```
ALTER TABLE ONLY posts ADD CONSTRAINT posts_user_id_key UNIQUE (user_id);
...
```
```

The `ALTER TABLE` statement goes through `handleAlterTableStmt`, which processes `AT\_AddConstraint` commands by calling `constraintToRelationship`. But that function returns early for anything except foreign keys:

```
```typescript
if (constraint.contype !== 'CONSTR_FOREIGN') {
  return ok(undefined)
}
...
```
```

The UNIQUE constraint is dropped. `column.unique` stays `false`.

The same gap exists for two other SQL patterns. Table-level UNIQUE constraints inside `CREATE TABLE` (`UNIQUE (user_id)` as a separate clause) are skipped because `handleCreateStmt` only processes elements where `'ColumnDef'` in `elt`. And `CREATE UNIQUE INDEX` statements are captured by `handleIndexStmt` as Index objects with `unique: true`, but the index uniqueness is never propagated to the column's `unique` field.

### ## Why the inline schema.rb case avoids this problem

For the schema.rb path that drives one-to-one detection in these tests, uniqueness is detected when the column call itself includes `unique: true`, such as `t.bigint "user_id", unique: true`. The schema.rb parser reads that inline attribute through `extractColumnOptions`, which handles `unique` as a dedicated keyword.

Unique indexes in schema.rb, such as `t.index ["user_id"], unique: true`, are parsed as index metadata instead. They do not by themselves set `column.unique`, so they are not the schema.rb mechanism that `handleOneToOneRelationships` uses for cardinality.

The SQL format in the failing pg\_dump-style case has no equivalent inline column flag. PostgreSQL splits these apart: the column definition carries type and nullability, while constraints live in separate statements (either table-level in `CREATE TABLE` or via `ALTER TABLE`). The SQL parser was built to handle the inline form but not the separated form.

### ## Why the SQL parser's one-to-one test passes

The `foreign key (one-to-one)` test in `frontend/packages/db-structure/src/parser/sql/postgresql/index.test.ts` declares:

```
```sql
CREATE TABLE posts (
  id BIGSERIAL PRIMARY KEY,
  user_id INT REFERENCES users(id) UNIQUE
);
```
```

The UNIQUE is inline on the column definition, hitting the one path the SQL parser handles. The test never uses the separated form (``ALTER TABLE ADD CONSTRAINT UNIQUE`` or ``CREATE UNIQUE INDEX``). The ALTER TABLE variant of the one-to-one test also puts UNIQUE inline in the CREATE TABLE and only uses ALTER TABLE for the foreign key itself. Because both tests exercise the single working path, they pass despite the gaps.

The test file also contains a skip'd test with a FIXME comment (``// FIXME: 'CONSTRAINT' statement is not supported yet``) for table-level CONSTRAINT syntax inside CREATE TABLE. The skip acknowledges the parser developers were aware of the limitation for table-level constraints.

## ## The root cause

The discrepancy is a format mismatch. The SQL parser's uniqueness detection was built for inline column constraints (the form used in the tests), but ``pg_dump`` emits uniqueness through ALTER TABLE and CREATE INDEX statements (which the parser drops). The schema.rb parser avoids this problem for inline column-level ``unique: true`` declarations because those set ``column.unique`` directly. The shared ``handleOneToOneRelationships`` utility is correct; it reads ``column.unique`` correctly. The issue is that the SQL parser's upstream code fails to populate that field for the most common PostgreSQL output patterns.

## Golden Answer Review

Golden Answer QC Feedback

**\*\*Grading Item 1: Internal consistency and technical accuracy (3/3)\*\***

No contradictions or unsupported claims; code references and logic align with repo context.

---

**\*\*Grading Item 2: Repo-grounded analysis (3/3)\*\***

References concrete files, functions, and behaviors within the repository.

---

**\*\*Grading Item 3: Direct answer to prompt (2/2)\*\***

Explains exactly which UNIQUE patterns are detected and why.

---

**\*\*Grading Item 4: Reasoning chain (3/3)\*\***

Walks through the causal steps from parser behavior to observed discrepancy.

---

**\*\*Grading Item 5: Technical precision (2/2)\*\***

Mechanistic descriptions specify how data flows and why gaps occur.

---

**\*\*Grading Item 6: Factual accuracy (2/2)\*\***

Statements about code and framework behavior are correct for this repo.

---

**\*\*Grading Item 7: Critical rubric requirements (3/3)\*\***

All must-have rubric elements are addressed.

---

**\*\*Grading Item 8: Penalty avoidance (2/2)\*\***

No penalty-triggering behaviors present.

---

**\*\*Grading Item 9: Comprehensiveness (2/2)\*\***

Covers all major aspects of the discrepancy and supporting evidence.

---

**\*\*Grading Item 10: Clarity for non-experts (2/2)\*\***

Clear structure and language suitable for onboarding readers.

---

**\*\*Grading Item 11: Concrete supporting details (2/2)\*\***

Includes file paths, function names, and specific SQL patterns.

---

**\*\*Grading Item 12: Intent-specific expectations (3/3)\*\***

Meets root-cause-analysis expectations, focusing on diagnosis.

---

**\*\*Grading Item 13: PR Triage structure (2/2)\*\***

Not applicable; full credit granted.

---

**\*\*Grading Item 14: ASCII-only (1/1)\*\***

No non-ASCII characters used.

---

FINAL\_SCORE: 32

## Validate Golden Answer

Golden Answer Validation

```
{
 "results": [
 {
 "id": "reasoning-48560",
 "tag": "reasoning",
 "weight": "bonus",
 "description": "Notes handleCreateStmt only processes elements where ColumnDef in elt, skipping table-level Constraint nodes entirely.",
 "rationale": "A third gap.",
 "dependent_on": [],
```



```

 "pass": true,
 "evidence": "\u201cTable-level UNIQUE constraints inside CREATE TABLE (`UNIQUE (user_id)` as a
separate clause) are skipped because `handleCreateStmt` only processes elements where `ColumnDef` in
elt`.\u201d",
 "reasoning": "The golden answer explicitly states that handleCreateStmt restricts processing to
elements with ColumnDef, explaining why table-level constraint nodes are skipped, directly satisfying the
criterion."
 },
 {
 "id": "reasoning-48557",
 "tag": "reasoning",
 "weight": "critical",
 "description": "Notes the SQL parser's one-to-one test uses inline UNIQUE on the column definition.",
 "rationale": "The prompt explicitly asks why the test suite doesn't catch this.",
 "dependent_on": [
 "reasoning-48554"
],
 "pass": true,
 "evidence": "\u201cThe `foreign key (one-to-one)` test \u2026 declares \u2026 `user_id INT
REFERENCES users(id) UNIQUE`. The UNIQUE is inline on the column definition, hitting the one path the
SQL parser handles. The test never uses the separated form \u2026 Because both tests exercise the
single working path, they pass despite the gaps.\u201d",
 "reasoning": "The answer explicitly states that the SQL parser's one-to-one test places the
UNIQUE constraint inline on the column definition, explaining why the tests pass and showing
understanding of the test setup."
 },
 {
 "id": "reasoning-48558",
 "tag": "reasoning",
 "weight": "critical",
 "description": "Notes CREATE UNIQUE INDEX does not result in column.unique being set to true.",
 "rationale": "A second distinct gap in the SQL parser's uniqueness handling.",
 "dependent_on": [],
 "pass": true,
 "evidence": "And `CREATE UNIQUE INDEX` statements are captured by `handleIndexStmt` as Index
objects with `unique: true`, but the index uniqueness is never propagated to the column's `unique` field.",

```

"reasoning": "The golden answer explicitly states that CREATE UNIQUE INDEX statements do not cause column.unique to be set to true, highlighting this limitation in the SQL parser."

},

{

"id": "reasoning-48561",

"tag": "reasoning",

"weight": "critical",

"description": "Notes the ALTER TABLE variant of the one-to-one test also places UNIQUE inline on the column in CREATE TABLE, not in the ALTER TABLE statement.",

"rationale": "Shows both SQL one-to-one tests exercise the same path.",

"dependent\_on": [

"reasoning-48557"

],

"pass": true,

"evidence": "\u201cThe `foreign key (one-to-one)` test ... declares \u2026 UNIQUE is inline \u2026 The ALTER TABLE variant of the one-to-one test also puts UNIQUE inline in the CREATE TABLE and only uses ALTER TABLE for the foreign key itself.\u201d",

"reasoning": "The golden answer explicitly states that the ALTER TABLE one-to-one test still places the UNIQUE constraint inline in the CREATE TABLE definition, demonstrating awareness that both tests exercise the same code path."

},

{

"id": "reasoning-48562",

"tag": "reasoning",

"weight": "critical",

"description": "Notes Rails schema.rb format always inlines column attributes on the column definition call as a design convention.",

"rationale": "Explains the architectural reason the schema.rb parser works.",

"dependent\_on": [],

"pass": true,

"evidence": "Rails' `schema.rb` dumper bakes column attributes directly into the column definition call. A unique column appears as `t.bigint `\"user\_id\", unique: true`, not as a separate constraint or index statement.",

"reasoning": "The answer explicitly states that Rails schema.rb inlines column attributes on the column definition call, matching the criterion."

},

```

{
 "id": "reasoning-48555",
 "tag": "reasoning",
 "weight": "critical",
 "description": "Identifies ALTER TABLE ADD CONSTRAINT UNIQUE is dropped by the SQL converter
without updating column.unique.",
 "rationale": "The most impactful gap since pg_dump uses this pattern.",
 "dependent_on": [],
 "pass": true,
 "evidence": "\u201cThe `ALTER TABLE` statement goes through `handleAlterTableStmt`, which
processes `AT_AddConstraint` commands by calling `constraintToRelationship`. But that function returns
early for anything except foreign keys\u2026 The UNIQUE constraint is dropped. `column.unique` stays
`false`.\u201d",
 "reasoning": "The answer explicitly states that ALTER TABLE ADD CONSTRAINT UNIQUE statements
are processed but discarded because the converter returns early unless the constraint is foreign, meaning
column.unique is never updated."
},
{
 "id": "reasoning-48565",
 "tag": "reasoning",
 "weight": "penalty",
 "description": "Attributes the issue to pr.patch changes rather than pre-existing converter behavior.",
 "rationale": "The SQL converter code at the base commit already has these gaps.",
 "dependent_on": [],
 "pass": true,
 "evidence": "The response explains the existing limitations of the SQL parser without attributing them
to any recent patch: \u201cThe SQL parser\u2019s uniqueness detection was built for inline column
constraints\u2026 pg_dump emits uniqueness through ALTER TABLE and CREATE INDEX statements
(which the parser drops).\u201d",
 "reasoning": "The answer attributes the issue to longstanding behavior in the SQL parser\u2019s
handling of constraints, not to any PR or new patch changes, so it meets the criterion."
},
{
 "id": "reasoning-48563",
 "tag": "reasoning",
 "weight": "penalty",

```

```

 "description": "Claims both parsers have the same uniqueness handling bug.",
 "rationale": "The schema.rb parser handles inline unique: true correctly.",
 "dependent_on": [],
 "pass": true,
 "evidence": "\u201cWhen a column is declared with `unique: true` \u2026 The `case 'unique` branch sets `column.unique = value` instanceof TrueNode` \u2026 The PostgreSQL SQL parser \u2026 works for \u2026 inline \u2026 But `pg_dump` does not emit uniqueness this way \u2026 The UNIQUE constraint is dropped.\u201d",
 "reasoning": "The golden answer differentiates the parsers, explaining the schema.rb parser correctly handles inline unique attributes while the SQL parser fails for pg_dump patterns; it does not claim both share the same bug."
 },
 {
 "id": "reasoning-48564",
 "tag": "reasoning",
 "weight": "penalty",
 "description": "Claims handleOneToOneRelationships is the root cause rather than the upstream uniqueness handling in the SQL converter.",
 "rationale": "handleOneToOneRelationships correctly reads column.unique.",
 "dependent_on": [],
 "pass": true,
 "evidence": "\u201cThis function iterates every relationship and promotes it from `ONE_TO_MANY` to `ONE_TO_ONE` when the foreign key column has `column.unique === true`. No other code path touches cardinality. So the discrepancy comes down to whether each parser sets `column.unique` before this function runs.\u201d",
 "reasoning": "The golden answer explicitly states that handleOneToOneRelationships functions correctly and attributes the discrepancy to upstream SQL parser logic failing to set column.unique, thus avoiding the prohibited claim."
 },
 {
 "id": "reasoning-48552",
 "tag": "reasoning",
 "weight": "bonus",
 "description": "Identifies handleOneToOneRelationships as the shared utility both parsers call, which checks column.unique to determine cardinality.",
 "rationale": "The shared utility is the pivot point. The discrepancy must be upstream of it.",

```

```

 "dependent_on": [],
 "pass": true,
 "evidence": "\u201cBoth the schema.rb and PostgreSQL SQL parsers call the same
`handleOneToOneRelationships` function \u2026 This function \u2026 promotes it from `ONE_TO_MANY`
to `ONE_TO_ONE` when the foreign key column has `column.unique === true`.\u201d",
 "reasoning": "The answer explicitly states that both parsers use the shared
`handleOneToOneRelationships` utility and explains that it inspects `column.unique` to determine
cardinality, directly satisfying the criterion."
 },
 {
 "id": "reasoning-48556",
 "tag": "reasoning",
 "weight": "bonus",
 "description": "Explains pg_dump separates constraints into ALTER TABLE statements instead of
inlining them in column definitions.",
 "rationale": "The root cause is a format mismatch.",
 "dependent_on": [],
 "pass": true,
 "evidence": "\u201cBut pg_dump does not emit uniqueness this way. A standard pg_dump output
separates constraints from table definitions: ... ALTER TABLE ONLY posts ADD CONSTRAINT
posts_user_id_key UNIQUE (user_id);\u201d",
 "reasoning": "The answer explicitly states that pg_dump separates UNIQUE constraints into ALTER
TABLE statements, satisfying the criterion."
 },
 {
 "id": "reasoning-48553",
 "tag": "reasoning",
 "weight": "critical",
 "description": "Explains the schema.rb parser sets column.unique through extractColumnOptions when
the column definition includes unique: true.",
 "rationale": "Understanding WHY schema.rb works is half of explaining the discrepancy.",
 "dependent_on": [],
 "pass": true,
 "evidence": "\u201cIn the schema.rb parser
(`frontend/packages/db-structure/src/parser/schemarb/parser.ts`), `extractColumnDetails` delegates to

```

``extractColumnOptions``, which reads keyword arguments from the column definition\u2026 The ``case 'unique`` branch sets ``column.unique = value instanceof TrueNode``.\u201d",

"reasoning": "The answer explicitly describes that schema.rb\u2019s parser uses

``extractColumnOptions`` to read ``unique: true`` from the column definition and set ``column.unique``, satisfying the requirement to explain how schema.rb handles the attribute."

},

{

"id": "reasoning-48559",

"tag": "reasoning",

"weight": "bonus",

"description": "Notes `constraintToRelationship` checks `contype` against `CONSTR_FOREIGN` and returns undefined for all other constraint types.",

"rationale": "Traces the exact code that discards UNIQUE constraints.",

"dependent\_on": [],

"pass": true,

"evidence": "\u201chandleAlterTableStmt \u2026 processes ``AT_AddConstraint`` commands by calling ``constraintToRelationship``. But that function returns early for anything except foreign keys:\n\n```\ntypescript\nif (constraint.contype !== 'CONSTR\_FOREIGN') {\n return\n ok(undefined)\n}\n```\u201d",

"reasoning": "The golden answer explicitly states that ``constraintToRelationship`` checks whether ``constraint.contype`` equals ``CONSTR_FOREIGN`` and otherwise returns undefined, showing how UNIQUE constraints are discarded."

},

{

"id": "reasoning-48554",

"tag": "reasoning",

"weight": "critical",

"description": "Explains the SQL parser only sets `column.unique` from inline `CONSTR_UNIQUE` in `ColumnDef` elements within `handleCreateStmt`.",

"rationale": "Understanding what the SQL parser DOES handle narrows the gap.",

"dependent\_on": [],

"pass": true,

"evidence": "\u201cThe PostgreSQL SQL parser ... handles column uniqueness inside ``handleCreateStmt``. It checks each ``ColumnDef`` element for inline ``CONSTR_UNIQUE`` or ``CONSTR_PRIMARY`` constraints\u201d",

"reasoning": "The answer clearly states that the SQL parser sets `column.unique` only when `handleCreateStmt` sees inline `CONSTR\_UNIQUE`/`CONSTR\_PRIMARY` constraints in `ColumnDef` elements, satisfying the criterion."

},

{

"id": "completeness-48573",

"tag": "completeness",

"weight": "bonus",

"description": "Locates the SQL parser test file at  
frontend/packages/db-structure/src/parser/sql/postgresql/index.test.ts.",

"rationale": "File path grounding.",

"dependent\_on": [],

"pass": true,

"evidence": "The `foreign key (one-to-one)` test in  
`frontend/packages/db-structure/src/parser/sql/postgresql/index.test.ts` declares:",

"reasoning": "The golden answer explicitly cites the SQL parser test file path, satisfying the grounding  
criterion."

},

{

"id": "completeness-48578",

"tag": "completeness",

"weight": "penalty",

"description": "Claims inline UNIQUE in CREATE TABLE SQL is also broken.",

"rationale": "Inline column-level UNIQUE is the one path the SQL parser handles correctly.",

"dependent\_on": [],

"pass": true,

"evidence": "\u201cThis works for `CREATE TABLE posts (user\_id INT UNIQUE)` where the UNIQUE  
constraint is part of the column definition.\u201d",

"reasoning": "The golden answer explicitly states that inline column-level UNIQUE constraints work, so  
it does not claim they are broken."

},

{

"id": "completeness-48577",

"tag": "completeness",

"weight": "penalty",

"description": "Claims the schema.rb parser also fails to detect column uniqueness from inline unique: true.",

"rationale": "The schema.rb parser handles this correctly.",

"dependent\_on": [],

"pass": true,

"evidence": "\u201cThe schema.rb parser \u2026 reads keyword arguments \u2026 When a column is declared with `unique: true` \u2026 The `case 'unique` branch sets `column.unique = value instanceof TrueNode`. \u2026 Rails' `schema.rb` format always puts column attributes inline \u2026 so `unique: true` appears right where the parser expects it.\u201d",

"reasoning": "The answer explicitly states that the schema.rb parser correctly handles inline `unique: true` attributes, so it does not claim a failure; thus the penalty criterion is satisfied."

},

{

"id": "completeness-48568",

"tag": "completeness",

"weight": "critical",

"description": "Locates handleOneToOneRelationships at frontend/packages/db-structure/src/parser/utils/handleOneToOneRelationships.ts.",

"rationale": "File path grounds the shared utility.",

"dependent\_on": [],

"pass": true,

"evidence": "\u201cBoth the schema.rb and PostgreSQL SQL parsers call the same `handleOneToOneRelationships` function from `frontend/packages/db-structure/src/parser/utils/handleOneToOneRelationships.ts`. \u201d",

"reasoning": "The golden answer explicitly states the function name and its exact file path within the repo, satisfying the requirement to locate it."

},

{

"id": "completeness-48574",

"tag": "completeness",

"weight": "bonus",

"description": "Provides concrete SQL examples showing the inline pattern that works versus the ALTER TABLE pattern that fails.",

"rationale": "Examples make the gap tangible.",

"dependent\_on": [],

"pass": true,



"evidence": "Works: \"This works for `CREATE TABLE posts (user\_id INT UNIQUE)` where the UNIQUE constraint is part of the column definition.\" Fails: \"But `pg\_dump` does not emit uniqueness this way... `ALTER TABLE ONLY` posts ADD CONSTRAINT posts\_user\_id\_key UNIQUE (user\_id);\"",

"reasoning": "The golden answer explicitly contrasts an inline `CREATE TABLE ... UNIQUE` example that succeeds with an `ALTER TABLE ... ADD CONSTRAINT ... UNIQUE` example that fails, satisfying the criterion for concrete SQL examples showing the working and failing patterns."

},

{

"id": "completeness-48571",

"tag": "completeness",

"weight": "critical",

"description": "Notes no SQL parser test exercises the scenario where uniqueness is added via ALTER TABLE rather than inline on the column.",

"rationale": "The prompt explicitly asks why the test suite doesn't catch this.",

"dependent\_on": [],

"pass": true,

"evidence": "\u201cThe SQL parser's one-to-one test \u2026 declares \u2026 UNIQUE is inline on the column definition\u2026 The ALTER TABLE variant of the one-to-one test also puts UNIQUE inline in the CREATE TABLE and only uses ALTER TABLE for the foreign key itself.\u201d",

"reasoning": "The answer explicitly explains that both SQL parser tests keep UNIQUE inline and do not cover the ALTER TABLE-added uniqueness scenario, fulfilling the requirement."

},

{

"id": "completeness-48572",

"tag": "completeness",

"weight": "critical",

"description": "Notes the FIXME comment and skip'd test in the SQL parser test file about CONSTRAINT statement support.",

"rationale": "Documentary evidence the parser creators knew about the limitation.",

"dependent\_on": [],

"pass": true,

"evidence": "The test file also contains a skip'd test with a FIXME comment (`// FIXME: 'CONSTRAINT' statement is not supported yet`) for table-level CONSTRAINT syntax inside CREATE TABLE.",

"reasoning": "The golden answer explicitly notes the skip'd test and its FIXME comment in the SQL parser test file about CONSTRAINT statement support, satisfying the criterion."

},

```

{
 "id": "completeness-48567",
 "tag": "completeness",
 "weight": "critical",
 "description": "Locates the schema.rb parser at
frontend/packages/db-structure/src/parser/schemarb/parser.ts.",
 "rationale": "File path grounds the schema.rb parser analysis.",
 "dependent_on": [],
 "pass": true,
 "evidence": "In the schema.rb parser
(`frontend/packages/db-structure/src/parser/schemarb/parser.ts`), `extractColumnDetails` delegates to
`extractColumnOptions`...",
 "reasoning": "The golden answer explicitly identifies the schema.rb parser's location at the required file
path."
},
{
 "id": "completeness-48570",
 "tag": "completeness",
 "weight": "critical",
 "description": "Identifies constraintToRelationship as the SQL converter function that filters out non-FK
constraints.",
 "rationale": "Names the specific function where UNIQUE constraints are discarded.",
 "dependent_on": [],
 "pass": true,
 "evidence": "The `ALTER TABLE` statement goes through `handleAlterTableStmt`, which processes
`AT_AddConstraint` commands by calling `constraintToRelationship`. But that function returns early for
anything except foreign keys: `if (constraint.contype !== 'CONSTR_FOREIGN') { return ok(undefined) }`.",
 "reasoning": "The golden answer explicitly identifies `constraintToRelationship` as the function invoked
by the SQL parser for ALTER TABLE constraints and explains that it filters out non-foreign-key constraints,
causing UNIQUE constraints to be discarded."
},
{
 "id": "completeness-48566",
 "tag": "completeness",
 "weight": "critical",

```

```

 "description": "Locates the SQL converter at
frontend/packages/db-structure/src/parser/sql/postgresql/converter.ts.",
 "rationale": "File path grounds the SQL parser analysis.",
 "dependent_on": [],
 "pass": true,
 "evidence": "\u201cThe PostgreSQL SQL parser
(`frontend/packages/db-structure/src/parser/sql/postgresql/converter.ts`) handles column
uniqueness\u2026\u201d",
 "reasoning": "The golden answer explicitly identifies the SQL converter file path, grounding its analysis
in the required location."
 },
 {
 "id": "completeness-48576",
 "tag": "completeness",
 "weight": "bonus",
 "description": "Notes handleIndexStmt stores index data in table.indexes without updating
column.unique.",
 "rationale": "Clarifies index data exists but is disconnected from column metadata.",
 "dependent_on": [],
 "pass": true,
 "evidence": "\u201cCREATE UNIQUE INDEX statements are captured by `handleIndexStmt` as Index
objects with `unique: true`, but the index uniqueness is never propagated to the column's `unique`
field.\u201d",
 "reasoning": "The golden answer explicitly states that `handleIndexStmt` records index uniqueness
separately without updating `column.unique`, directly satisfying the criterion."
 },
 {
 "id": "completeness-48569",
 "tag": "completeness",
 "weight": "critical",
 "description": "Identifies extractColumnOptions as the schema.rb parser function that reads the unique
keyword from column definitions.",
 "rationale": "Names the specific function in the schema.rb parser responsible for the working path.",
 "dependent_on": [],
 "pass": true,

```

```

 "evidence": "\u201cIn the schema.rb parser
(frontend/packages/db-structure/src/parser/schemarb/parser.ts`), `extractColumnDetails` delegates to
`extractColumnOptions`, which reads keyword arguments from the column definition \u2026 The `case
'unique` branch sets `column.unique = value instanceof TrueNode`.\u201d",
 "reasoning": "The golden answer explicitly names `extractColumnOptions` as the schema.rb parser
function that processes column options and handles the `unique` keyword, satisfying the criterion."
 },
 {
 "id": "completeness-48575",
 "tag": "completeness",
 "weight": "bonus",
 "description": "Notes table-level UNIQUE constraints in CREATE TABLE are also skipped because
handleCreateStmt only processes ColumnDef elements.",
 "rationale": "A third distinct gap.",
 "dependent_on": [],
 "pass": true,
 "evidence": "\u201cThe same gap exists for two other SQL patterns. Table-level UNIQUE constraints
inside CREATE TABLE (`UNIQUE (user_id)` as a separate clause) are skipped because `handleCreateStmt`
only processes elements where `ColumnDef` in elt`.\u201d",
 "reasoning": "The golden answer explicitly states that table-level UNIQUE constraints in CREATE TABLE
are skipped and explains that this happens because handleCreateStmt only processes ColumnDef
elements, meeting the criterion."
 },
 {
 "id": "style-48581",
 "tag": "style",
 "weight": "bonus",
 "description": "Avoids proposing a complete code patch, staying focused on diagnosis.",
 "rationale": "RCA intent is diagnosis, not implementation.",
 "dependent_on": [],
 "pass": true,
 "evidence": "The answer describes existing code behavior and shortcomings without proposing any
new code changes or patch-style solutions.",
 "reasoning": "It stays focused on diagnosis, explaining how current parsers work and where they fail,
without suggesting implementation steps or code modifications."
 },
}

```

```

{
 "id": "style-48580",
 "tag": "style",
 "weight": "bonus",
 "description": "References specific file paths, function names, and code patterns from the repository.",
 "rationale": "Repo-grounded answers are more verifiable.",
 "dependent_on": [],
 "pass": true,
 "evidence": "Mentions files like
`frontend/packages/db-structure/src/parser/utils/handleOneToOneRelationships.ts`,
`frontend/packages/db-structure/src/parser/schemarb/parser.ts`, and
`frontend/packages/db-structure/src/parser/sql/postgresql/converter.ts`, as well as functions
(`handleOneToOneRelationships`, `extractColumnDetails`, `handleCreateStmt`, etc.) and specific code
snippets from those locations.",
 "reasoning": "The answer references particular repository files, functions, and code snippets, showing
how unique constraints are parsed differently in schema.rb versus SQL parsing, fulfilling the
repo-grounded criterion."
},
{
 "id": "style-48579",
 "tag": "style",
 "weight": "bonus",
 "description": "Traces both parsers systematically, comparing their uniqueness handling paths.",
 "rationale": "A cross-parser comparison is clearer when both sides are traced explicitly.",
 "dependent_on": [],
 "pass": true,
 "evidence": "\u201cHow the schema.rb parser sets column.unique\u2026 Rails\u2019 schema.rb
parser reads these inline attributes through extractColumnOptions\u2026; How the SQL parser fails to set
column.unique\u2026 The PostgreSQL SQL parser\u2026 handles column uniqueness inside
handleCreateStmt\u2026 The UNIQUE constraint is dropped. `column.unique` stays `false`.\u201d",
 "reasoning": "The answer explicitly walks through both parsers\u2019 code paths, detailing how
schema.rb sets `column.unique` while the SQL parser misses it, thereby systematically comparing their
uniqueness handling."
}
],
"passed": 30,

```

```
"failed": 0,
"total": 30,
"score_percent": 100.0,
"summary": "Passed 30/30 criteria (100.0%)"
}
```

## Test Model

### Model Response

```
=====
RUN 1/3
```

```
=====

In the PostgreSQL SQL parser (see
`frontend/packages/db-structure/src/parser/sql/postgresql/converter.ts`), the only place a column's
`unique` flag is ever set is inside `handleCreateStmt`. When a `ColumnDef` node is processed, the code
looks only at that column's inline constraints:
```

```
```ts
columns[colDef.colname] = {
  ...
  unique:
    colDef.constraints
      ?.filter(isConstraintNode)
      .some((c) =>
        ['CONSTR_UNIQUE', 'CONSTR_PRIMARY'].includes(
          c.Constraint.contype ?? "",
        ),
      ) || false,
  ...
}
```
```

(lines 178-186)

So a column ends up marked `unique: true`` **only if** its own `ColumnDef`` contains `UNIQUE`` or `PRIMARY KEY`` directly in the column definition (mention `TEXT UNIQUE``, `user_id INT PRIMARY KEY``, etc.). That's why the parser's "one-to-one" test (which uses the inline form `user_id INT REFERENCES users(id) UNIQUE``; see `index.test.ts`` around lines 183-194) passes—the inline constraint is converted into `colDef.constraints`` and triggers the check above.

Any `UNIQUE`` constraint that appears outside the column definition is ignored for column metadata:

- \* Table-level constraints such as `CONSTRAINT users_email_key UNIQUE (email)`` or `UNIQUE (email)`` are emitted by `pg_dump``, but they show up as separate `Constraint`` entries in the table element list, not inside `ColumnDef``, and there is no code that maps them back to the referenced column.

- \* Likewise, `CREATE UNIQUE INDEX ...`` statements are parsed by `handleIndexStmt`` (lines 227-265) to populate the table's `indexes``, but that code never updates `columns[col].unique``.

Because `handleOneToOneRelationships`` (lines 3-15 in `parser/utils/handleOneToOneRelationships.ts``) decides whether a relationship is `ONE_TO_ONE`` by checking `foreignColumn?.unique``, only relationships whose foreign key column had an inline `UNIQUE`/PRIMARY KEY`` constraint will switch to one-to-one. Everything driven by table-level `UNIQUE`` constraints (the usual `pg_dump`` output) will keep `unique: false`` on the column and therefore stay `ONE_TO_MANY``.

So, in short: the parser currently "picks up" only inline column constraints (`colname ... UNIQUE/PRIMARY KEY``). `UNIQUE`` constraints expressed via table constraints or indexes are not reflected in the column metadata, which is why your `pg_dump`` output isn't affecting the one-to-one detection.

=====

RUN 2/3

=====

In the PostgreSQL SQL parser (`frontend/packages/db-structure/src/parser/sql/postgresql/converter.ts``) the only place where the `Column.unique`` flag is set is inside the per-column loop of `handleCreateStmt``. For each `ColumnDef``, it looks at the inline constraints that belong to that column and sets:

```
```ts
unique: colDef.constraints
  ?.filter(isConstraintNode)
  .some((c) =>
```

```
['CONSTR_UNIQUE', 'CONSTR_PRIMARY'].includes(c.Constraint.contype ?? ")  
) || false,  
...
```

(see roughly lines 150-190 of that file). In other words, a column ends up marked as ``unique: true`` only when the ORIGINAL table definition includes an inline constraint such as:

```
```sql  
mention TEXT UNIQUE
-- or
user_id INT REFERENCES users(id) UNIQUE
...
```

This is the pattern covered by the unit test “foreign key (one-to-one)” in ``frontend/packages/db-structure/src/parser/sql/postgresql/index.test.ts`` (around lines 180-210). Because the UNIQUE is inline, ``colDef.constraints`` contains ``CONSTR_UNIQUE``, so the ``unique`` flag is set and ``handleOneToOneRelationships`` later upgrades the relationship to ``ONE_TO_ONE``.

However, `pg_dump` normally emits UNIQUE constraints as table-level constraints or ALTER TABLE statements, for example:

```
```sql  
ALTER TABLE ONLY public.users  
ADD CONSTRAINT users_email_key UNIQUE (email);  
...
```

or as standalone ``CREATE UNIQUE INDEX ...`` commands. Those statements are parsed into either ``Constraint`` nodes that are siblings of the columns (not part of any ``ColumnDef``) or ``IndexStmt`` nodes. The current converter ignores both with respect to column metadata:

* Table-level ``Constraint`` nodes are never inspected when building ``columns``. Only ``ColumnDef`` entries are processed (see the ``for (const elt of createStmt.tableElts)`` loop).

* Unique ``IndexStmt``s are stored under ``table.indexes``, but nothing propagates their uniqueness back onto the participating columns.

The ALTER TABLE handler (`handleAlterTableStmt``) even filters exclusively for foreign key additions; it doesn't look at UNIQUE constraints at all.

Therefore the deciding factor is **how the UNIQUE constraint appears in the SQL**. Inline column constraints are picked up; anything expressed as a table-level constraint, an ALTER TABLE ADD CONSTRAINT, or a CREATE UNIQUE INDEX is currently ignored by the column metadata, so relationships whose uniqueness is enforced that way never see `foreignColumn.unique = true``.

To make pg_dump output work, you'd need to extend the converter to (a) process table-level `Constraint`` nodes of type `CONSTR_UNIQUE`/CONSTR_PRIMARY``, (b) process UNIQUE constraints added in ALTER TABLE commands, and/or (c) infer column uniqueness from unique indexes. Until then, only inline UNIQUE syntax will be reflected in the parser's column metadata.

```
=====
RUN 3/3
```

```
=====
The SQL parser decides whether a column is "unique" solely inside convertToSchema` when it walks each ColumnDef` inside a CREATE TABLE ...` statement (see frontend/packages/db-structure/src/parser/sql/postgresql/converter.ts`, lines 132-196). For every column it sets column.unique = true` only if one of that column's inline constraints has contype === 'CONSTR_UNIQUE'` (or CONSTR_PRIMARY`). No other place in the parser ever flips that flag.
```

What the parser does **not** do right now:

- * It never looks at table-level `Constraint`` nodes that appear alongside `ColumnDef`` entries in `CreateStmt.tableElts``. That means `UNIQUE (user_id)`` or `CONSTRAINT users_user_id_key UNIQUE (user_id)``—the pattern `pg_dump`` prefers—are ignored entirely, so the column metadata for `user_id`` keeps `unique: false``.

- * It never revisits columns when it later processes `CREATE UNIQUE INDEX ...`` or `ALTER TABLE ... ADD CONSTRAINT ... UNIQUE ...``. `handleIndexStmt`` (lines 227-265) just records the index object; it doesn't propagate uniqueness back to the column. And `handleAlterTableStmt`` (lines 327-360) only cares about foreign keys.

Because `handleOneToOneRelationships`` (`frontend/packages/db-structure/src/parser/utils/handleOneToOneRelationships.ts``, lines 4-13) upgrades a relationship to `ONE_TO_ONE`` **only when** the foreign column already has `unique: true``, the feature “works” in the unit tests (they all declare `user_id INT ... UNIQUE`` inline) but fails for `pg_dump``-style uniqueness (table constraints or separate indexes). In short, the parser picks up UNIQUE constraints only when they are written inline in the column definition inside `CREATE TABLE``; any UNIQUE declared elsewhere is currently ignored, which is why some of your `pg_dump` constraints never show up in the column metadata.

Evaluation JSON

```
{
  "points": 24.0,
  "maxPoints": 53.0,
  "score_percent_weight": 45,
  "summary_weight": "24.0/53.0 points (45%)",
  "eval_runs": 3,
  "eval_runs_succeeded": 3,
  "per_run_points": [
    23.0,
    24.0,
    25.0
  ],
  "results": [
    {
      "id": "reasoning-48560",
      "weight": "bonus",
      "pass": true,
      "points": 1.0
    },
    {
      "id": "reasoning-48557",
      "weight": "critical",
      "pass": true,
      "points": 3.0
    },
  ],
}
```

```
{
  "id": "reasoning-48558",
  "weight": "critical",
  "pass": true,
  "points": 3.0
},
{
  "id": "reasoning-48561",
  "weight": "critical",
  "pass": false,
  "points": 0.0
},
{
  "id": "reasoning-48562",
  "weight": "critical",
  "pass": false,
  "points": 0.0
},
{
  "id": "reasoning-48555",
  "weight": "critical",
  "pass": false,
  "points": 0.0
},
{
  "id": "reasoning-48565",
  "weight": "penalty",
  "pass": false,
  "points": 0.0
},
{
  "id": "reasoning-48563",
  "weight": "penalty",
  "pass": false,
  "points": 0.0
},
}
```

```
{
  "id": "reasoning-48564",
  "weight": "penalty",
  "pass": false,
  "points": 0.0
},
{
  "id": "reasoning-48552",
  "weight": "bonus",
  "pass": true,
  "points": 1.0
},
{
  "id": "reasoning-48556",
  "weight": "bonus",
  "pass": false,
  "points": 0.0
},
{
  "id": "reasoning-48553",
  "weight": "critical",
  "pass": false,
  "points": 0.0
},
{
  "id": "reasoning-48559",
  "weight": "bonus",
  "pass": false,
  "points": 0.0
},
{
  "id": "reasoning-48554",
  "weight": "critical",
  "pass": true,
  "points": 3.0
},
}
```

```
{
  "id": "completeness-48573",
  "weight": "bonus",
  "pass": true,
  "points": 1.0
},
{
  "id": "completeness-48578",
  "weight": "penalty",
  "pass": false,
  "points": 0.0
},
{
  "id": "completeness-48577",
  "weight": "penalty",
  "pass": false,
  "points": 0.0
},
{
  "id": "completeness-48568",
  "weight": "critical",
  "pass": true,
  "points": 3.0
},
{
  "id": "completeness-48574",
  "weight": "bonus",
  "pass": true,
  "points": 1.0
},
{
  "id": "completeness-48571",
  "weight": "critical",
  "pass": false,
  "points": 0.0
},
}
```

```
{
  "id": "completeness-48572",
  "weight": "critical",
  "pass": false,
  "points": 0.0
},
{
  "id": "completeness-48567",
  "weight": "critical",
  "pass": false,
  "points": 0.0
},
{
  "id": "completeness-48570",
  "weight": "critical",
  "pass": false,
  "points": 0.0
},
{
  "id": "completeness-48566",
  "weight": "critical",
  "pass": true,
  "points": 3.0
},
{
  "id": "completeness-48576",
  "weight": "bonus",
  "pass": true,
  "points": 1.0
},
{
  "id": "completeness-48569",
  "weight": "critical",
  "pass": false,
  "points": 0.0
},
}
```

```
{
  "id": "completeness-48575",
  "weight": "bonus",
  "pass": true,
  "points": 1.0
},
{
  "id": "style-48581",
  "weight": "bonus",
  "pass": true,
  "points": 1.0
},
{
  "id": "style-48580",
  "weight": "bonus",
  "pass": true,
  "points": 1.0
},
{
  "id": "style-48579",
  "weight": "bonus",
  "pass": false,
  "points": 0.0
}
}
```

model eval score 45.3

difficulty medium

2 Example Task 5680 PR Triage & Impact Assessment

Example Task 5680

Name: milvus-io-milvus-38876-pr
Intent: PR Triage & Impact Assessment
Difficulty: Medium
Language: Go

Repo

Repository: <https://github.com/milvus-io/milvus>

Checkout (commit): 76c66f07be6b04da93b5e722c8dc2fadd98daef6

Prompt

What is the change in pr.patch actually doing under the covers, beyond the headline "unblock previously-skipped Go client search tests"? Which parts of the milvus Go client test suite and the search-parameter API surface ripple from it? And what concrete issues should I flag to the author before I approve this for merge?

pr.patch (this exists on disk only; do not paste into prompt)

```
diff --git a/tests/go_client/testcases/groupby_search_test.go
b/tests/go_client/testcases/groupby_search_test.go
index 72b8a7b8bdc4b..7ad196d633242 100644
--- a/tests/go_client/testcases/groupby_search_test.go
+++ b/tests/go_client/testcases/groupby_search_test.go
@@ -90,7 +90,6 @@ func prepareDataForGroupBySearch(t *testing.T, loopInsert
int, insertNi int, idx
    // -> verify every top passage is the top of whole group
    // output_fields: pk + groupBy
    func TestSearchGroupByFloatDefault(t *testing.T) {
-        t.Skip("https://github.com/milvus-io/milvus/issues/38343")
        t.Parallel()
        for _, idx := range genGroupByVectorIndex(entity.L2) {
            // prepare data
@@ -148,7 +147,6 @@ func TestSearchGroupByFloatDefault(t *testing.T) {
    }

    func TestSearchGroupByFloatDefaultCosine(t *testing.T) {
-        t.Skip("https://github.com/milvus-io/milvus/issues/38343")
        t.Parallel()
        for _, idx := range genGroupByVectorIndex(entity.COSINE) {
            // prepare data
@@ -297,7 +295,6 @@ func TestSearchGroupByBinaryDefault(t *testing.T) {
```

```

// default Bounded ConsistencyLevel -> succ ??
// strong ConsistencyLevel -> error
func TestSearchGroupByBinaryGrowing(t *testing.T) {
-   t.Skip("https://github.com/milvus-io/milvus/issues/38343")
    t.Parallel()
    for _, metricType := range hp.SupportBinIvfFlatMetricType {
        idxBinIvfFlat := index.NewBinIvfFlatIndex(metricType, 128)
@@ -329,6 +326,7 @@ func TestSearchGroupByBinaryGrowing(t *testing.T) {

// groupBy in growing segments, maybe growing index or brute force
func TestSearchGroupByFloatGrowing(t *testing.T) {
+   t.Parallel()
    for _, metricType := range hp.SupportFloatMetricType {
        idxHnsw := index.NewHNSWIndex(metricType, 8, 96)
        mc, ctx, collName := prepareDataForGroupBySearch(t, 100, 200,
idxHnsw, true)
@@ -382,6 +380,7 @@ func TestSearchGroupByFloatGrowing(t *testing.T) {

// groupBy + pagination
func TestSearchGroupByPagination(t *testing.T) {
+   t.Parallel()
        // create index and load
        idx := index.NewHNSWIndex(entity.COSINE, 8, 96)
        mc, ctx, collName := prepareDataForGroupBySearch(t, 10, 1000, idx, false)
@@ -442,19 +441,16 @@ func TestSearchGroupByIterator(t *testing.T) {

// groupBy + range search -> not supported
func TestSearchGroupByRangeSearch(t *testing.T) {
-   t.Skipf("Waiting for support for specifying search parameters")
+   t.Skipf("https://github.com/milvus-io/milvus/issues/38846")
        idxHnsw := index.NewHNSWIndex(entity.COSINE, 8, 96)
        mc, ctx, collName := prepareDataForGroupBySearch(t, 1, 1000, idxHnsw,
true)

        // groupBy search with range
        queryVec := hp.GenSearchVectors(common.DefaultNq, common.DefaultDim,
entity.FieldTypeFloatVector)

-   // sp, _ := index.NewHNSWIndexSearchParam(50)
-   // sp.AddRadius(0)
-   // sp.AddRangeFilter(0.8)
-
        // range search
-   _, err := mc.Search(ctx, client.NewSearchOption(collName,
common.DefaultLimit,
queryVec).WithGroupByField(common.DefaultVarcharFieldName).WithANNSField(common
.DefaultFloatVecFieldName))

```

```

+     _, err := mc.Search(ctx, client.NewSearchOption(collName,
common.DefaultLimit,
queryVec).WithGroupByField(common.DefaultVarcharFieldName)).
+
WithANNSField(common.DefaultFloatVecFieldName).WithSearchParam("radius",
"0").WithSearchParam("range_filter", "0.8"))
    common.CheckErr(t, err, false, "Not allowed to do range-search when doing
search-group-by")
}

diff --git a/tests/go_client/testcases/query_test.go
b/tests/go_client/testcases/query_test.go
index ca863fb4d60e3..d28c5cad4082 100644
--- a/tests/go_client/testcases/query_test.go
+++ b/tests/go_client/testcases/query_test.go
@@ -165,7 +165,6 @@ func TestQueryWithoutExpr(t *testing.T) {
    // test query with part not existed field ["aa", "$meat"]: error or as dynamic
    field
    // test query with repeated field: ["*", "$meat"], ["floatVec", floatVec"]
    unique field
    func TestQueryOutputFields(t *testing.T) {
-        t.Skip("verify TODO")
        t.Parallel()
        ctx := hp.CreateContext(t, time.Second*common.DefaultTimeout)
        mc := createDefaultMilvusClient(ctx, t)
@@ -193,16 +192,17 @@ func TestQueryOutputFields(t *testing.T) {
        common.CheckErr(t, err1, false, "not exist")
    }

-        // query with not existed field -> output field as dynamic or error
+        // query with not existed field -> output empty data field as
    dynamic or error
        fakeName := "aaa"
        res2, err2 := mc.Query(ctx,
client.NewQueryOption(schema.CollectionName).WithConsistencyLevel(entity.ClStrong).WithFilter(expr).WithOutputFields(fakeName))
        if enableDynamic {
            common.CheckErr(t, err2, true)
            for _, c := range res2.Fields {
-                log.Debug("data", zap.String("name", c.Name()),
zap.Any("type", c.Type()), zap.Any("data", c.FieldData()))
+                log.Info("data", zap.String("name", c.Name()),
zap.Any("type", c.Type()), zap.Any("data", c.FieldData()))
            }
            common.CheckOutputFields(t,
[]string{common.DefaultInt64FieldName, fakeName}, res2.Fields)
-                dynamicColumn := hp.MergeColumnsToDynamic(10,
hp.GenDynamicColumnData(0, 10), common.DefaultDynamicFieldName)

```

```

+           dynamicColumn := hp.MergeColumnsToDynamic(10,
[]column.Column{}, common.DefaultDynamicFieldName)
+           // dynamicColumn := hp.MergeColumnsToDynamic(10,
hp.GenDynamicColumnData(0, 10), common.DefaultDynamicFieldName)
           expColumns := []column.Column{
               hp.GenColumnData(10, entity.FieldTypeInt64,
*hp.TNewDataOption()),
               column.NewColumnDynamic(dynamicColumn, fakeName),
diff --git a/tests/go_client/testcases/search_test.go
b/tests/go_client/testcases/search_test.go
index b84a5dda6cbe2..cb3e7373fb4ac 100644
--- a/tests/go_client/testcases/search_test.go
+++ b/tests/go_client/testcases/search_test.go
@@ -3,6 +3,7 @@ package testcases
import (
    "fmt"
    "math/rand"
+   "strconv"
    "testing"
    "time"

@@ -453,7 +454,6 @@ func TestSearchEmptyInvalidVectors(t *testing.T) {

    // test search metric type isn't the same with index metric type
    func TestSearchNotMatchMetricType(t *testing.T) {
-       t.Skip("Waiting for support for specifying search parameters")
        ctx := hp.CreateContext(t, time.Second*common.DefaultTimeout)
        mc := createDefaultMilvusClient(ctx, t)

@@ -465,7 +465,7 @@ func TestSearchNotMatchMetricType(t *testing.T) {
        prepare.Load(ctx, t, mc, hp.NewLoadParams(schema.CollectionName))

        vectors := hp.GenSearchVectors(1, common.DefaultDim,
entity.FieldTypeFloatVector)
-       _, errSearchEmpty := mc.Search(ctx,
client.NewSearchOption(schema.CollectionName, common.DefaultLimit, vectors))
+       _, errSearchEmpty := mc.Search(ctx,
client.NewSearchOption(schema.CollectionName, common.DefaultLimit,
vectors).WithSearchParam("metric_type", "L2"))
        common.CheckErr(t, errSearchEmpty, false, "metric type not match: invalid
parameter")
    }

@@ -507,12 +507,6 @@ func TestSearchInvalidOffset(t *testing.T) {

    // test search with invalid search params
    func TestSearchInvalidSearchParams(t *testing.T) {
-       t.Skip("Waiting for support for specifying search parameters")
-   }

```

```

-
-// search with index hnsw search param ef < topK -> error
-func TestSearchEfHnsw(t *testing.T) {
-    t.Skip("Waiting for support for specifying search parameters")
-    ctx := hp.CreateContext(t, time.Second*common.DefaultTimeout)
-    mc := createDefaultMilvusClient(ctx, t)

@@ -524,18 +518,29 @@ func TestSearchEfHnsw(t *testing.T) {
    prepare.Load(ctx, t, mc, hp.NewLoadParams(schema.CollectionName))

    vectors := hp.GenSearchVectors(1, common.DefaultDim,
entity.FieldTypeFloatVector)
-    _, err := mc.Search(ctx, client.NewSearchOption(schema.CollectionName,
common.DefaultLimit, vectors))
-    common.CheckErr(t, err, false, "ef(7) should be larger than k(10)")
-}

-// test search params mismatch index type, hnsw index and ivf sq8 search param
-> search with default hnsw params, ef=topK
-func TestSearchSearchParamsMismatchIndex(t *testing.T) {
-    t.Skip("Waiting for support for specifying search parameters")
+    // search with invalid hnsw ef
+    invalidEfs := []int{-1, 0, 32769}
+    for _, invalidEf := range invalidEfs {
+        _, errHnsw := mc.Search(ctx,
client.NewSearchOption(schema.CollectionName, common.DefaultLimit,
vectors).WithSearchParam("ef", strconv.Itoa(invalidEf)))
+        common.CheckErr(t, errHnsw, true, "No error for invalid search
params")
+    }
+
+    // test search params mismatch index type, hnsw index and ivf sq8 search
param -> search with default hnsw params, ef=topK
+    invalidNprobes := []int{-1, 0, 65537}
+    for _, invalidNprobe := range invalidNprobes {
+        _, errHnsw := mc.Search(ctx,
client.NewSearchOption(schema.CollectionName, common.DefaultLimit,
vectors).WithSearchParam("nprobe", strconv.Itoa(invalidNprobe)))
+        common.CheckErr(t, errHnsw, true, "No error for invalid search
params")
+    }
+
+    // search with index hnsw search param ef < topK -> error
+    res, err := mc.Search(ctx, client.NewSearchOption(schema.CollectionName,
common.DefaultLimit, vectors).WithSearchParam("ef", "7"))
+    common.CheckErr(t, err, true, "ef(7) should be larger than k(10), but no
error")
+    common.CheckSearchResult(t, res, 1, common.DefaultLimit)
+}

```

```

// search with index scann search param ef < topK -> error
func TestSearchInvalidScannReorderK(t *testing.T) {
-   t.Skip("Waiting for support for specifying search parameters")
    ctx := hp.CreateContext(t, time.Second*common.DefaultTimeout)
    mc := createDefaultMilvusClient(ctx, t)

@@ -548,36 +553,49 @@ func TestSearchInvalidScannReorderK(t *testing.T) {
    prepare.Load(ctx, t, mc, hp.NewLoadParams(schema.CollectionName))

    // search with invalid reorder_k < topK
+   vectors := hp.GenSearchVectors(1, common.DefaultDim,
entity.FieldTypeFloatVector)
+
+   // search with invalid hnsw ef
+   _, errScann := mc.Search(ctx,
client.NewSearchOption(schema.CollectionName, common.DefaultLimit, vectors).
+       WithSearchParam("nprobe", "8").WithSearchParam("reorder_k",
strconv.Itoa(common.DefaultLimit-1)))
+   common.CheckErr(t, errScann, true, "No error for invalid search params")

    // valid scann index search reorder_k
+   res, err := mc.Search(ctx, client.NewSearchOption(schema.CollectionName,
common.DefaultLimit, vectors).
+       WithSearchParam("nprobe", "8").WithSearchParam("reorder_k", "20"))
+   common.CheckErr(t, err, true)
+   common.CheckSearchResult(t, res, 1, common.DefaultLimit)
}

// test search with scann index params: with_raw_data and metrics_type [L2,
IP, COSINE]
func TestSearchScannAllMetricsWithRawData(t *testing.T) {
-   t.Skip("Waiting for support scann index params withRawData")
    t.Parallel()
-   /*for _, withRawData := range []bool{true, false} {
+   for _, withRawData := range []bool{true, false} {
        for _, metricType := range []entity.MetricType{entity.L2,
entity.IP, entity.COSINE} {
            ctx := hp.CreateContext(t, time.Second*common.DefaultTimeout)
            mc := createDefaultMilvusClient(ctx, t)

-           prepare, schema := hp.CollPrepare.CreateCollection(ctx, t,
mc, hp.NewCreateCollectionParams(hp.Int64VecJSON), hp.TNewFieldsOption(),
hp.TNewSchemaOption())
-           prepare.InsertData(ctx, t, mc, hp.NewInsertParams(schema,
500), hp.TNewDataOption())
+           prepare, schema := hp.CollPrepare.CreateCollection(ctx, t,
mc, hp.NewCreateCollectionParams(hp.Int64VecJSON),

```

```

+             hp.TNewFieldsOption(),
hp.TNewSchemaOption().TWithEnableDynamicField(true))
+             prepare.InsertData(ctx, t, mc, hp.NewInsertParams(schema),
hp.TNewDataOption())
+             prepare.FlushData(ctx, t, mc, schema.CollectionName)
+             prepare.CreateIndex(ctx, t, mc,
hp.TNewIndexParams(schema).TWithFieldIndex(map[string]index.Index{
-             common.DefaultFloatVecFieldName:
index.NewSCANNIndex(entity.COSINE, 16),
+             common.DefaultFloatVecFieldName:
index.NewSCANNIndex(metricType, 16, withRawData),
+             }))
+             prepare.Load(ctx, t, mc,
hp.NewLoadParams(schema.CollectionName))

+             // search and output all fields
-             vectors := hp.GenSearchVectors(1, common.DefaultDim,
entity.FieldTypeFloatVector)
-             resSearch, errSearch := mc.Search(ctx,
client.NewSearchOption(schema.CollectionName, common.DefaultLimit,
vectors).WithConsistencyLevel(entity.ClStrong).WithOutputFields([]string{"*"}))
+             vectors := hp.GenSearchVectors(common.DefaultNq,
common.DefaultDim, entity.FieldTypeFloatVector)
+             resSearch, errSearch := mc.Search(ctx,
client.NewSearchOption(schema.CollectionName, common.DefaultLimit, vectors).
+             WithConsistencyLevel(entity.ClStrong).WithOutputFields("*"))
+             common.CheckErr(t, errSearch, true)
-             common.CheckOutputFields(t,
[]string{common.DefaultInt64FieldName, common.DefaultFloatFieldName,
-             common.DefaultJSONFieldName,
common.DefaultFloatVecFieldName, common.DefaultDynamicFieldName},
resSearch[0].Fields)
-             common.CheckSearchResult(t, resSearch, 1,
common.DefaultLimit)
+             common.CheckOutputFields(t, []string{
+             common.DefaultInt64FieldName,
common.DefaultJSONFieldName,
+             common.DefaultFloatVecFieldName,
common.DefaultDynamicFieldName,
+             }, resSearch[0].Fields)
+             common.CheckSearchResult(t, resSearch, common.DefaultNq,
common.DefaultLimit)
+             }
-             }*/
+             }
+             }

+             // test search with valid expression

```

```

@@ -1061,7 +1079,7 @@ func TestSearchSparseVectorNotSupported(t *testing.T) {
}

func TestRangeSearchSparseVector(t *testing.T) {
-   t.Skip("Waiting for support range search")
+   t.Skipf("https://github.com/milvus-io/milvus/issues/38846")
   ctx := hp.CreateContext(t, time.Second*common.DefaultTimeout*2)
   mc := createDefaultMilvusClient(ctx, t)

@@ -1072,5 +1090,30 @@ func TestRangeSearchSparseVector(t *testing.T) {
   prepare.InsertData(ctx, t, mc, hp.NewInsertParams(schema),
hp.TNewDataOption().TWithSparseMaxLen(128))
   prepare.FlushData(ctx, t, mc, schema.CollectionName)

-   // TODO range search
+   // range search
+   queryVec := hp.GenSearchVectors(common.DefaultNq, common.DefaultDim,
entity.FieldTypeSparseVector)
+
+   resRange, errSearch := mc.Search(ctx,
client.NewSearchOption(schema.CollectionName, common.DefaultLimit,
queryVec).WithSearchParam("drop_ratio_search", "0.2"))
+   common.CheckErr(t, errSearch, true)
+   require.Len(t, resRange, common.DefaultNq)
+   for _, res := range resRange {
+       log.Info("default search", zap.Any("score", res.Scores))
+   }
+
+   radius := 10
+   rangeFilter := 30
+   resRange, errSearch = mc.Search(ctx,
client.NewSearchOption(schema.CollectionName, common.DefaultLimit, queryVec).
+       WithSearchParam("drop_ratio_search",
"0.2").WithSearchParam("radius",
strconv.Itoa(radius)).WithSearchParam("range_filter",
strconv.Itoa(rangeFilter)))
+   common.CheckErr(t, errSearch, true)
+   common.CheckErr(t, errSearch, true)
+   require.Len(t, resRange, common.DefaultNq)
+   for _, res := range resRange {
+       log.Info("range search", zap.Any("score", res.Scores))
+   }
+   for _, res := range resRange {
+       for _, s := range res.Scores {
+           require.GreaterOrEqual(t, s, float32(10))
+           require.Less(t, s, float32(30))
+       }
+   }
}

```


LLM Prompt Review

Prompt QC Feedback

****Grading Item 1 (1/1)****

The question requests an in-depth explanation of a patch's effect, indicating a non-trivial analysis.

****Grading Item 2 (1/1)****

It reads as a realistic peer-to-peer developer question without role-play language.

****Grading Item 3 (1/1)****

It avoids asking for elementary definitions or generic explanations.

****Grading Item 4 (1/1)****

Understanding the change, its ripple effects, and issues requires substantial repo exploration.

****Grading Item 5 (1/1)****

The prompt is purely interrogative and contains no inline repo content.

****Grading Item 6 (1/1)****

The question is specific and clearly grounded in repo context with verifiable answers.

****Grading Item 7 (1/1)****

It aligns with the task intent of analyzing repository changes.

****Grading Item 8 (1/1)****

It demands reasoning about system behavior, impacts, and potential issues.

****Grading Item 9 (1/1)****

No meta-instructions or format directives appear.

****Grading Item 10 (1/1)****

All characters are ASCII.

FINAL_SCORE: 10

Rubric

ID	tag	description	weight	rationale	depende nt_on
69396	reasoning	Identifies that three previously-skipped tests in	critical	The un-skipping is the headline activation in the GroupBy file.	

		tests/go_client/testcases/groupby_search_test.go lose their issue-38343 t.Skip guards (TestSearchGroupByFloatDefault, TestSearchGroupByFloatDefaultCosine, TestSearchGroupByBinaryGrowing).		Anchoring the triage on the three named tests is how the reader connects the issue-38343 reference to the concrete CI coverage that just turned on.	
69397	reasoning	Identifies that TestSearchSearchParamsMismatchIndex is deleted from search_test.go.	critical	TestSearchSearchParamsMismatchIndex was a t.Skip-only stub at the base commit (body == one t.Skip line). Its deletion is half of the consolidation that reduces three function names to one. Naming this deletion is what bounds the consolidation claim on the smaller of the two donors.	
69398	reasoning	Identifies that TestSearchEfHnsw is deleted from search_test.go.	critical	TestSearchEfHnsw at the base commit had a full body of setup plus a search asserting CheckErr(t, err, false, 'ef(7) should be larger than k(10)'), all gated behind a leading t.Skip. Naming the deletion (separately from the merge into TestSearchInvalidSearchParams) is what lets the reviewer track which donor function carried the original error-direction assertion that gets inverted post-merge.	
69399	reasoning	Identifies that the new TestSearchInvalidSearchParams body absorbs the setup scaffolding (ctx, mc, prepare, schema, prepare.Load) plus search loops from the deleted TestSearchEfHnsw.	critical	The merge is what makes the consolidation coherent. Without naming the merge a reader sees only function-name deletions, missing the through-line that the new TestSearchInvalidSearchParams body absorbs the donor's work. The previous TestSearchInvalidSearchParams body contained only a t.Skip line.	69398
69400	reasoning	Identifies that the new TestSearchInvalidSearchParams asserts success for invalid ef values in {-1, 0,	critical	The CheckErr helper at response_checker.go:19-41 treats the second argument as	

		32769} via CheckErr called with expErrNil=true.		expErrNil. When expErrNil is true the helper calls require.NoError. The third argument is then an unused message. The new body issues searches with invalidEfs {-1, 0, 32769} where each call asserts no error. The test name promises invalid-param coverage. The assertion direction documents silent acceptance. Without surfacing this contradiction a reviewer ships an assertion direction that points the wrong way.	
69401	reasoning	Identifies that the new TestSearchInvalidSearchParams asserts success for invalid nprobe values in {-1, 0, 65537} via CheckErr called with expErrNil=true.	critical	The nprobe loop is the second of two assertion-direction inversions in the same body. Catching only the ef loop misses half of the silent-acceptance pattern. The nprobe parameter exercises a different index family than ef, so it is not redundant evidence of the same call path.	
69402	reasoning	Identifies that the ef=7 case in TestSearchInvalidSearchParams asserts success even though DefaultLimit=10 (the historical validation rule required ef > k).	critical	The pre-PR equivalent assertion was CheckErr(t, err, false, 'ef(7) should be larger than k(10)') which expected the error. The post-PR assertion flips to expErrNil=true with message 'ef(7) should be larger than k(10), but no error', so the message documents the behaviour change in plain prose. A reviewer who reads only the literal CheckErr call without unpacking the helper semantics misses the inverted assertion entirely.	69400
69403	reasoning	States that the PR makes no changes to production code under client/, internal/, or pkg/.	critical	Anchoring the triage in the test-only scope bounds the severity ceiling. A reviewer who imagines production-side risk inflates the severity, asking for review of code that does not	

				move in this patch.	
69404	reasoning	Identifies the migration to the new WithSearchParam(key, value) builder API on searchOption defined in client/milvusclient/read_options.go.	critical	The reactivated and new tests reach the production search-param wire format through .WithSearchParam(...) calls (radius, range_filter, metric_type, ef, nprobe, drop_ratio_search, reorder_k). Naming the builder is what connects the un-skip pattern to the actual API unlock the PR exercises for the first time in this acceptance path.	
69405	reasoning	Identifies that TestSearchGroupByRangeSearch is re-skipped under issue 38846 (its t.Skip message rotates from 'Waiting for support for specifying search parameters' to a #38846 reference).	critical	The re-skip pattern is easy to miss because the t.Skip line is preserved. A reviewer who reads only the skip line treats this test as inactive without noticing that the issue reference has rotated from a generic message to a specific tracking issue.	
69406	reasoning	Identifies that TestRangeSearchSparseVector is re-skipped under issue 38846 (its t.Skip message rotates from 'Waiting for support range search' to a #38846 reference).	critical	The same re-skip pattern lands a second time in search_test.go. Catching only the GroupBy variant misses the parallel rotation in the sparse-vector test. Both re-skips share the #38846 follow-up but live in different test files, so they are independently observable.	
69407	reasoning	Identifies that the body of TestSearchGroupByRangeSearch is migrated to the new WithSearchParam API.	critical	The body-migration-under-skip pattern is what would surprise a future contributor who closes #38846 expecting an empty body, instead landing a fully-implemented assertion on first un-skip. Without naming the migrated body a reviewer treats the re-skip as a no-op rather than as staged work. The migrated call replaces a commented-out sp.AddRadius/sp.AddRangeFilter with	

				.WithSearchParam('radius', '0').WithSearchParam('range_filter', '0.8').	
69408	reasoning	Identifies that the body of TestRangeSearchSparseVector contains a WithSearchParam-based range search asserting per-score bounds in [10, 30).	critical	The sparse-vector test ships the most substantive body-under-skip in this PR (per-score assertions inside a half-open interval). A future contributor who closes #38846 inherits live assertions on first un-skip. Catching this body separately from the GroupBy variant matters because the assertions check actual returned scores rather than only call success.	69406
69409	reasoning	Identifies the semantic flip in TestQueryOutputFields where MergeColumnsToDynamic switches from GenDynamicColumnData(0, 10) to an empty []column.Column{} slice.	critical	MergeColumnsToDynamic at data_helper.go:433 iterates the columns slice per row to assemble each row's dynamic JSON object. The empty slice yields empty per-row maps. The test moves from asserting a populated dynamic-field response (ten rows of generated data) to asserting empty dynamic data when the requested output field name does not exist. This is a spec change rather than a refactor, so it warrants a one-line note from the author.	
69410	reasoning	Notes the new three-argument NewSCANNIndex(metricType, 16, withRawData) wiring in TestSearchScannAllMetricsWithRawData.	bonus	The pre-PR call was index.NewSCANNIndex(entity.COSINE, 16) inside the skipped /* ... */ block. The current production signature at client/index/scann.go:44 takes three arguments. Naming the wire-up shows the reviewer understood why the test could only run after the skip lifted: the body's prior 2-arg call would not compile against current production.	
69411	reasoning	Notes the dead commented-out	bonus	Leaving a one-line comment	69409

		MergeColumnsToDynamic call left in TestQueryOutputFields immediately above the new live call.		that mirrors the previous behaviour adds no value, rotting quickly. A reviewer who flags the dead-code line is doing the lowest-cost housekeeping ask the PR would benefit from. The observation also pairs well with the wider concern that the semantic flip in this test needs a PR-description note.	
69412	reasoning	Notes the duplicate common.CheckErr(t, errSearch, true) line on consecutive lines inside TestRangeSearchSparseVector.	bonus	The duplicate is a clear copy-paste artefact. It runs twice harmlessly today but degrades readability, surfacing if someone changed one of the two without realising they had a sibling. A reviewer who catches the duplicate is reading the body of a re-skipped test rather than skipping past it on the t.Skip line.	
69413	reasoning	Notes that the 'but no error' suffix in the ef=7 CheckErr message wording documents an observed behaviour change.	bonus	The message text only renders when CheckErr is invoked with expErrNil=false where an error is present. Because expErrNil is true here, the message is a dead string. The author's wording ('ef(7) should be larger than k(10), but no error') is therefore documentation of a behaviour change the author observed, not an assertion of it. The smoking-gun observation distinguishes a careful read of the helper semantics from a surface read of the CheckErr call.	69402
69414	reasoning	Notes the log.Debug to log.Info change inside TestQueryOutputFields.	bonus	Promoting the per-row dynamic-field dump from debug to info level means CI runs will surface the row dump in standard log output going forward. Whether that is wanted is a one-line question from the author. A reviewer	

				who flags it is reading the diff for noise-level changes rather than only assertion changes.	
69415	reasoning	Names a specific path under client/, internal/, or pkg/ as modified by this PR.	penalty	The diff is restricted to tests/go_client/testcases/group by_search_test.go, query_test.go, search_test.go. Naming any non-test path as modified misreads the diff, routing the review toward production layers that do not move in this patch. The penalty fires on a fabricated specific surface rather than on a missing general scope claim, so it captures a positive misstatement instead of an omission.	
69416	reasoning	Cites a specific authentication, authorisation, RBAC, or untrusted-input handler as affected by the diff under review.	penalty	No such file is touched by the PR. Citing one fabricates a security implication the diff does not carry, inflating the severity of a test-only change. The penalty targets a positive false claim about a specific code path rather than an omitted security framing.	
69417	reasoning	Recommends reverting the un-skip changes as the appropriate fix for the inverted-assertion concern in TestSearchInvalidSearchParams.	penalty	The un-skips are correct work. Only the assertion direction inside the new body is the question. The right fix is to flip the second CheckErr argument from true to false if the intent is to assert rejection, or to rename the test if the intent is to document silent acceptance. Reverting throws away the consolidation, plus the migration to the new builder API.	
69418	reasoning	Recommends preserving the deleted stubs (TestSearchEfHnsw, TestSearchSearchParamsMismatchIndex) for git-blame continuity.	penalty	Skip-only stubs carry no executable coverage and keeping them blocks the consolidation that makes the merged TestSearchInvalidSearchPara	

				ms body the right review target.	
69419	reasoning	Recommends redesigning the CheckErr helper or the WithSearchParam API as the primary triage outcome.	penalty	A redesign-first answer skips the actual review job. The change is a concrete diff to approve or request-changes on, not a test-helper or builder-API exercise. Recommending a redesign leaves the inverted-assertion question unanswered.	
69420	reasoning	Recommends splitting the un-skip work from the consolidation refactor as a hard blocker.	penalty	Splitting at review time adds two PRs of overhead without producing a behaviour change. The consolidation is what makes the un-skips coherent. The right outcome is to ask the author to call out the consolidation in the PR description, not to demand a split.	
69421	completeness	Enumerates the three test files changed by the diff (tests/go_client/testcases/groupby_search_test.go, query_test.go, search_test.go).	critical	The mechanical surface of the PR spans three files under tests/go_client/testcases. A triage that does not enumerate the files is incomplete because downstream blast-radius/severity claims rest on knowing which surfaces moved.	
69422	completeness	Names the WithSearchParam builder API in client/milvusclient/read_options.go as a production surface reached by the reactivated tests but not modified by this PR.	critical	The reactivated tests reach the production search-param wire format through .WithSearchParam(...) calls. Naming the builder is what grounds the no-production-change claim against the actual API the PR exercises for the first time in this acceptance path.	
69423	completeness	Names NewSCANNIndex in client/index/scann.go as a production surface reached by the reactivated TestSearchScannAllMetricsWithRawData but not modified by this PR.	critical	The pre-PR test body called the two-argument form which would not compile against current production. Naming the constructor is what shows the reviewer understood why the test could only run after the	

				skip lifted, while the production signature itself is left untouched.	
69424	completeness	Concludes the PR introduces no classic security regression in authentication, authorisation, RBAC, untrusted-input parsing, or cryptographic boundaries.	critical	Test-only PRs invite the lazy reading that drops the security analysis entirely. Explicitly concluding no classic regression is what makes the security framing complete, distinguishing a thorough triage from one that omits the security frame.	
69425	completeness	Includes an alternative-fixes paragraph that explicitly preempts at least one wrong-fix temptation from the set {reverting the un-skip changes, preserving the deleted stubs, splitting the un-skip from the consolidation}.	critical	Preempting wrong fixes is what makes the recommendation actionable. Showing the reviewer thought through alternatives before settling on a recommendation distinguishes a triage from a bullet list of complaints.	
69428	style	Uses plain-text identifiers for function names in the response prose.	bonus	Plain-text function names help the reader visually parse references at a glance, avoiding the generated-artifact feel of backtick-wrapped or monospace-wrapped tokens that breaks reading flow inside a prose review.	
69429	style	Uses plain-text identifiers for type names in the response prose.	bonus	Plain-text type identifiers keep symbol formatting uniform across the response. Mixing backtick-wrapped types with plain-text functions creates visual inconsistency that distracts the reader from the analysis itself.	
69430	style	Uses double-quoted file paths in the response prose.	bonus	Double-quoted file paths visually separate path references from surrounding prose. Unquoted paths blur into ordinary words (especially short relative paths like cmd/ or pkg/), making the reader work harder to spot file references inside the analysis.	

LLM Rubric Review

Rubric QC Feedback

****Grading Item 1: The rubric is organized into exactly three top-level evaluation categories... (2/2)****

All three required categories—reasoning, completeness, and style—are present with no extras.

****Grading Item 2: The rubric contains a reasonable number of grading items... (2/2)****

Thirty-three grading items appropriately match the task complexity and cover the space without feeling sparse or bloated.

****Grading Item 3: Every grading item is atomic and self-contained... (2/2)****

Each item targets a single clearly defined expectation; none stack multiple unrelated checks together.

****Grading Item 4: Grading item descriptions are clear, specific, and measurable... (2/2)****

Descriptions cite precise tests, functions, or behaviors, making compliance straightforward to assess.

****Grading Item 5: Every grading item is derivable from the provided `golden\_answer` ... (2/2)****

All requirements trace back to facts or emphases explicitly present in the golden answer.

****Grading Item 6: The rubric is comprehensive relative to the `golden\_answer` ... (2/2)****

Major themes—test migrations, assertion inversions, API surfaces, and follow-up concerns—are all represented with appropriate weight.

****Grading Item 7: The rubric would award full marks to the provided `golden\_answer` ... (2/2)****

The golden answer satisfies every critical requirement and avoids all penalties, so it would score full points.

****Grading Item 8: The rubric is fair to valid alternative reasoning paths... (2/2)****

While detailed, the items allow any accurate repo-grounded explanation to earn credit; they do not demand a single phrasing.

****Grading Item 9: The `reasoning` category evaluates correctness of reasoning, technical judgment... (2/2)****

Reasoning items focus on interpreting code changes, their implications, and technical judgments rather than mere coverage.

****Grading Item 10: The `completeness` category evaluates whether the response addresses all major parts... (2/2)****

Completeness items ensure scope, touched files, API surfaces, and alternative-fix coverage, matching the task demands.

****Grading Item 11: The `style` category evaluates clarity, organization, readability... (2/2)****

Style items address presentation aspects—identifier formatting and path quoting—without mixing in technical correctness.

****Grading Item 12: Grading items are placed in appropriate categories... (2/2)****

Each requirement resides in the logical category (reasoning for analysis points, completeness for coverage, style for prose).

****Grading Item 13: The rubric includes the task-wide expectations... (2/2)****

Reasoning quality, completeness, and style expectations are all explicitly represented.

****Grading Item 14: When the task intent requires it, the rubric includes appropriate intent-specific grading items... (2/2)****

Intent-specific needs—like naming wrong-fix temptations—are captured, aligning with the triage task.

****Grading Item 15: Weights use only valid values... and are well calibrated... (2/2)****

All items use critical/bonus/penalty appropriately, reserving penalties for harmful behavior and criticals for must-haves.

****Grading Item 16: Penalty items target genuinely harmful or misleading behavior... (2/2)****

Each penalty addresses a specific incorrect or misleading recommendation rather than being a simple negation of a positive requirement.

****Grading Item 17: Dependency usage is logically sound... (2/2)****

Dependencies (e.g., needing the base observation before a derivative one) are used sparingly and correctly, with no inverted logic.

****Grading Item 18: The rubric is not overprescriptive... (2/2)****

Despite its detail, the rubric allows varied yet accurate explanations; it does not require parroting exact wording from the golden answer.

****Grading Item 19: Each grading item contributes meaningfully... (2/2)****

No redundant or low-value items are present; each adds a distinct evaluative axis tied to the task needs.

FINAL_SCORE: {38}

Golden Answer

The headline understates what the PR is doing. Underneath the "unblock previously-skipped tests" framing, this is a coordinated migration of the Go client acceptance suite onto a new WithSearchParam builder API, a consolidation that deletes one skip-only stub plus one fully-bodied but skipped helper test (folding the helper's body into a sibling), a re-skip of two scenarios under a different unresolved issue, and a set of new assertions that document a server-side behaviour change rather than verifying it. Production code does not move. The headline is broadly correct, but one block of new assertions reads as the opposite of what the test names promise, and that

is the most important thing to flag before merge. Notation throughout this triage: function names appear in plain text, type names appear in plain text, file paths are wrapped in double quotes, the analysis runs in prose with no fenced code blocks.

Code Change Summary

"tests/go_client/testcases/groupby_search_test.go" gets four kinds of edit. Three previously-skipped tests lose their `t.Skip("https://github.com/milvus-io/milvus/issues/38343")` guards and become active: `TestSearchGroupByFloatDefault`, `TestSearchGroupByFloatDefaultCosine`, and `TestSearchGroupByBinaryGrowing`. Two tests gain a `t.Parallel()` call: `TestSearchGroupByFloatGrowing` and `TestSearchGroupByPagination`. Every other `GroupBy` test in the file (`TestSearchGroupByFloatDefault` at line 94, `TestSearchGroupByFloatDefaultCosine` at line 152, `TestSearchGroupByBinaryDefault` at line 265, `TestSearchGroupByBinaryGrowing` at line 301) already calls `t.Parallel()` at the base commit and is not retouched, so after this PR every active `GroupBy` test in the file runs in parallel. `TestSearchGroupByRangeSearch` is re-skipped under a different issue (now `Skipf("https://github.com/milvus-io/milvus/issues/38846")`) but its body is rewritten in the process: the commented-out `sp.AddRadius(0)` and `sp.AddRangeFilter(0.8)` lines are deleted, and the search call is updated from the old `sp-style` search-param mechanism to the new chained `.WithSearchParam("radius", "0").WithSearchParam("range_filter", "0.8")` API. The body will not run until #38846 is resolved, but the API migration work is landed.

"tests/go_client/testcases/query_test.go" gets a single test reactivated and a small semantic flip inside it. `TestQueryOutputFields` loses its `t.Skip("verify TODO")` guard. Inside the `enableDynamic` branch, `log.Debug` is changed to `log.Info` so the debug-level row dump is now always printed, and the `dynamicColumn` construction is changed from `hp.MergeColumnsToDynamic(10, hp.GenDynamicColumnData(0, 10), common.DefaultDynamicFieldName)` to `hp.MergeColumnsToDynamic(10, []column.Column{}, common.DefaultDynamicFieldName)`. The old call is left in the file as a commented-out line. `MergeColumnsToDynamic` in

"tests/go_client/testcases/helper/data_helper.go" iterates the columns slice per row to assemble each row's dynamic JSON, and with an empty slice every row's map is empty. The test thereby moves from asserting that the query returns ten rows of populated dynamic data to asserting that querying a non-existent output field name returns the field with empty dynamic data per row.

"tests/go_client/testcases/search_test.go" gets the bulk of the change. A new import for `strconv` lands at the top. `TestSearchNotMatchMetricType` loses its skip and the search

call gains `.WithSearchParam("metric_type", "L2")` so the test now passes the metric type through the new builder rather than through the old default-config path. Two functions are deleted entirely: `TestSearchSearchParamsMismatchIndex` (which was a true skip-only stub at the base commit, body == one `t.Skip` line) and `TestSearchEfHnsw` (which already had a full body of setup plus a search asserting `common.CheckErr(t, err, false, "ef(7) should be larger than k(10)")`, all gated behind a leading `t.Skip`). The `TestSearchEfHnsw` body is what gets folded into `TestSearchInvalidSearchParams`, which previously contained only a `t.Skip` line. The new body of `TestSearchInvalidSearchParams` loops over `invalidEfs := []int{-1, 0, 32769}`, calling `mc.Search` with `WithSearchParam("ef", strconv.Itoa(...))` for each, then over `invalidNprobes := []int{-1, 0, 65537}` similarly, then issues one more search with `ef=7` (which is below `DefaultLimit=10`), and runs `CheckSearchResult` expecting one `nq` row of `DefaultLimit` results. `TestSearchInvalidScannReorderK` loses its skip and gains a body that issues an invalid search with `reorder_k=DefaultLimit-1` followed by a valid search with `reorder_k=20`. `TestSearchScannAllMetricsWithRawData` loses its skip and its surrounding `/* ... */` comment block, and the body is updated: the schema option gains `TWithEnableDynamicField(true)`, the insert no longer hardcodes a row count of 500, and the index constructor is updated from `index.NewSCANNIndex(entity.COSINE, 16)` to the new three-argument `index.NewSCANNIndex(metricType, 16, withRawData)` so both `withRawData=true` and `withRawData=false` get exercised across the L2/IP/COSINE metric types. The search query count rises from 1 to `common.DefaultNq=5` and the asserted output-field list drops `common.DefaultFloatFieldName`. `TestRangeSearchSparseVector` loses its old skip and is re-skipped under issue #38846 with a fully-implemented body: a default search using `WithSearchParam("drop_ratio_search", "0.2")`, then a range search adding `WithSearchParam("radius", "10").WithSearchParam("range_filter", "30")`, and finally per-score assertions that every returned score is in the half-open interval `[10, 30)`. The body contains a duplicate `common.CheckErr(t, errSearch, true)` line on consecutive lines, which is a copy-paste artefact.

That is the entire diff. No production code is touched.

Blast Radius

The change that ripples furthest is the wiring of the new `WithSearchParam` API. The builder method is defined twice in `"client/milvusclient/read_options.go"`: on the underlying `annRequest` at line 248 and on the public `searchOption` at line 354, both with signature `WithSearchParam(key, value string)`. Client-side validation does not exist, and the builder accumulates pairs into a `searchParam map[string]string` field at line 71 and merges them into the protobuf `SearchRequest's SearchParams` field via

entity.MapKvPairs(params) on the wire. Any rejection of bad values happens server-side. Every newly-active or migrated test in this PR is now reaching the server-side validator (or whatever should be acting as one) for the first time through this acceptance path.

The new SCANN tests reach "client/index/scann.go" at line 44 where NewSCANNIndex(metricType MetricType, nlist int, withRawData bool) is defined. Pre-PR the test code called the two-argument form which would not compile against current production, so the test was a compile-broken stub kept alive only by the t.Skip guard. The reactivation lines up the test with the production three-argument signature.

The reactivated TestQueryOutputFields reaches "tests/go_client/testcases/helper/data_helper.go" MergeColumnsToDynamic at line 433, and the new dynamic-column-empty assertion exercises a previously-unasserted shape: a Column whose per-row data is an empty JSON object. The test fakeName is a literal "aaa" and the test now pins what the query response looks like when the requested output field name does not exist on the schema.

The CheckErr helper in "tests/go_client/common/response_checker.go" lines 19 through 41 deserves special note because it shapes the most important triage observation in this PR. Its signature is CheckErr(t *testing.T, actualErr error, expErrNil bool, expErrorMsg ...string). When expErrNil is true the function calls require.NoError(t, actualErr), and the variadic expErrorMsg is then never inspected. When expErrNil is false the function calls require.Error and then checks that the error string contains one of the supplied substrings.

What does not ripple. Production search-param validation, the SCANN index implementation, the range-search support, the sparse-vector handler, and the dynamic-field codec all retain their pre-PR behaviour because the PR does not edit them. Test files outside the three named ones are not touched. The CI cost rises by the number of newly-active test bodies and is bounded by the t.Parallel additions on two of the GroupBy tests.

Severity and Complexity

I read this as Low-Medium. Test-only is a low ceiling, but one block of new assertions inverts the meaning the test names suggest, and that should not merge without an author confirmation.

Most consequential: the new bodies of `TestSearchInvalidSearchParams` and `TestSearchInvalidScannReorderK` assert success, not failure, for cases the test names describe as "invalid". Each `invalidEf` in `{-1, 0, 32769}` is exercised with `common.CheckErr(t, errHnsw, true, "No error for invalid search params")`. The `expErrNil=true` argument means `require.NoError`, and the "No error for invalid search params" string is the unused variadic arg. The same pattern repeats for `invalidNprobes` in `{-1, 0, 65537}`. After both loops the test runs one more search with `ef=7` (where `DefaultLimit=10` so `ef<topK` by the historical validation rule) and asserts `common.CheckErr(t, err, true, "ef(7) should be larger than k(10), but no error")`. The "but no error" suffix in that message wording is the smoking gun: the author knows `ef=7` used to error and is documenting that it no longer does.

`TestSearchInvalidScannReorderK` has the same shape, asserting success for `reorder_k = DefaultLimit - 1 = 9` against a `topK` of 10. Pre-PR the equivalent direct test asserted the opposite: `common.CheckErr(t, err, false, "ef(7) should be larger than k(10)")`. Three readings are possible. Either server-side validation of these parameters was removed elsewhere and this PR is documenting the change (which would be a production regression flagged in the wrong place), or the `WithSearchParam` plumbing does not reach the validator the way the older config-driven plumbing did (which would be a client-side wiring gap), or the test intent is to land a documentation-only assertion of current behaviour and leave the rejection assertion for follow-up. Any of the three needs to be named in the PR description. The fix if the intent was to assert rejection is to flip the second `CheckErr` argument from `true` to `false`, and if the intent is to assert current silent-acceptance, the test name and the third-arg messages should be reworded so a future reader does not read the asserts upside down.

Next: the `MergeColumnsToDynamic` change in `TestQueryOutputFields` is a semantic flip rather than a refactor. Pre-PR the test asserted that a query whose output-field list includes a non-existent name "aaa" returns the dynamic column populated with ten rows of generated dynamic data. Post-PR it asserts the dynamic column is returned with empty per-row maps. The old behaviour was either wrong or the spec changed, and the PR description does not say which. The right shape is to either restore the populated-data assertion (if the old behaviour was the intended spec) or document why the empty-data assertion is now correct. The leftover commented-out `hp.MergeColumnsToDynamic(10, hp.GenDynamicColumnData(0, 10), ...)` line should be removed before merge regardless because dead code in test files rots quickly.

Next: the re-skip of `TestSearchGroupByRangeSearch` and `TestRangeSearchSparseVector` under issue #38846 is reasonable, but the bodies are now fully implemented behind the skip. If a future contributor closes #38846 without re-reading the bodies, the range-search assertions (the score must be in `[10, 30)`, the

duplicate CheckErr line on consecutive lines) ship as part of the unblock. Cleaning up the duplicate CheckErr now costs one line and saves a future contributor a confused minute.

Smaller items. The t.Parallel additions to TestSearchGroupByFloatGrowing and TestSearchGroupByPagination bring the GroupBy tests in this file into uniform parallelism (every active GroupBy test in the file now calls t.Parallel) which is good housekeeping rather than a flag. log.Debug to log.Info in TestQueryOutputFields is a noise change that will surface row dumps in every CI run, and whether that is wanted is worth one line from the author. The new strconv import is paired only with WithSearchParam string-conversion calls, and if a future test wants to pass numeric params, the helper should be extracted instead of copying strconv.Itoa everywhere.

Alternative fixes that are tempting but wrong. Reverting the un-skip changes because the assertions look inverted is wrong: the un-skips are correct work, only the assertion direction inside TestSearchInvalidSearchParams is the question, and reverting throws away the consolidation of two stub functions plus the migration to the new builder API. Demanding the deleted TestSearchEfHnsw and TestSearchSearchParamsMismatchIndex stubs be preserved for git-blame continuity is wrong, and deleting unused skip-only functions is housekeeping and the merge into TestSearchInvalidSearchParams is exactly the kind of consolidation a reviewer should encourage. Splitting the un-skip work from the consolidation refactor as a hard block is wrong because the consolidation is what makes the un-skips coherent, and splitting at review time adds two PRs of overhead without producing a behaviour change.

Recommended verdict: approve with comments. Required actions: confirm with the author whether TestSearchInvalidSearchParams and TestSearchInvalidScannReorderK intend to assert rejection or to document silent acceptance, and adjust the second CheckErr argument or the test names accordingly, remove the duplicate common.CheckErr(t, errSearch, true) line in TestRangeSearchSparseVector, and remove the commented-out MergeColumnsToDynamic line in TestQueryOutputFields.

Security Risk

No classic security regression. The diff does not introduce or change authentication, authorisation, RBAC, untrusted-input parsing on the wire, or cryptographic boundaries. The bug-shape concern around silent acceptance of invalid search params is an integrity-of-results question for the search engine, not an access-control or input-parsing vulnerability. Customer-facing permission decisions are unaffected.

Operational and observability risk concentrates on whatever explains the silent acceptance of invalid ef/nprobe/reorder_k values. If server-side validation was removed elsewhere, the production-side gap is the real concern but it lives outside this PR. A reviewer who notices the assertion direction in this PR should open a follow-up issue against the search-param validator (or whichever production layer used to emit the ef(7) should be larger than k(10) error) rather than treat this as a test-only PR with no production implication.

Test coverage gaps. The newly active tests assert happy-path behaviour for the new WithSearchParam API across a representative set of parameters (ef, nprobe, radius, range_filter, metric_type, drop_ratio_search, reorder_k). They do not cover the failure modes the test names imply they should cover. If the server should be validating these params and silently accepts them today, no test in this PR catches a regression where validation is removed or weakened in a future change. Worth listing as a follow-up rather than blocking on, since the PR's stated goal is to land the test-reactivation under the new API rather than to verify validation semantics.

Golden Answer Review

Golden Answer QC Feedback

****Grading Item 1 (3/3)****

The response is internally consistent, references specific behaviors, and avoids contradictions.

****Grading Item 2 (3/3)****

It is clearly grounded in repo exploration, detailing concrete files, tests, helper functions, and APIs from the codebase.

****Grading Item 3 (2/2)****

It directly answers the prompt: describes what the change is doing, which parts of the test suite and API are affected, and what issues to flag.

****Grading Item 4 (3/3)****

The reasoning chain is explicit, explaining how observations about the diff lead to conclusions about behavior, impacts, and reviewer concerns.

****Grading Item 5 (2/2)****

Technical explanations are precise, describing mechanisms (e.g., CheckErr semantics, WithSearchParam wiring) concretely.

****Grading Item 6 (2/2)****

All technical statements align with the repo context and behaviors, with no apparent inaccuracies.

****Grading Item 7 (3/3)****

All critical rubric requirements are met: enumerated files, production surfaces, security conclusion, alternative-fixes section, and detailed diff observations.

****Grading Item 8 (2/2)****

No penalty-triggering behaviors appear; the analysis avoids fabricated impacts or incorrect recommendations.

****Grading Item 9 (2/2)****

The coverage is comprehensive for the task, addressing major changes, ripples, and reviewer flags.

****Grading Item 10 (2/2)****

Writing is organized and understandable to non-experts, with clear headings and explanations.

****Grading Item 11 (2/2)****

Includes concrete details such as file paths, function names, helper behavior, and parameter values.

****Grading Item 12 (3/3)****

Follows PR Triage expectations, focusing on code changes, blast radius, severity, and security risk in an appropriate style.

****Grading Item 13 (2/2)****

Provides the four required PR Triage sections explicitly: code change summary, blast radius, severity/complexity, and security risk.

****Grading Item 14 (1/1)****

Contains only ASCII characters.

FINAL_SCORE: 32

Validate Golden Answer

Golden Answer Validation

```
{
  "results": [
    {
      "id": "reasoning-64318",
      "tag": "reasoning",
      "weight": "penalty",
      "description": "Recommends redesigning the CheckErr helper or the WithSearchParam API as the
primary triage outcome.",
      "rationale": "A redesign-first answer skips the actual review job. The change is a concrete diff to
approve or request-changes on, not a test-helper or builder-API exercise. Recommending a redesign
leaves the inverted-assertion question unanswered.",
      "dependent_on": [],
      "pass": true,
      "evidence": "The response analyzes the existing CheckErr helper but does not advise modifying the
helper or WithSearchParam API; instead it recommends specific test-level fixes such as adjusting
assertions and removing duplicate lines.",
      "reasoning": "The criterion is satisfied because the golden answer focuses on reviewing the diff and
suggesting test corrections rather than proposing a redesign of core helpers or APIs."
    },
    {
      "id": "reasoning-64312",
      "tag": "reasoning",
      "weight": "bonus",
      "description": "Notes that the 'but no error' suffix in the ef=7 CheckErr message wording documents an
observed behaviour change.",
      "rationale": "The message text only renders when CheckErr is invoked with expErrNil=false and an error
is present. Because expErrNil is true here the message is dead string. The author's wording ('ef(7) should
be larger than k(10), but no error') is therefore documentation of a behaviour change the author observed,
```

not an assertion of it. The smoking-gun observation distinguishes a careful read of the helper semantics from a surface read of the CheckErr call.",

```
"dependent_on": [  
  "reasoning-64303"  
],
```

```
"pass": true,
```

```
"evidence": "\u201cAfter both loops the test runs one more search with ef=7 ... and asserts ... The '\u201dbut  
no error\u201c suffix in that message wording is the smoking gun: the author knows ef=7 used to error and is  
documenting that it no longer does.\u201d",
```

```
"reasoning": "The answer explicitly highlights the 'but no error' wording as evidence that the author is  
documenting an observed change in behaviour rather than asserting it, satisfying the criterion."
```

```
},
```

```
{
```

```
"id": "reasoning-64308",
```

```
"tag": "reasoning",
```

```
"weight": "bonus",
```

```
"description": "Notes the new three-argument NewSCANNIndex(metricType, 16, withRawData) wiring in  
TestSearchScannAllMetricsWithRawData.",
```

```
"rationale": "The pre-PR call was index.NewSCANNIndex(entity.COSINE, 16) inside the skipped /* ... */  
block. The current production signature at client/index/scann.go:44 takes three arguments. Naming the  
wire-up shows the reviewer understood why the test could only run after the skip lifted: the body's prior  
2-arg call would not compile against current production.",
```

```
"dependent_on": [],
```

```
"pass": true,
```

```
"evidence": "\u201cTestSearchScannAllMetricsWithRawData \u2026 the index constructor is updated from  
index.NewSCANNIndex(entity.COSINE, 16) to the new three-argument  
index.NewSCANNIndex(metricType, 16, withRawData)\u201c and \u201cThe new SCANN tests reach \u2026  
NewSCANNIndex(metricType MetricType, nlist int, withRawData bool)\u2026 Pre-PR the test code called  
the two-argument form\u2026 The reactivation lines up the test with the production three-argument  
signature.\u201d",
```

```
"reasoning": "The golden answer explicitly notes that the SCANN test now uses the three-argument  
NewSCANNIndex(metricType, 16, withRawData) signature and contrasts it with the previous  
two-argument version, demonstrating clear understanding of the wiring change."
```

```
},
```

```
{
```

```
"id": "reasoning-64304",
```



```

    "tag": "reasoning",
    "weight": "critical",
    "description": "States that the PR makes no changes to production code.",
    "rationale": "Anchoring the triage in the test-only scope bounds the severity ceiling. A reviewer who
imagines production-side risk inflates the severity and asks for review of code that does not move in this
patch.",
    "dependent_on": [],
    "pass": true,
    "evidence": "\"Production code does not move.\" and later \"That is the entire diff. No production code
is touched.\"
    \"reasoning\": \"The golden answer explicitly states multiple times that the PR only affects tests and that
no production code files change, satisfying the criterion.\"
  },
  {
    \"id\": \"reasoning-64301\",
    \"tag\": \"reasoning\",
    \"weight\": \"critical\",
    \"description\": \"Identifies that TestSearchEfHnsw and TestSearchSearchParamsMismatchIndex are
deleted as skip-only stubs and their scaffolding is merged into TestSearchInvalidSearchParams.\",
    \"rationale\": \"The consolidation is the load-bearing refactor in search_test.go. Without naming the
deletions and the merge a reader cannot judge that the new TestSearchInvalidSearchParams body was
previously distributed across three function names. The observation also bounds the test-count claim
correctly: three functions become one.\",
    \"dependent_on\": [],
    \"pass\": true,
    \"evidence\": \"\u201cThe two skip-only stub functions TestSearchEfHnsw and
TestSearchSearchParamsMismatchIndex are deleted entirely, and their setup scaffolding ... is merged
into TestSearchInvalidSearchParams which previously had only a t.Skip line.\u201d\",
    \"reasoning\": \"The response explicitly states that both skip-only stubs are removed and their scaffolding
combined into TestSearchInvalidSearchParams, satisfying the requirement to describe the consolidation.\"
  },
  {
    \"id\": \"reasoning-64305\",
    \"tag\": \"reasoning\",
    \"weight\": \"critical\",

```

"description": "Identifies the migration to the new WithSearchParam(key, value) builder API on searchOption defined in client/milvusclient/read_options.go.",

"rationale": "The reactivated and new tests reach the production search-param wire format through .WithSearchParam(...) calls (radius, range_filter, metric_type, ef, nprobe, drop_ratio_search, reorder_k). Naming the builder is what connects the un-skip pattern to the actual API unlock the PR is exercising for the first time in this acceptance path.",

"dependent_on": [],

"pass": true,

"evidence": "\u201cThe change that ripples furthest is the wiring of the new WithSearchParam API. The builder method is defined twice in \u2018client/milvusclient/read_options.go\u2019: on the underlying annRequest at line 248 and on the public searchOption at line 354, both with signature WithSearchParam(key, value string).\u201d",

"reasoning": "The golden answer explicitly identifies the migration to the WithSearchParam builder API, referencing its definition in client/milvusclient/read_options.go and explaining how it is used, thus satisfying the criterion."

},

{

"id": "reasoning-64307",

"tag": "reasoning",

"weight": "critical",

"description": "Identifies the semantic flip in TestQueryOutputFields where MergeColumnsToDynamic switches from GenDynamicColumnData(0, 10) to an empty column slice.",

"rationale": "MergeColumnsToDynamic at data_helper.go iterates the columns slice per row to assemble each row's dynamic JSON object. The empty slice yields empty per-row maps. The test moves from asserting a populated dynamic-field response (ten rows of generated data) to asserting empty dynamic data when the requested output field name does not exist. This is a spec change rather than a refactor and warrants a one-line note from the author.",

"dependent_on": [],

"pass": true,

"evidence": "\u201cthe dynamicColumn construction is changed from hp.MergeColumnsToDynamic(10, hp.GenDynamicColumnData(0, 10), common.DefaultDynamicFieldName) to hp.MergeColumnsToDynamic(10, []column.Column{}, common.DefaultDynamicFieldName)\u2026With an empty slice every row's map is empty. The test thereby moves from asserting that the query returns ten rows of populated dynamic data to asserting that \u2026 returns \u2026 empty dynamic data per row.\u201d",

"reasoning": "The golden answer explicitly explains that MergeColumnsToDynamic now receives an empty column slice, producing empty dynamic data and altering the test's semantics, satisfying the criterion."

},

{

"id": "reasoning-64309",

"tag": "reasoning",

"weight": "bonus",

"description": "Notes that t.Parallel is added to TestSearchGroupByFloatGrowing and TestSearchGroupByPagination but not to TestSearchGroupByBinaryDefault in the same file.",

"rationale": "The parallelism decision is inconsistent across three otherwise-similar growing tests. Whether to parallelise is a fair call, but the asymmetry reads as oversight rather than a deliberate split. A reviewer who notes the asymmetry is checking that t.Parallel additions are systematic rather than incidental.",

"dependent_on": [],

"pass": true,

"evidence": "Two tests gain a t.Parallel() call: TestSearchGroupByFloatGrowing and TestSearchGroupByPagination. The third growing test in the file, TestSearchGroupByBinaryDefault, does not get t.Parallel(), so the parallel-coverage decision is inconsistent.",

"reasoning": "The golden answer explicitly points out the asymmetric addition of t.Parallel calls across the three growing tests and comments on the inconsistency, fulfilling the criterion."

},

{

"id": "reasoning-64313",

"tag": "reasoning",

"weight": "bonus",

"description": "Notes the log.Debug to log.Info change in TestQueryOutputFields that promotes per-row dynamic-field data to standard CI output.",

"rationale": "Promoting debug logs to info level is a noise change. CI runs will surface the row dump in standard log output going forward. Whether that is wanted is a one-line question from the author. A reviewer who flags it is reading the diff for noise-level changes rather than only assertion changes.",

"dependent_on": [],

"pass": true,

"evidence": "\"Inside the enableDynamic branch, log.Debug is changed to log.Info so the debug-level row dump is now always printed\" and later \"log.Debug to log.Info in TestQueryOutputFields is a noise

change that will surface row dumps in every CI run; whether that is wanted is worth one line from the author.\""

"reasoning": "The golden answer explicitly notes the log level change in TestQueryOutputFields, including its effect on CI output, satisfying the criterion."

},

{

"id": "reasoning-64302",

"tag": "reasoning",

"weight": "critical",

"description": "Identifies that the new TestSearchInvalidSearchParams asserts success for invalid ef and nprobe values via CheckErr with expErrNil=true.",

"rationale": "The CheckErr helper at response_checker.go:19-41 treats the second argument as expErrNil and the third as an unused message when expErrNil is true. The new body issues searches with invalidEfs {-1, 0, 32769} and invalidNprobes {-1, 0, 65537} and asserts each call returns no error. The test name promises invalid-param coverage; the assertions document silent acceptance. Without surfacing this contradiction a reviewer ships an assertion direction that points the wrong way.",

"dependent_on": [],

"pass": true,

"evidence": "\u201cMost consequential: the new bodies of TestSearchInvalidSearchParams and TestSearchInvalidScannReorderK assert success, not failure, for cases the test names describe as 'invalid'. Each invalidEf in {-1, 0, 32769} is exercised with common.CheckErr(t, errHnsw, true, 'No error for invalid search params'). The expErrNil=true argument means require.NoError\u2026 The same pattern repeats for invalidNprobes in {-1, 0, 65537}.\u201d",

"reasoning": "The golden answer explicitly explains that TestSearchInvalidSearchParams now uses CheckErr with expErrNil=true for invalid ef and nprobe values, highlighting that the tests assert success despite the name promising invalid-param coverage."

},

{

"id": "reasoning-64315",

"tag": "reasoning",

"weight": "penalty",

"description": "Asserts that the PR introduces a classic security regression (authentication, authorisation, RBAC, untrusted-input parsing, or network surface).",

"rationale": "The diff is acceptance-test reactivation and consolidation. Asserting a classic security issue inflates the severity and misroutes the fix toward production layers that do not move in this patch.",

"dependent_on": [],

```

    "pass": true,
    "evidence": "\u201cNo classic security regression. The diff does not introduce or change authentication, authorisation, RBAC, untrusted-input parsing on the wire, or cryptographic boundaries.\u201d",
    "reasoning": "The golden answer explicitly states that there is no classic security regression, so it does not exhibit the harmful behavior of asserting one."
  },
  {
    "id": "reasoning-64306",
    "tag": "reasoning",
    "weight": "critical",
    "description": "Identifies that TestSearchGroupByRangeSearch and TestRangeSearchSparseVector are re-skipped under issue 38846 with their bodies migrated to the new WithSearchParam API.",
    "rationale": "The re-skip pattern is easy to miss: the t.Skip is preserved but the body is now fully implemented and waits for #38846 to be resolved. A reviewer who reads only the t.Skip line treats the tests as inactive without noticing that the API migration work landed under the skip guard.",
    "dependent_on": [],
    "pass": true,
    "evidence": "\"TestSearchGroupByRangeSearch is re-skipped under a different issue (now Skipf(\">https://github.com/milvus-io/milvus/issues/38846\") but its body is rewritten... updated ... to the new chained .WithSearchParam...\" and \"TestRangeSearchSparseVector loses its old skip and is re-skipped under issue #38846 with a fully-implemented body: a default search using WithSearchParam...\"",
    "reasoning": "The golden answer explicitly notes both tests being re-skipped under issue 38846 and describes their bodies being migrated to the WithSearchParam API, satisfying the criterion."
  },
  {
    "id": "reasoning-64314",
    "tag": "reasoning",
    "weight": "penalty",
    "description": "Asserts as fact that the PR modifies production code (the WithSearchParam builder, the NewSCANNIndex constructor, or the server-side search-param validator).",
    "rationale": "The PR is test-only across three test files. Claiming production-side changes misreads the diff entirely and would have a reviewer ask for review of code that does not move in this patch.",
    "dependent_on": [],
    "pass": true,

```

```

    "evidence": "\"That is the entire diff. No production code is touched.\",",
    "reasoning": "The golden answer explicitly states that no production code is modified and never claims
otherwise, thus avoiding the prohibited assertion."
},
{
    "id": "reasoning-64319",
    "tag": "reasoning",
    "weight": "penalty",
    "description": "Recommends splitting the un-skip work from the consolidation refactor as a hard
blocker.",
    "rationale": "Splitting at review time adds two PRs of overhead without producing a behaviour change.
The consolidation is what makes the un-skips coherent. The right outcome is to ask the author to call out
the consolidation in the PR description, not to demand a split.",
    "dependent_on": [],
    "pass": true,
    "evidence": "\"Alternative fixes that are tempting but wrong. ... Splitting the un-skip work from the
consolidation refactor as a hard block is wrong because the consolidation is what makes the un-skips
coherent...\",",
    "reasoning": "The golden answer explicitly argues against demanding a split of the un-skip work from
the consolidation refactor, so it does not recommend the harmful behavior."
},
{
    "id": "reasoning-64303",
    "tag": "reasoning",
    "weight": "critical",
    "description": "Identifies that the ef=7 case in TestSearchInvalidSearchParams asserts success even
though DefaultLimit=10 and the historical validation rule was ef should be larger than k.",
    "rationale": "The pre-PR equivalent assertion was CheckErr(t, err, false, 'ef(7) should be larger than
k(10)') which expected the error. The post-PR assertion flips to expErrNil=true with message 'ef(7) should
be larger than k(10), but no error', so the message documents the behaviour change in plain prose. A
reviewer who reads only the literal CheckErr call without unpacking the helper semantics misses the
inverted assertion entirely.",
    "dependent_on": [
        "reasoning-64302"
    ],
    "pass": true,

```

"evidence": "\u201cAfter both loops the test runs one more search with ef=7 (where DefaultLimit=10 so ef<topK by the historical validation rule) and asserts common.CheckErr(t, err, true, \"ef(7) should be larger than k(10), but no error\"). The \"but no error\" suffix in that message wording is the smoking gun: the author knows ef=7 used to error and is documenting that it no longer does.\u201d",

"reasoning": "The golden answer explicitly highlights that the ef=7 check now expects success despite historically requiring ef>k, noting both the DefaultLimit=10 context and how the new assertion documents the deviation."

},

{

"id": "reasoning-64310",

"tag": "reasoning",

"weight": "bonus",

"description": "Notes the dead commented-out MergeColumnsToDynamic call left in TestQueryOutputFields immediately above the new live call.",

"rationale": "Leaving a one-line comment that mirrors the previous behaviour adds no value and rots quickly. A reviewer who flags the dead-code line is doing the lowest-cost housekeeping ask the PR would benefit from. The observation also paired well with the wider concern that the semantic flip in this test needs a PR-description note.",

"dependent_on": [],

"pass": true,

"evidence": "The leftover commented-out hp.MergeColumnsToDynamic(10, hp.GenDynamicColumnData(0, 10), ...) line should be removed before merge regardless because dead code in test files rots quickly.",

"reasoning": "The answer explicitly notes the commented-out MergeColumnsToDynamic call and recommends removing it, satisfying the criterion."

},

{

"id": "reasoning-64311",

"tag": "reasoning",

"weight": "bonus",

"description": "Notes the duplicate common.CheckErr(t, errSearch, true) line on consecutive lines inside TestRangeSearchSparseVector.",

"rationale": "The duplicate is a clear copy-paste artefact. It runs twice harmlessly today but degrades readability and would surface if someone changed one of the two without realising they had a sibling. A reviewer who catches the duplicate is reading the body of a re-skipped test rather than skipping past it on the t.Skip line.",

```

    "dependent_on": [],
    "pass": true,
    "evidence": "The body contains a duplicate common.CheckErr(t, errSearch, true) line on consecutive
lines, which is a copy-paste artefact.",
    "reasoning": "The golden answer explicitly highlights the duplicate CheckErr line in
TestRangeSearchSparseVector, satisfying the requirement to note this issue."
  },
  {
    "id": "reasoning-64317",
    "tag": "reasoning",
    "weight": "penalty",
    "description": "Recommends preserving the deleted TestSearchEfHnsw and
TestSearchSearchParamsMismatchIndex stubs for git-blame continuity.",
    "rationale": "Both deleted functions were skip-only stubs with no test logic. Preserving them after their
content has been merged into TestSearchInvalidSearchParams adds maintenance cost without producing
test coverage. The consolidation is housekeeping a reviewer should encourage.",
    "dependent_on": [],
    "pass": true,
    "evidence": "\"Demanding the deleted TestSearchEfHnsw and TestSearchSearchParamsMismatchIndex
stubs be preserved for git-blame continuity is wrong; deleting unused skip-only functions is housekeeping
and the merge into TestSearchInvalidSearchParams is exactly the kind of consolidation a reviewer should
encourage.\"\"",
    "reasoning": "The golden answer explicitly discourages preserving the deleted stubs, aligning with the
rubric and avoiding the penalized behavior."
  },
  {
    "id": "reasoning-64300",
    "tag": "reasoning",
    "weight": "critical",
    "description": "Identifies that three previously-skipped tests in
tests/go_client/testcases/groupby_search_test.go lose their issue-38343 t.Skip guards and become
active (TestSearchGroupByFloatDefault, TestSearchGroupByFloatDefaultCosine,
TestSearchGroupByBinaryGrowing).",
    "rationale": "The un-skipping is the headline activation in the GroupBy file. Anchoring the triage on the
three named tests is how the reader connects the issue-38343 reference to the concrete CI coverage that
just turned on.",

```



```

    "dependent_on": [],
    "pass": true,
    "evidence": "\"Three previously-skipped tests lose their
t.Skip(\\\"https://github.com/milvus-io/milvus/issues/38343\\\") guards and become active:
TestSearchGroupByFloatDefault, TestSearchGroupByFloatDefaultCosine, and
TestSearchGroupByBinaryGrowing.\\\"",
    "reasoning": "The golden answer explicitly identifies all three specified tests in groupby_search_test.go
as having their issue-38343 skips removed and reactivated, satisfying the criterion directly."
},
{
    "id": "reasoning-64316",
    "tag": "reasoning",
    "weight": "penalty",
    "description": "Recommends reverting the un-skip changes as the appropriate fix for the
inverted-assertion concern in TestSearchInvalidSearchParams.",
    "rationale": "The un-skips are correct work. Only the assertion direction inside the new body is the
question, and the right fix is to flip the second CheckErr argument from true to false if the intent is to
assert rejection, or to rename the test if the intent is to document silent acceptance. Reverting throws
away the consolidation and the migration to the new builder API.",
    "dependent_on": [],
    "pass": true,
    "evidence": "\"Reverting the un-skip changes because the assertions look inverted is wrong: the
un-skips are correct work...\\\"",
    "reasoning": "The golden answer explicitly states that reverting the un-skip changes would be incorrect,
thereby avoiding the penalized behavior."
},
{
    "id": "completeness-64321",
    "tag": "completeness",
    "weight": "critical",
    "description": "Names the production surface the new tests reach without the PR touching it
(WithSearchParam builder, NewSCANNIndex three-argument constructor, server-side search-param
validator).",
    "rationale": "The diff is test-only and the reactivated tests reach production code the PR leaves intact. A
triage that does not name the production surface cannot ground the blast-radius claim that the new tests

```

exercise unmodified code. The reader needs the surface named to verify the no-production-change claim.",

```
"dependent_on": [],
```

```
"pass": true,
```

```
"evidence": "\"The change that ripples furthest is the wiring of the new WithSearchParam API. The builder method is defined twice in \\\"client/milvusclient/read_options.go\\\" ...\\\"; \\\"The new SCANN tests reach \\\"client/index/scann.go\\\" ... NewSCANNIndex(metricType MetricType, nlist int, withRawData bool)\\\"; \\\"Every newly-active or migrated test in this PR is now reaching the server-side validator ...\\\"\",
```

```
"reasoning": "The answer explicitly identifies each untouched production surface impacted by the tests\u2014WithSearchParam builder, three-argument NewSCANNIndex, and the server-side search-parameter validator\u2014meeting the criterion."
```

```
},
```

```
{
```

```
"id": "completeness-64323",
```

```
"tag": "completeness",
```

```
"weight": "critical",
```

```
"description": "Includes an alternative-fixes paragraph preempting wrong-fix temptations such as reverting the un-skip changes, preserving the deleted stubs, or splitting the un-skip from the consolidation.",
```

```
"rationale": "Preempting wrong fixes is what makes the Severity section actionable. Showing the reviewer thought through alternatives before settling on the recommendation distinguishes a triage from a bullet list of complaints.",
```

```
"dependent_on": [],
```

```
"pass": true,
```

```
"evidence": "\u201cAlternative fixes that are tempting but wrong. Reverting the un-skip changes because the assertions look inverted is wrong\u201d Demanding the deleted TestSearchEfHnsw and TestSearchSearchParamsMismatchIndex stubs be preserved\u201d is wrong\u201d Splitting the un-skip work from the consolidation refactor\u201d is wrong\u201d\u201d",
```

```
"reasoning": "The answer contains a dedicated paragraph explicitly preempting wrong fixes, discussing why reverting un-skips, preserving deleted stubs, or splitting the work would be incorrect, satisfying the criterion."
```

```
},
```

```
{
```

```
"id": "completeness-64320",
```

```
"tag": "completeness",
```

```
"weight": "critical",
```

```

    "description": "Enumerates the three test files changed by the diff.",
    "rationale": "The mechanical surface of the PR spans three files under tests/go_client/testcases
(groupby_search_test.go, query_test.go, search_test.go). A triage that does not enumerate the files is
incomplete because downstream blast-radius and severity claims rest on knowing which surfaces
moved.",
    "dependent_on": [],
    "pass": true,
    "evidence": "The answer explicitly states: \"\"tests/go_client/testcases/groupby_search_test.go\" gets
four kinds of edit...\"; \"\"tests/go_client/testcases/query_test.go\" gets a single test reactivated...\"; and
\"\"tests/go_client/testcases/search_test.go\" gets the bulk of the change.\"",
    "reasoning": "It clearly enumerates all three modified test files, satisfying the requirement to identify the
mechanical surface of the PR."
  },
  {
    "id": "completeness-64324",
    "tag": "completeness",
    "weight": "penalty",
    "description": "Centers the response on a redesign proposal (proposing a new CheckErr helper,
restructuring the WithSearchParam builder, or rewriting the test-organisation pattern) rather than triaging
the diff as written.",
    "rationale": "A redesign-first answer skips the actual review job. The change is a concrete diff to
approve or request-changes on, and an answer that pivots to design exercises does not produce an
approval decision the author can act on.",
    "dependent_on": [],
    "pass": true,
    "evidence": "The response concentrates on triaging the diff, summarizing the code changes, blast
radius, and specific issues to flag (e.g., inverted assertions, duplicate CheckErr).",
    "reasoning": "The answer does not pivot to proposing new helper designs or architectures; it remains
focused on evaluating the existing change and what needs attention before merge."
  },
  {
    "id": "completeness-64322",
    "tag": "completeness",
    "weight": "critical",
    "description": "Concludes the PR introduces no classic security regression.",

```

"rationale": "Test-only PRs invite the lazy reading that drops the security analysis entirely. Explicitly concluding no classic regression is what makes the security framing complete and distinguishes a thorough triage from one that omits the security frame.",

"dependent_on": [],

"pass": true,

"evidence": "Security Risk\\n\\nNo classic security regression.",

"reasoning": "The golden answer explicitly states that there is no classic security regression, directly satisfying the criterion requiring such a conclusion."

},

{

"id": "style-64325",

"tag": "style",

"weight": "bonus",

"description": "Organises the response into the four PRT&IA sections (Code Change Summary, Blast Radius, Severity and Complexity, Security Risk) or uses equivalent named sections that cover the same content.",

"rationale": "The four-section structure is the canonical PRT&IA output shape and helps the reader absorb a multi-concern review without losing track of which concern lives where.",

"dependent_on": [],

"pass": true,

"evidence": "The response explicitly uses sections titled \\u201cCode Change Summary,\\u201d \\u201cBlast Radius,\\u201d \\u201cSeverity and Complexity,\\u201d and \\u201cSecurity Risk.\\u201d",

"reasoning": "The golden answer mirrors the requested four-section structure, ensuring the content is organised into the canonical PRT&IA sections."

},

{

"id": "style-64326",

"tag": "style",

"weight": "bonus",

"description": "Uses plain-text identifiers for function and type names and double-quoted file paths.",

"rationale": "The project's prose convention pairs plain-text identifiers with double-quoted paths.

Consistent formatting helps the reader visually parse references at a glance and avoids the generated-artifact feel of backtick-wrapped tokens.",

"dependent_on": [],

"pass": true,

```

    "evidence": "File paths such as \"tests/go_client/testcases/groupby_search_test.go\" and
\"client/milvusclient/read_options.go\" are enclosed in double quotes, and identifiers like
TestSearchGroupByFloatDefault and TestSearchInvalidSearchParams appear in plain text.",
    "reasoning": "All references to functions and file paths follow the stated convention: identifiers are plain
text and paths are double-quoted."
  }
],
"passed": 27,
"failed": 0,
"total": 27,
"score_percent": 100.0,
"summary": "Passed 27/27 criteria (100.0%)"
}

```

Test Model

```

=====
RUN 1/3
=====
### What actually changed beyond "unlock previously-skipped Go client search tests"?

```

The patch does more than just drop a few `t.Skip`` calls:

- Group-by search coverage is back on**
 - ``TestSearchGroupByFloatDefault``, ``TestSearchGroupByFloatDefaultCosine``, and ``TestSearchGroupByBinaryGrowing`` are now active again after removing the skip that referenced issue #38343 (see ``tests/go_client/testcases/groupby_search_test.go``, lines 92-199 and 297-327).
 - ``TestSearchGroupByFloatGrowing`` and ``TestSearchGroupByPagination`` were also marked ``t.Parallel()`` so they run concurrently (lines 330-405).
 - The range-search test still skips, but the reason was updated and the body now shows how ``WithSearchParam("radius", ...) / ("range_filter", ...)`` will be used once issue #38846 is fixed (lines 443-459).
- The previously skipped query test is live**
 - ``TestQueryOutputFields`` no longer calls ``t.Skip`` and the expected dynamic-field payload was altered (see ``tests/go_client/testcases/query_test.go``, lines 165-236). The change swaps ``hp.MergeColumnsToDynamic(.., hp.GenDynamicColumnData(..)`` for an empty dynamic column when asserting output for a fake field.
- Large portions of ``search_test.go`` now exercise real search parameters**

* Additional tests now pass explicit search parameters to the client, e.g. ``WithSearchParam("metric_type", "L2")`` in ``TestSearchNotMatchMetricType`` (lines 455-469), HNSW ``ef`/`nprobe`` checks plus SCANN ``reorder_k`` (lines 509-580), and sparse-vector range-search params (lines 1061-1095).

* ``TestSearchScannAllMetricsWithRawData`` is fully enabled again and loops through metrics and the ``with_raw_data`` flag (lines 555-581).

* All these tests rely on ``client.NewSearchOption(...).WithSearchParam(key, value)``, which feeds the ``searchParam`` map in ``client/milvusclient/read_options.go`` (lines 248-356). That map is merged into ``SearchParams`` in the generated protobuf request (lines 103-139), so the API surface being covered is the existing ``WithSearchParam`` helper on ``SearchOption`/`annRequest``.

4. **Range-search tests for sparse vectors now show the intended API usage**

Even though ``TestRangeSearchSparseVector`` still skips pending server support, it now demonstrates how ``drop_ratio_search``, ``radius``, and ``range_filter`` parameters will be supplied once enabled (lines 1079-1105).

Ripple effects on the Go client surface

Although no production code changed, the tests now actively exercise a wider slice of the Go SDK:

- * Search options now ensure that arbitrary ANN/search params (metric type, ef, nprobe, radius, filters, SCANN reorder_k, sparse drop ratios, etc.) get serialized through ``WithSearchParam`` (see ``client/milvusclient/read_options.go``, lines 248-356).
- * Group-by search coverage stresses combinations of ``WithGroupByField``, ``WithANNSField``, ``WithConsistencyLevel``, and the concurrency of those calls.
- * Query output-field handling (dynamic field projection vs. fake columns) is exercised more thoroughly in ``TestQueryOutputFields``.

Issues to raise before approving

1. **Dynamic-field expectation now contradicts the inserted data**

In ``TestQueryOutputFields``, the expected column for the fake field ``aaa`` is built from ``hp.MergeColumnsToDynamic(10, []column.Column{}, common.DefaultDynamicFieldName)`` (lines 193-210).

*The inserted rows still include real dynamic columns (because ``hp.CollPrepare.InsertData`` automatically appends ``hp.GenDynamicColumnData``).

As a result, the expected JSON (``{}`` for every row) will never match the actual dynamic payload (``dynamicNumber``, ``dynamicString``, etc.). This test will fail as soon as CI runs it. The previous code (now commented out) that merged the generated dynamic columns was correct; it should be restored.

2. **“Invalid search params” tests now expect success instead of failure**

In ``TestSearchInvalidSearchParams`` (lines 521-544) the new loops call ``common.CheckErr(t, err, true, "No error for invalid search params")`` even though the comments say the calls *should* error. Passing ``true`` to ``CheckErr`` asserts that no error occurred, so any server-side validation will make the test fail for the wrong reason. The same mistake appears again when searching with ``ef=7``, which is explicitly documented as invalid (lines 539-544). The expectation flags need to be flipped to ``false``.

3. ****SCANN invalid-parameter test has the same logic bug****

`TestSearchInvalidScannReorderK` now asserts success for the invalid `reorder\_k < topK` case (lines 547-558). If the server correctly rejects that request, this test will fail. The expectation should be `common.CheckErr(t, errScann, false, ...)`.

4. ****Verify that the un-skipped group-by binary test is actually expected to pass****

`TestSearchGroupByBinaryGrowing` still asserts the server should *reject* binary vector group-by (lines 297-327). It was previously skipped for issue #38343; if that server issue is still open, the test will immediately fail. Confirm with the feature owner before turning it on in CI.

Addressing the three logic issues above (dynamic field expectations and the two invalid-search-param tests) is necessary before the patch can be considered safe to merge; otherwise the newly enabled tests will either fail outright or assert the opposite of the intended behavior.

```
=====
RUN 2/3
=====
### What the patch really does
```

1. ****Enables the Go client to exercise per-search overrides****

All of the newly-unskipped tests now call `client.NewSearchOption(...).WithSearchParam(key, value)` so the SDK pushes arbitrary key/value pairs down to the server. Internally (`client/milvusclient/read\_options.go`, lines 115-176, 230-276) these keys are merged into the `searchParam` map that becomes the `SearchRequest.SearchParams` payload. Those entries override the defaults that were previously hard-wired by the index definition (e.g., HNSW `ef`, IVF `nprobe`, SCANN's `reorder\_k`, sparse range parameters such as `drop\_ratio\_search`, `radius`, `range\_filter`, and even `metric\_type`). Until now that code path existed but none of the regression tests actually drove it.

2. ****Re-enables the Go test suites that used to be blocked on "missing search param support."****

* `tests/go\_client/testcases/groupby\_search\_test.go` – the float/cosine group-by suites and the binary growing suite now run (lines 92, 150, 298), `TestSearchGroupByFloatGrowing` / `...Pagination` are also marked `t.Parallel` (lines 329 and 383), and the "range search + group by" negative test now explicitly sends `radius` / `range\_filter` even though it still skips via issue #38846 (lines 442-455).

* `tests/go\_client/testcases/search\_test.go` – the previously skipped blocks (`TestSearchNotMatchMetricType`, `TestSearchInvalidSearchParams`, `TestSearchInvalidScannReorderK`, `TestSearchScannAllMetricsWithRawData`, `TestRangeSearchSparseVector`) are now active and drive the various `WithSearchParam` combinations (lines 468-1112). This not only covers metric mismatch but also validates invalid `ef`, `nprobe`, SCANN's `reorder\_k`, sparse range searches, and the SCANN `with\_raw\_data` flag across all supported metrics.

* `tests/go\_client/testcases/query\_test.go` – `TestQueryOutputFields` is no longer skipped (lines 167-236) so the client's `\$meta` / dynamic-field handling is now exercised.

3. ****Small cleanup around logging and skipping****

* Query test logging switched from `log.Debug` to `log.Info` when printing the field payloads (line 201).

* The remaining range-search tests are now tied to the tracking issue (#38846) rather than the old “waiting for support” string.

Ripple across the Go client surface

* **Search parameter API** – WithSearchParam is now used in real tests for:

- Metric overrides (``metric_type``), HNSW tuning (``ef``), IVF (``nprobe``), SCANN's runtime controls (``reorder_k``, ``with_raw_data``), sparse range parameters (``drop_ratio_search``, ``radius``, ``range_filter``), and even restricting group-by + range search.
- These tests indirectly verify the SDK's ability to merge those overrides into the proto request via ``annRequest.searchRequest()`` (see ``client/milvusclient/read_options.go``, lines 115-176).

* **Group-by search** – The float, cosine, binary, growing, and pagination scenarios now execute end-to-end (tests/go_client/testcases/groupby_search_test.go, lines 92-404). They all rely on the search-param surface to keep index-specific knobs in sync (e.g., HNSW on growing segments). Removing the skip means any regression in per-search overrides will now fail these suites.

* **Query dynamic output handling** – Because ``TestQueryOutputFields`` now runs, the SDK's logic that maps “unknown” output field names to ``$meta`` slices (``client/milvusclient/read.go``, lines 109-175) finally has coverage.

Issues to flag before approving

1. **The “invalid search param” tests currently assert success instead of failure.**

* In ``TestSearchInvalidSearchParams`` and ``TestSearchInvalidScannReorderK``, every negative case calls ``common.CheckErr(t, err, true, "No error for invalid search params")`` (search_test.go lines 525-538 and 559-567). ``expErrNil=true`` tells ``CheckErr`` to require **no error**, so the test would happily pass even if the server silently accepted illegal ``ef``, ``nprobe``, or ``reorder_k`` values. The intent (per the comments) is the opposite. Please flip those expectations to ``false``, or these tests will never catch the regressions they're meant to guard.

2. **``TestQueryOutputFields`` now asserts empty dynamic payloads even though the fixture inserts dynamic data.**

* The new expectation builds ``dynamicColumn := hp.MergeColumnsToDynamic(10, []column.Column{}, common.DefaultDynamicFieldName)`` (query_test.go lines 199-208), i.e., an array of ``{}`` documents. But the helper ``CollPrepare.InsertData`` still injects real dynamic columns whenever ``enableDynamic == true`` (see ``tests/go_client/testcases/helper/data_helper.go``, lines 206-236). The actual server response for ``WithOutputFields("aaa")`` should therefore wrap the real ``$meta`` JSON, not an empty map. This change either masks a server-side bug or makes the test incorrect; worth clarifying before we rely on it.

3. **Potential flakiness/perf regressions from enabling very heavy tests in parallel.**

* ``TestSearchGroupByFloatDefault``, ``...Cosine``, ``...BinaryGrowing``, ``...FloatGrowing``, ``...Pagination``, plus the SCANN and sparse range suites, all iterate over multiple index types and NQ batches (100×200 inserts, repeated per metric). Running them all (``t.Parallel``) increases total load/CPU significantly

compared to the previously skipped state. Make sure CI capacity and timeouts are adjusted accordingly, or be prepared for flakes/timeouts when multiple of these run concurrently.

Addressing (1) is critical because it undermines the safety net we're trying to add; (2) needs clarification so we don't bake an incorrect expectation into the suite; and (3) is operational but should be acknowledged before merging.

Once those are resolved/acknowledged, the patch does achieve its headline goal: the SDK's customizable search-parameter plumbing now has real test coverage, and the group-by/search suites are no longer artificially blocked.

=====

RUN 3/3

=====

****What changed beyond the headline****

***Group-by search tests* (`tests/go\_client/testcases/groupby\_search\_test.go`)**

- The three float-based group-by suites (`TestSearchGroupByFloatDefault`, `...Cosine`, and `...BinaryGrowing`) no longer short-circuit with `t.Skip` so they now execute end-to-end, exercising all supported float and binary index combinations (lines 90-329).
- The "growing" and pagination variants now run with `t.Parallel()` (lines 329-345, 382-389), so multiple heavy data-setup tests execute concurrently.
- The range-search case for group-by is still skipped, but the body now demonstrates how to attach per-search parameters via `.WithSearchParam("radius", ...).WithSearchParam("range\_filter", ...)` (lines 443-454).

***Query output-fields test* (`tests/go\_client/testcases/query\_test.go`)**

- `TestQueryOutputFields` is now active (skip removed at line ~165) and its expected dynamic-field column is built from an empty dynamic source (`hp.MergeColumnsToDynamic(10, []column.Column{}, "\$meta")` at lines 193-206). That change reflects the behavior when requesting a field name that only exists in `\$meta`: the SDK should return an empty column rather than synthetic data.

***Search tests* (`tests/go\_client/testcases/search\_test.go`)**

- The suite imports `strconv` to format numeric search parameters (line 6).
- Previously skipped tests now run and explicitly drive the search-parameter API:
 - * `TestSearchNotMatchMetricType` sets `metric\_type` through `.WithSearchParam("metric\_type", "L2")` to verify mismatch handling (lines 455-470).
 - * `TestSearchInvalidSearchParams` now issues searches that override `ef` and `nprobe` via `.WithSearchParam`, and also tries the "ef < topK" case (lines 508-540).
 - * `TestSearchInvalidScannReorderK` sets SCANN-specific `nprobe`/`reorder\_k` pairs (lines 542-567).
 - * `TestSearchScannAllMetricsWithRawData` now actually iterates over metric types and the `with\_raw\_data` toggle, pulling `\*` output fields (lines 570-598).
 - * `TestRangeSearchSparseVector` no longer skips; it attaches sparse-vector-specific parameters (`drop\_ratio\_search`, `radius`, `range\_filter`) and asserts the returned scores fall inside the requested window (lines 1061-1097).

Collectively these tests exercise the `milvusclient.SearchOption.WithSearchParam`` and `annRequest.searchParam`` plumbing defined in `client/milvusclient/read_options.go`` (lines 83-138 & 248-356). They demonstrate that callers can now provide arbitrary key/value overrides (metric type, ef, nprobe, SCANN knobs, sparse range-search knobs, etc.) when constructing `SearchOption`s`.

****Surface area affected****

- ****Go client API****: no code changes in the client package itself, but the `WithSearchParam`` fluent API (both on `searchOption`` and `annRequest``) is now covered by concrete tests using metric, ef/nprobe, SCANN, and sparse-range parameters.
- ****Test suites****: group-by search, query output fields, and multiple search behaviors (metric mismatch, invalid params, SCANN, sparse range search) now run by default and, for the group-by cases, in parallel. The sparse range-search test also now verifies score bounds.

****Issues to flag before approval****

1. ****Invalid-parameter tests still expect success****

In `TestSearchInvalidSearchParams`` and `TestSearchInvalidScannReorderK``, the comments say “invalid ... -> error”, but the assertions call `common.CheckErr(t, err, true, "No error for invalid search params")``, i.e. they require `err == nil`` (lines 522-567). That means these tests currently *pass* when no validation happens* and will immediately fail if Milvus starts returning the expected errors. If the intent is to ensure invalid parameters are rejected, `expErrNil`` should be `false`` and the expected error message should be asserted.

2. ****Redundant/typo'd assertion in sparse range-search test****

`TestRangeSearchSparseVector`` calls `common.CheckErr(t, errSearch, true)`` twice in a row right after issuing the range search (lines 1103-1106). The second invocation is redundant (and still reads the same `errSearch``). While harmless, it's a sign the test might have been copy/pasted without cleaning up; it's worth trimming to avoid confusion.

3. ****Increased parallel load for heavy group-by tests****

Adding `t.Parallel()`` to `TestSearchGroupByFloatGrowing`` and `TestSearchGroupByPagination`` (lines 329-345 and 382-389) means each test now spins up large collections (hundreds of inserts and index builds) concurrently. On constrained CI environments this could exacerbate timeouts or resource contention. If CI instability shows up, we may need to gate these behind runtime controls.

If the expectation for item (1) is indeed “should error”, correcting the `CheckErr`` flags is important before declaring the tests “unblocked”, otherwise they provide a false sense of coverage.

Model Evaluation

```
{  
  "points": 32.7,  
  "maxPoints": 65.0,
```

```
"score_percent_weight": 50,
"summary_weight": "32.7/65.0 points (50%)",
"eval_runs": 3,
"eval_runs_succeeded": 3,
"per_run_points": [
  35.0,
  30.0,
  33.0
],
"results": [
  {
    "id": "reasoning-69416",
    "weight": "penalty",
    "pass": false,
    "points": 0.0
  },
  {
    "id": "reasoning-69397",
    "weight": "critical",
    "pass": false,
    "points": 0.0
  },
  {
    "id": "reasoning-69407",
    "weight": "critical",
    "pass": true,
    "points": 3.0
  },
  {
    "id": "reasoning-69418",
    "weight": "penalty",
    "pass": false,
    "points": 0.0
  },
  {
    "id": "reasoning-69410",
```

```
"weight": "bonus",
"pass": false,
"points": 0.0
},
{
  "id": "reasoning-69409",
  "weight": "critical",
  "pass": true,
  "points": 3.0
},
{
  "id": "reasoning-69417",
  "weight": "penalty",
  "pass": false,
  "points": 0.0
},
{
  "id": "reasoning-69400",
  "weight": "critical",
  "pass": true,
  "points": 3.0
},
{
  "id": "reasoning-69403",
  "weight": "critical",
  "pass": true,
  "points": 3.0
},
{
  "id": "reasoning-69405",
  "weight": "critical",
  "pass": true,
  "points": 3.0
},
{
  "id": "reasoning-69402",
```

```
"weight": "critical",
"pass": true,
"points": 3.0
},
{
  "id": "reasoning-69406",
  "weight": "critical",
  "pass": true,
  "points": 3.0
},
{
  "id": "reasoning-69411",
  "weight": "bonus",
  "pass": false,
  "points": 0.0
},
{
  "id": "reasoning-69412",
  "weight": "bonus",
  "pass": false,
  "points": 0.0
},
{
  "id": "reasoning-69404",
  "weight": "critical",
  "pass": true,
  "points": 3.0
},
{
  "id": "reasoning-69399",
  "weight": "critical",
  "pass": false,
  "points": 0.0
},
{
  "id": "reasoning-69413",
```

```
"weight": "bonus",
"pass": false,
"points": 0.0
},
{
  "id": "reasoning-69401",
  "weight": "critical",
  "pass": false,
  "points": 0.0
},
{
  "id": "reasoning-69398",
  "weight": "critical",
  "pass": false,
  "points": 0.0
},
{
  "id": "reasoning-69414",
  "weight": "bonus",
  "pass": false,
  "points": 0.0
},
{
  "id": "reasoning-69396",
  "weight": "critical",
  "pass": true,
  "points": 3.0
},
{
  "id": "reasoning-69415",
  "weight": "penalty",
  "pass": true,
  "points": -1.0
},
{
  "id": "reasoning-69408",
```

```
"weight": "critical",
"pass": true,
"points": 3.0
},
{
  "id": "reasoning-69420",
  "weight": "penalty",
  "pass": false,
  "points": 0.0
},
{
  "id": "reasoning-69419",
  "weight": "penalty",
  "pass": false,
  "points": 0.0
},
{
  "id": "completeness-69424",
  "weight": "critical",
  "pass": false,
  "points": 0.0
},
{
  "id": "completeness-69425",
  "weight": "critical",
  "pass": false,
  "points": 0.0
},
{
  "id": "completeness-69422",
  "weight": "critical",
  "pass": true,
  "points": 3.0
},
{
  "id": "completeness-69423",
```

```
"weight": "critical",
"pass": false,
"points": 0.0
},
{
  "id": "completeness-69421",
  "weight": "critical",
  "pass": true,
  "points": 3.0
},
{
  "id": "style-69429",
  "weight": "bonus",
  "pass": false,
  "points": 0.0
},
{
  "id": "style-69428",
  "weight": "bonus",
  "pass": false,
  "points": 0.0
},
{
  "id": "style-69430",
  "weight": "bonus",
  "pass": false,
  "points": 0.0
}
}
]
```

model eval score
difficulty

50.3
medium

3 Example Task 5354 Code Onboarding & Comprehension

Example Task 5354

Name: recharts-recharts-comprehension
Intent: Code Onboarding & Comprehension
Difficulty: Hard
Language: JavaScript

Repo

Repository: <https://github.com/recharts/recharts>

Checkout (commit): 979d921631364d33696d9a5b6f5fba561af5bf5e

Prompt

The src/chart has all the charts but I see they import and then all of them call a function. I am looking to contribute to this library and I thought each chart is its own mini world. But this is confusing me. Can you help me understand why and how this works?

LLM Prompt Review

Prompt QC Feedback

****Grading Item 1 (1/1)****

The question probes a structural pattern across multiple chart modules, which is more complex than a trivial lookup.

****Grading Item 2 (1/1)****

It reads like a natural peer-to-peer inquiry without artificial role-play cues.

****Grading Item 3 (1/1)****

It does not request elementary definitions or basic syntax knowledge.

****Grading Item 4 (1/1)****

Understanding why the charts import and call a shared function requires inspecting repository structure and code flow.

****Grading Item 5 (1/1)****

The prompt is formulated purely as a question and contains no inline code or pasted context.

****Grading Item 6 (0/1)****

The request is quite vague (“why and how this works”) without specifying which function or mechanism, making it difficult to identify a concrete, verifiable answer.

****Grading Item 7 (1/1)****

It aligns with the intent, focusing on understanding repository architecture.

****Grading Item 8 (1/1)****

It invites reasoning about design and behavior rather than a simple factual lookup.

****Grading Item 9 (1/1)****

There are no meta-instructions or formatting directives.

****Grading Item 10 (1/1)****

Only ASCII characters are used.

****Grading Item 11 (1/1)****

The prompt targets a single intent—understanding the architectural pattern for charts.

FINAL_SCORE: 10

Rubric

ID	tag	description	weight	rationale	dependent_on
33993	reasoning	Identifies <code>`generateCategoricalChart`</code> as a factory function	critical	Recognising the factory pattern is the absolute foundation for understanding the codebase and the library	
34002	reasoning	Explains that individual chart components act as thin wrappers or configuration objects	critical	Demonstrates an understanding of dependency injection at the chart level	33993
34005	reasoning	Describes <code>`getDisplayedData`</code> as determining the currently visible slice of data	critical	Data slicing is a core preliminary step before any rendering or coordinate math can occur	
34006	reasoning	States that <code>`getTooltipData`</code> translates raw UI coordinates into resolved tooltip state objects	critical	Understanding the translation of raw DOM events to React state is required for component interactivity	
34007	reasoning	Identifies <code>`getAxisMapByAxes`</code> as the primary function computing complete axis domain configurations	critical	Axis computation is the most complex logic in the file and identifying the primary driver is essential	
34008	reasoning	Notes that <code>`getAxisMapByAxes`</code> takes a calculation shortcut if the user explicitly specifies the axis range	critical	Separating explicit user configuration from implicit data calculation is necessary for a complete understanding	34007

34049	reasoning	Names <code>`getAxisMap`</code> as the router mechanism between the two axis generation strategies	critical	This is the core flow control mechanism for axis generation	
34050	reasoning	Identifies <code>`getAxisNameByLayout`</code> as defining which axis is numeric and which is categorical based on chart layout	critical	This mapping is the foundational truth for layout, stack groups and formatting across the file	
34051	reasoning	Describes <code>`calculateOffset`</code> as isolating the central drawable plotting area	critical	Required to understand how the final rendering boundaries are defined apart from peripheral elements	
34054	reasoning	Explains that <code>`getMouseInfo`</code> translates raw browser screen coordinates into chart relative coordinates	critical	Custom event coordinate translation is a high risk area for interaction bugs and requires thorough understanding	
34055	reasoning	Explains that <code>`getAxisMapByAxes`</code> stretches axis boundaries to accommodate extra markers like reference areas	bonus	Shows comprehensive understanding of how peripheral visual elements affect core chart layout math	
34057	reasoning	Identifies <code>`updateStateOfAxisMapsOffsetAndStackGroups`</code> as the core state recalculation pipeline	critical	This is the engine function that drives the entire data-to-visual transformation sequence	

34058	reasoning	States that <code>`tooltipTicksGenerator`</code> produces arrays sorted by pixel coordinate for efficient nearest tick scanning	bonus	Explains the underlying performance optimization used for the tooltip hover mechanism	
34059	reasoning	Describes <code>`hasGraphicalBarItem`</code> as dynamically determining the use of <code>`scaleBand`</code> along with <code>`scalePoint`</code>	bonus	Identifies how specific chart configurations dynamically alter the underlying mathematical scale functions	
34070	reasoning	Advises trusting the native browser X and Y mouse coordinates directly for tooltips	penalty	Actively bypasses the vital custom <code>`getMouseInfo`</code> translation math, which would cause broken UI bounds	
34071	reasoning	Asserts that <code>`AreaChart`</code> and <code>`BarChart`</code> possess completely distinct rendering logic engines	penalty	Directly contradicts the foundational factory pattern, misleading the developer	
34074	reasoning	Identifies <code>`getAxisMapByItems`</code> as generating a synthetic numeric axis for charts without explicit axis components	critical	Understanding the fallback rendering path is crucial for a complete view	
34075	completeness	Identifies <code>`CategoricalChartWrapper`</code> as the returned React class component	critical	The factory ultimately produces this class and ignoring it misses the entire React integration and lifecycle layer	

34076	completeness	Explains the wrapper component's reliance on a dispatch table ('renderMap') rather than rendering standard JSX children directly	critical	Explaining the custom render tree is vital for auditing how properties are injected into chart children	
34078	completeness	Details the logic inside 'getTooltipContent' for handling duplicate keys via 'findEntryInArray'	bonus	Demonstrates deep inspection of edge cases in the data lookup process	
34079	completeness	Mentions that 'CategoricalChartWrapper' establishes a mouse movement speed limit in its constructor	bonus	Highlights a critical performance and anti crash safe guard built into the UI layer	34075
34107	completeness	Suggests removing or bypassing the Redux 'RechartsStoreProvider' wrapper	penalty	Actively breaks the state management context required by the library at the highest level of the component	
34108	reasoning	Claims that 'getOffset' used by 'getMouseInfo' does not account for any borders on the '<html>' element	penalty	The function also subtracts document.documentElement.clientTop/clientLeft which makes this safe for such rare cases, hence this is wrong.	
34112	style	Formats the explanation using structured hierarchical elements like markdown headers or numbered lists	bonus	Given the sheer volume of functions in the target file, structural formatting is essential for visual parsing	

34113	style	Presents inline code elements and function names using monospace markdown formatting	bonus	Maintains strict technical clarity by distinguishing variable/function names from explanation	
34123	reasoning	Claims that the GraphicalChild passed by every chart component contains the tooltip logic for that chart.	penalty	Tooltip logic is the same for all charts and nothing to do with the GraphicalChild passed by every chart	
34130	completeness	Mentions <code>`calculateActiveTickIndex`</code> as a dependent function called within the <code>`getTooltipData`</code> execution flow	bonus	Shows the understanding of the internal dependency chain of the tooltip coordinate resolution	
34131	completeness	Identifies the specific array method <code>`data.slice()`</code> utilized within <code>`getDisplayedData`</code>	bonus	Highlights the specific and safe JavaScript operation used to isolate the visible data segment	
34140	reasoning	Identifies that <code>`getAxisMapByAxes`</code> adds hidden sequence numbers to deduplicate specific data connections	bonus	It checks you saw how duplicate categories get stable ordering inside <code>getAxisMapByAxes</code> before domain math, which is easy to miss when you only explain the factory.	
34141	reasoning	Mentions that <code>`getTooltipContent`</code> explicitly filters out null entries before returning the final payload array	bonus	Stating that <code>getTooltipContent</code> removes empty series slots from the merged payload so the tooltip list matches real series only, which is finer than the main hover and state pipeline.	
34142	completeness	States that <code>`calculateActiveTickIndex`</code> uses the sorted tick array to perform an efficient left-right comparison scan	bonus	Specific algorithmic approach used for the nearest neighbour hover detection	

34143	reasoning	Notes that the wrapper pre-populates the default tooltip during the mount phase if a `defaultIndex` is provided	bonus	Captures a specific initialization behavior of the interactive tooltip	
35755	completeness	Mentions that `renderByOrder` walks through the JSX children in order applying the right handler from `renderMap`	critical	This function is important to preserve the DOM order while optionally enforcing that certain component types only render once.	
35860	completeness	Mentions that `getMouseInfo` handovers to `handleMouseMove` `handleMouseEnter` or `handleMouseLeave` handlers after getting the coordinates	bonus	Raw browser interactions and peripheral device interactions are natively handled after coordinate computation	

LLM Rubric Review

Rubric QC Feedback

****Grading Item 1:** The rubric is organized into exactly three top-level evaluation categories: `reasoning`, `completeness`, and `style`. All three categories are present. (2/2)**

Rubric uses only those three top-level keys.

****Grading Item 2:** The rubric contains a reasonable number of grading items for the task complexity and covers the task holistically. (2/2)**

There are 34 total grading items, which is within the suggested range and reflects the complex golden answer.

****Grading Item 3: Every grading item is atomic and self-contained, evaluating exactly one distinct point. (2/2)****

Each description targets a single behavior or fact; no stacked multi-part checks were found.

****Grading Item 4: Grading item descriptions are clear, specific, and measurable. (2/2)****

Descriptions name concrete functions/behaviors, making evaluation unambiguous.

****Grading Item 5: Every grading item is derivable from the provided `golden\_answer`. (0/2)****

Penalty item `reasoning-34108` references `getOffset` handling `` borders, which the golden answer never mentions, so that requirement is not derivable.

****Grading Item 6: The rubric is comprehensive relative to the `golden\_answer`. (2/2)****

All major concepts from the golden answer—factory pattern, axis builders, tooltip flow, wrapper lifecycle—are represented.

****Grading Item 7: The rubric would award full marks to the provided `golden\_answer`. (2/2)****

The golden answer satisfies every critical item and triggers no penalties, so it would earn full credit.

****Grading Item 8: The rubric is fair to valid alternative reasoning paths. (2/2)****

Although detailed, items allow any repo-grounded explanation covering the same mechanics, not just the golden answer's wording.

****Grading Item 9: The `reasoning` category evaluates correctness of reasoning, technical judgment, conceptual understanding, and explanation. (2/2)****

Reasoning items all target understanding of internal mechanisms and why they behave as described.

****Grading Item 10: The `completeness` category evaluates whether the response addresses all major parts of the question. (2/2)****

Completeness items focus on covering structural/rendering flow pieces required by the prompt.

****Grading Item 11: The `style` category evaluates clarity, organization, readability, and professionalism. (2/2)****

Style items concern formatting and code-style presentation, not technical content.

****Grading Item 12: Grading items are placed in appropriate categories. (2/2)****

Each item aligns with its category's focus; no misplaced requirements observed.

****Grading Item 13: The rubric includes the task-wide expectations that should always be present: reasoning quality, completeness, and code style. (2/2)****

Each of the three categories contains relevant grading items covering these expectations.

****Grading Item 14: Intent-specific grading items are included when required. (2/2)****

The task is an architectural explanation; rubric items appropriately cover factory mechanics, tooltips, and axis logic without omitting necessary intent-specific checks.

****Grading Item 15: Weights use only valid values and are well calibrated. (2/2)****

Weights are limited to `critical`, `bonus`, or `penalty`, and their usage matches the importance of each requirement.

****Grading Item 16: Penalty items target genuinely harmful or misleading behavior and are not simple negations. (2/2)****

All penalties call out concrete harmful misconceptions rather than just negating positive requirements.

****Grading Item 17: Dependency usage is logically sound. (2/2)****

Dependencies appear only where one item relies on another prerequisite, and the directions are appropriate.

****Grading Item 18: The rubric is not overprescriptive. (1/2)****

While comprehensive, the large number of critical items tied to very specific functions risks penalizing otherwise solid explanations that focus on higher-level architecture, making it somewhat overprescriptive.

****Grading Item 19: Each grading item contributes meaningfully to evaluating answer quality. (2/2)****

Items cover distinct, valuable aspects of understanding the chart factory, with minimal redundancy.

FINAL_SCORE: 35

Golden Answer

`AreaChart`, `BarChart`, `LineChart` etc all depend on a factory function `generateCategoricalChart`. This function takes a chart configuration as an argument and returns a React component `CategoricalChart`. Simply put, all the charts are just thin wrappers around the component this function produces.

This means, every chart type is just a configuration, not a separate codebase. All the charts share 100% of the same logic defined once. Each chart only declares what makes it unique. Without the factory, adding a bug fix would require changing separate chart files instead of one. The factory is essentially dependency injection at the chart level.

The file `src/chart/generateCategoricalChart.tsx` has all the required code along with some imports for helpers and utils. It exports among all the other exports, `generateCategoricalChart` which is further imported by all the individual chart files. The file can be broken down into the following major sections:

1. Imports and Constants (L1 - L125)
2. Data Helpers (L143 - L260)

3. Axis Map Builders (L275 - L613)
4. State Initialiser (L621 - L647)
5. Layout Helpers (L649 - L752)
6. Types and Props (L754 - 800)
7. Factory Function (L802 - L1965)

Important Helper and Build Functions

1. ``getDisplayedData`` (L143) determines which slice of data is currently visible in the chart, respecting two different data sources and the brush window. It is used by functions like ``getAxisMapByAxes`` to know how many data points are visible on this axis, inside ``getFormatItems`` to pass the right displayed data to each series and inside ``getTooltipContent`` to look up the payload at the active index. This function uses ``data.slice()`` which is safe.

2. ``getTooltipContent`` (L179) builds an array of tooltip payload entries, one per graphical series, for the currently hovered data point. It first calls ``getDisplayedData`` to get the current visible data slice, checks for ``activeIndex`` returning ``null`` if invalid, and for each graphical item:

- a. Determines which data source to use for that item.
- b. Applies the brush window slice to that data.
- c. Uses ``findEntryInArray`` to look up by ``activeLabel`` value if duplicates are not allowed or uses ``activeIndex`` if duplicates are allowed or there is no ``dataKey``
- d. Calls ``getTooltipItem(child, payload)`` to build the ``{name, value, color, dataKey, ...}`` entry.

It finally filters out any ``null`` entries.

This function is called by ``getTooltipData`` during mouse movements to build the full tooltip state, ``applySyncEvent`` when a synced chart broadcasts its tooltip position to this chart and ``displayDefaultTooltip`` to pre populate the tooltip at mount with ``defaultIndex``.

3. ``getTooltipData`` (L233) converts a raw mouse or touch coordinate into a fully resolved tooltip state object. It first uses the ``rangeObj {x, y}`` (or ``state.chartX`` and ``state.chartY``) and passes them to ``calculateTooltipPos(rangeData, layout)`` which extracts the single relevant coordinate (x for horizontal, y for vertical, angle for polar). The return value is a number which is passed to ``calculateActiveTickIndex`` that calculates the ``activeIndex`` closest to the ``pos``. If ``activeIndex`` is zero or greater (and ``tooltipTicks`` is present), ``activeLabel``, ``activePayload`` and ``activeCoordinate`` are fetched and returned, else ``null`` is returned. It is called by ``getMouseInfo`` and ``getDerivedStateFromProps``.

4. ``getAxisMapByAxes`` (L275) is the primary axis configuration builder. Given a set of ``<XAxis>`` or ``<YAxis>`` React elements, it produces a map of ``axisId`` which is complete axis config object with computed domain. This is the most complex function in the file. It uses a decision tree to compute ``domain`` for each axis element which works as follows:

- a. First, it checks if the user explicitly told it what the axis range should be. If yes, it takes a performance shortcut by just using what the user specified and completely skips scanning the actual data.

- b. If the above step fails (user did not explicitly tell), then it has to figure it out from the data. To do this, it checks whether the axis is tied to a specific piece of data:

- i. If it has a specific data connection: The function does some deduplication by adding hidden sequence numbers, filters out empty or null values and check sif the data includes error bars and stretches the axis limits so they don't get cut off.

- ii. If it does not have a specific data connection: It defaults to assigning basic sequence numbers based on how many items are there, calculates the total range needed to fit the combined, stacked groups (if any) and it defaults to looking at all the chart items sharing that axis to find the lowest and highest values.

- c. Once the system has calculated the core range, it does two final checks before drawing the axis:

- i. It checks if you added extra markers, like a target line or a shaded reference area and if these fall outside the current range, it stretches the axis boundaries to make sure they are visible.

- ii. It applies any specific minimum or maximum limits the user might have provided to fine tune the final result.

5. ``getAxisMapByItems`` (L459) is the fallback axis builder. It is used when the user adds ``<Line>`` or ``<Bar>`` but does not add any explicit ``<XAxis>`` or ``<YAxis>`` children. It creates a synthetic, hidden axis configuration based purely on what the graphical items need. They are never visually rendered, just used for scale computation. The key difference with ``getAxisMayByAxes`` is ``getAxisMapByItems`` does not try to compute any complex domain types, but it just needs a numeric domain that is good enough to render the chart without explicit axis elements.

6. ``getAxisMap`` (L555) is the single entry point for axis map construction. It is a router between the two strategies ``getAxisMapByAxes`` and ``getAxisMapByItems``. It simply checks if the axes are present and routes the rendering of the axis to either of the functions. If no graphical items also exists, it simple returns a blank object.

7. ``tooltipTicksGenerator`` (L605) extracts tooltip tick coordinates from the categorical axis map. It returns two arrays, ``tooltipTicks`` which preserves the data order index and

``orderedTooltipTicks`` which is sorted by pixel coordinate, enabling ``calculateActiveTickIndex`` to do an efficient left-right comparison scan to find the nearest tick to the mouse. The result is merged into component state in ``updateStateOfAxisMapsOffsetAndStackGroups``. The three fields are read directly in ``getTooltipData`` and ``applySyncEvent``.

8. ``createDefaultState`` (L621) creates a clean initial state object for the chart. It is called on the first render and whenever the chart needs to fully reset.

9. ``hasGraphicalBarItem`` (L649) returns ``true`` if any graphical item's display name contains "Bar" i.e. if the chart has any ``<Bar>`` or ``<RadialBar>`` component. Bar charts need ``scaleBand`` where each category takes equal space, so this function is used to determine if ``scaleBand`` is needed or ``scalePoint`` does the job.

10. ``getAxisNameByLayout`` (L661) returns a ``{numericAxisName, cateAxisName}`` pair telling the rest of the code, which axis role is which, based on chart layout. The entire axis map construction, stack group computation and ``getFormatItems`` rely on knowing which axis carries numeric data and which carries categories. This function is the single source of truth of that mapping, changing it would flip axes for the whole chart. It is called at the very top of both ``updateStateOfAxisMapsOffsetAndStackGroups`` and ``getFormatItems``.

11. ``calculateOffset`` (L684) is the last function before the main factory function in this file. It calculates the drawable area of the chart which is the rectangle inside which data is actually plotted by subtracting all peripheral elements from the total chart dimensions.

``generateCategoricalChart`` Factory Function

The function receives the following configuration options:

```
``ts
{
  chartName,          // e.g. "AreaChart", "BarChart"
  GraphicalChild,     // e.g. Area, Bar, Line
  defaultTooltipEventType, // 'axis' or 'item'
  validateTooltipEventTypes,
  axisComponents,     // axis types this chart supports
  formatAxisMap,      // function to finalize axis scales
  defaultProps,       // chart-specific prop defaults
  tooltipPayloadSearcher
}
```

...

There are functions within and some important ones are described as follows:

1. ``getFormatItems`` (L812) loops through each graphical child and computes its display data like positions, stacked data, bar sizes, tick arrays by calling ``item.type.getComposedData()``. The results are stored as ``formattedGraphicalItems`` in state.
2. ``updateStateOfAxisMapsOffsetAndStackGroups`` (L934) is the core state recalculation pipeline. It runs whenever data or layout or children change.
 - a. Finds all graphical items.
 - b. Computes stack groups
 - c. Builds axis maps
 - d. Calculates the chart offset.
 - e. Applies ``formatAxisMap`` to finalise D3 scales on each axis.
 - f. Generates tooltip ticks
 - g. Calls ``getFormatItems`` to get the final positioned data for each series.
3. ``CategoricalChartWrapper`` (L1020) is the React class component. It is the main chart component. The lifecycle is as follows:
 - i. ``constructor`` and ``componentWillUnmount``: When the chart is first created, it sets up an ID for cropping the chart area and creates a speed-limit for mouse movements so the browser doesn't crash when the cursor is moved around. When the chart is removed, it cleans up those listeners so it does not leave junk behind in your browser's memory.
 - ii. ``getDerivedStateFromProps``: This is the engine. Whenever the data changes, window is resized, or a prop is tweaked, this method catches it. It recalculates everything, where the axes go, how things stack, and what needs to change.
 - iii. ``componentDidMount``, ``componentDidUpdate``, ``getSnapshotBeforeUpdate``: These handle the finer details. They make sure the default tooltip pops up, keep the accessibility tools (like screen readers) in sync with visual changes, and listen for syncing events from other charts.

When a click, touch or drag events across the chart are detected, all of that action funnels through one master method called ``getMouseInfo()``. Instead of trusting the browser's raw X/Y coordinates, ``getMouseInfo()`` does its own math. It grabs the exact boundaries of the chart container, translates screen coordinates into chart coordinates, checks to make sure the cursor is actually inside the active chart area. It finds the nearest tick mark on the axis and figures out what data point belongs to it if the hover is

over a general chart or it grabs the raw coordinates if hovering over a specific bar or dot. It then hands it off to the right handler, `handleMouseMove`` `handleMouseEnter`` or `handleMouseLeave``.

Rather than rendering `{children}` directly, the component uses a dispatch table (`renderMap``). `renderByOrder`` walks the JSX children in order and applies the right handler from this map.

The factory ultimately returns this thin functional wrapper that:

1. Creates a Redux store options (`ChartOptions``)
2. Wraps `CategoricalChartWrapper`` in `RechartsStoreProvider``

Golden Answer Review

Golden Answer QC Feedback

****Grading Item 1:**** The `golden_answer`` is internally consistent and technically accurate. It does not contradict itself, make unsupported claims, or rely on incorrect technical reasoning. If code snippets or concrete fixes are included, they should be coherent and should not obviously reference undefined variables, missing imports, or broken logic. (3/3)

All described behaviors and functions align logically, with no contradictions or implausible mechanics detected.

****Grading Item 2:**** The `golden_answer`` is clearly grounded in repo exploration rather than generic software advice. It references specific files, functions, modules, call flows, code locations, or concrete repo behaviors where appropriate. (3/3)

It cites `src/chart/generateCategoricalChart.tsx``, enumerates line ranges, and explains concrete functions/calls throughout the repo.

****Grading Item 3:**** The ``golden_answer`` directly answers the question asked in the ``prompt_statement`` without drifting into unrelated explanation or omitting the central conclusion. (2/2)

It explicitly clarifies why charts share a factory and how that structure works, matching the prompt.

****Grading Item 4:**** The ``golden_answer`` explains the reasoning chain behind its conclusion. It shows how the answer follows from the observed repo behavior, architecture, data flow, control flow, change impact, or causal chain, rather than simply stating conclusions. (3/3)

Each described function includes causal explanations of how data flows and why behaviors occur.

****Grading Item 5:**** Technical explanations are precise and mechanistically accurate, avoiding vague phrases like "causes issues" or "creates confusion" when a concrete explanation of what happens is needed. (2/2)

Mechanisms (e.g., tooltip resolution, axis range shortcuts) are spelled out concretely.

****Grading Item 6:**** Technical details are factually accurate for the repo context, framework behavior, language behavior, and any stated versions or assumptions. (2/2)

Descriptions match Recharts' architecture and do not introduce incorrect assumptions.

****Grading Item 7:**** The `golden\_answer` satisfies all critical grading items in the associated rubric. Every must-have requirement identified in the rubric is addressed. (3/3)

All critical reasoning/completeness points (factory pattern, axis routing, wrapper class, renderMap, etc.) are covered.

****Grading Item 8:**** The `golden\_answer` avoids all penalty behaviors identified in the associated rubric. It does not include harmful, misleading, or clearly incorrect content that would trigger penalty grading items. (2/2)

No flagged penalty statements (e.g., distinct logic engines, bypassing store, trusting raw coords) appear.

****Grading Item 9:**** The `golden\_answer` is appropriately comprehensive for the prompt. It covers the major required points without being so thin that important reasoning or implications are missing. (2/2)

It offers broad coverage of helpers, factory, wrapper, and interaction flow.

****Grading Item 10:**** The `golden\_answer` remains understandable to a non-expert reader. It is clear, organized, and professional, and where the task calls for explanation-oriented writing, it uses instructive language that helps a new engineer follow the reasoning. (2/2)

Complex content is organized via sections and plain-language explanations.

****Grading Item 11:**** Where appropriate, the `golden\_answer` includes concrete supporting details rather than only abstract statements. Examples may include file paths, function names, call sequences, inline code snippets, configuration details, or exact code-level implications. (2/2)

It references specific functions, line ranges, configuration objects, and call flows.

****Grading Item 12:**** The `golden\_answer` follows the intent-specific expectations for the task. (3/3)

As a repo comprehension response, it emphasizes architecture and behavior explanations without proposing fixes, satisfying intent.

****Grading Item 13:**** Not applicable (non-PR task). (2/2)

Automatically awarded full credit.

****Grading Item 14:**** The `golden\_answer` does NOT contain non-ASCII characters except when necessary. (1/1)

Only ASCII characters are used.

FINAL_SCORE: 32

Validate Golden Answer

Golden Answer Validation

```
{
  "results": [
    {
      "id": "reasoning-34071",
      "tag": "reasoning",
      "weight": "penalty",
      "description": "Asserts that `AreaChart` and `BarChart` possess completely distinct rendering logic engines",
      "rationale": "Directly contradicts the foundational factory pattern, misleading the developer",
      "dependent_on": [],
      "pass": true,
      "evidence": "\"All the charts share 100% of the same logic defined once.\""
```


"reasoning": "The response explicitly states that AreaChart and BarChart reuse the same factory-generated logic, directly contradicting any notion that they have separate rendering engines."

},

{

"id": "reasoning-34055",

"tag": "reasoning",

"weight": "bonus",

"description": "Explains that `getAxisMapByAxes` stretches axis boundaries to accommodate extra markers like reference areas",

"rationale": "Shows comprehensive understanding of how peripheral visual elements affect core chart layout math",

"dependent_on": [],

"pass": true,

"evidence": "it checks if you added extra markers, like a target line or a shaded reference area and if these fall outside the current range, it stretches the axis boundaries to make sure they are visible.",

"reasoning": "The golden answer explicitly states that `getAxisMapByAxes` adjusts axis boundaries when extra markers such as reference areas lie outside the initial range, satisfying the criterion."

},

{

"id": "reasoning-34049",

"tag": "reasoning",

"weight": "critical",

"description": "Names `getAxisMap` as the router mechanism between the two axis generation strategies",

"rationale": "This is the core flow control mechanism for axis generation",

"dependent_on": [],

"pass": true,

"evidence": "`getAxisMap` is the single entry point for axis map construction. It is a router between the two strategies `getAxisMapByAxes` and `getAxisMapByItems`.",

"reasoning": "The answer explicitly states that `getAxisMap` serves as the routing mechanism between the two axis generation strategies, directly satisfying the criterion."

},

{

"id": "reasoning-34008",

"tag": "reasoning",

"weight": "critical",

"description": "Notes that `getAxisMapByAxes` takes a calculation shortcut if the user explicitly specifies the axis range",

"rationale": "Separating explicit user configuration from implicit data calculation is necessary for a complete understanding",

"dependent_on": [

"reasoning-34007"

],

"pass": true,

"evidence": "\u201ca. First, it checks if the user explicitly told it what the axis range should be. If yes, it takes a performance shortcut by just using what the user specified and completely skips scanning the actual data.\u201d",

"reasoning": "The answer explicitly states that getAxisMapByAxes bypasses data scanning when a user supplies an axis range, demonstrating the required understanding."

},

{

"id": "reasoning-34002",

"tag": "reasoning",

"weight": "critical",

"description": "Explains that individual chart components act as thin wrappers or configuration objects",

```

    "rationale": "Demonstrates an understanding of dependency injection at the chart
level",
    "dependent_on": [
        "reasoning-33993"
    ],
    "pass": true,
    "evidence": "\u201cAreaChart, BarChart, LineChart etc all depend on a factory
function `generateCategoricalChart`. \u2026 all the charts are just thin wrappers around
the component this function produces.\u201d",
    "reasoning": "The answer explicitly states that each chart is a thin
wrapper/configuration around the generated component, directly fulfilling the criterion."
},
{
    "id": "reasoning-34059",
    "tag": "reasoning",
    "weight": "bonus",
    "description": "Describes `hasGraphicalBarItem` as dynamically determining the use
of `scaleBand` and `scalePoint`,
    "rationale": "Identifies how specific chart configurations dynamically alter the
underlying mathematical scale functions",
    "dependent_on": [],
    "pass": true,
    "evidence": "`hasGraphicalBarItem` ... returns `true` if any graphical item's display
name contains `Bar` ... Bar charts need `scaleBand` where each category takes equal
space, so this function is used to determine if `scaleBand` is needed or `scalePoint`
does the job.",
    "reasoning": "The answer explicitly explains that `hasGraphicalBarItem` detects bar
components and determines whether to use `scaleBand` versus `scalePoint`, satisfying
the criterion."
},

```

```
{
  "id": "reasoning-34005",
  "tag": "reasoning",
  "weight": "critical",
  "description": "Describes `getDisplayedData` as determining the currently visible slice
of data",
  "rationale": "Data slicing is a core preliminary step before any rendering or coordinate
math can occur",
  "dependent_on": [],
  "pass": true,
  "evidence": "`getDisplayedData` (L143) determines which slice of data is currently
visible in the chart, respecting two different data sources and the brush window.",
  "reasoning": "The answer explicitly states that `getDisplayedData` determines the
currently visible data slice, directly satisfying the criterion."
},
{
  "id": "reasoning-34057",
  "tag": "reasoning",
  "weight": "critical",
  "description": "Identifies `updateStateOfAxisMapsOffsetAndStackGroups` as the
core state recalculation pipeline",
  "rationale": "This is the engine function that drives the entire data-to-visual
transformation sequence",
  "dependent_on": [],
  "pass": true,
  "evidence": "\u201cupdateStateOfAxisMapsOffsetAndStackGroups` (L934) is the
core state recalculation pipeline. It runs whenever data or layout or children
change.\u201d",
```

```
"reasoning": "The golden answer explicitly labels  
`updateStateOfAxisMapsOffsetAndStackGroups` as the core state recalculation  
pipeline and describes its role, directly satisfying the criterion."  
},  
{  
  "id": "reasoning-34108",  
  "tag": "reasoning",  
  "weight": "penalty",  
  "description": "Claims that `getOffset` used by `getMouseInfo` does not account for  
any borders on the `` element",  
  "rationale": "The function also subtracts  
document.documentElement.clientTop/clientLeft which makes this safe for such rare  
cases, hence this is wrong.",  
  "dependent_on": [],  
  "pass": true,  
  "evidence": "The golden answer discusses `getMouseInfo()` but makes no mention  
of `getOffset`, nor does it claim that `getOffset` fails to account for borders on the  
`<html>` element.",  
  "reasoning": "Since the golden answer does not assert the incorrect behavior  
regarding `getOffset`, it avoids the penalized claim."  
},  
{  
  "id": "reasoning-34006",  
  "tag": "reasoning",  
  "weight": "critical",  
  "description": "States that `getTooltipData` translates raw UI coordinates into  
resolved tooltip state objects",  
  "rationale": "Understanding the translation of raw DOM events to React state is  
required for component interactivity",  
  "dependent_on": [],
```

"pass": true,

"evidence": "\u201c`getTooltipData` (L233) converts a raw mouse or touch coordinate into a fully resolved tooltip state object. It first uses the `rangeObj {x, y}`\u2026 and passes them to `calculateTooltipPos`\u2026 calculates the activeIndex\u2026 `activeLabel`, `activePayload` and `activeCoordinate` are fetched and returned, else `null` is returned.\u201d",

"reasoning": "The answer explicitly explains that `getTooltipData` takes raw mouse/touch coordinates and translates them into the tooltip state, describing the steps involved in this conversion. Therefore it clearly meets the criterion."

},

{

"id": "reasoning-34051",

"tag": "reasoning",

"weight": "critical",

"description": "Describes `calculateOffset` as isolating the central drawable plotting area",

"rationale": "Required to understand how the final rendering boundaries are defined apart from peripheral elements",

"dependent_on": [],

"pass": true,

"evidence": "\u201c`calculateOffset` (L684)\u2026 calculates the drawable area of the chart which is the rectangle inside which data is actually plotted by subtracting all peripheral elements from the total chart dimensions.\u201d",

"reasoning": "The answer explicitly states that `calculateOffset` computes the central drawable area by subtracting peripheral elements, directly meeting the criterion."

},

{

"id": "reasoning-34123",

"tag": "reasoning",

"weight": "penalty",

"description": "Claims that the GraphicalChild passed by every chart component contains the tooltip logic for that chart.",

"rationale": "Tooltip logic is the same for all charts and nothing to do with the GraphicalChild passed by every chart",

"dependent_on": [],

"pass": true,

"evidence": "The answer lists `GraphicalChild` only as part of the configuration object (`GraphicalChild`, // e.g. Area, Bar, Line`) and attributes tooltip handling to functions like `getTooltipContent`, `getTooltipData`, and `getMouseInfo`, showing tooltip logic is shared rather than tied to `GraphicalChild`.",

"reasoning": "Because the golden answer never claims that tooltip logic resides in the `GraphicalChild`, and instead explains tooltip behavior comes from shared helper functions, it does not violate the penalty criterion."

},

{

"id": "reasoning-34054",

"tag": "reasoning",

"weight": "critical",

"description": "Explains that `getMouseInfo` translates raw browser screen coordinates into chart relative coordinates",

"rationale": "Custom event coordinate translation is a high risk area for interaction bugs and requires thorough understanding",

"dependent_on": [],

"pass": true,

"evidence": "\u201cWhen a click, touch or drag events across the chart are detected, all of that action funnels through one master method called `getMouseInfo()`. Instead of trusting the browser's raw X/Y coordinates, `getMouseInfo()` does its own math. It grabs the exact boundaries of the chart container, translates screen coordinates into chart coordinates\u2026\u201d",

"reasoning": "The answer explicitly explains that `getMouseInfo` converts browser screen coordinates into chart-relative coordinates, directly satisfying the criterion."

},

{

"id": "reasoning-34074",

"tag": "reasoning",

"weight": "critical",

"description": "Identifies `getAxisMapByItems` as generating a synthetic numeric axis for charts without explicit axis components",

"rationale": "Understanding the fallback rendering path is crucial for a complete view",

"dependent_on": [],

"pass": true,

"evidence": "\u201c`getAxisMapByItems` (L555) is the fallback axis builder. It is used when the user adds `` or `` but does not add any explicit `` or `` children. It creates a synthetic, hidden axis configuration based purely on what the graphical items need. ... it just needs a numeric domain that is good enough to render the chart without explicit axis elements.\u201d",

"reasoning": "The golden answer explicitly notes that `getAxisMapByItems` handles cases without explicit axes by generating synthetic numeric axis configurations, fulfilling the criterion."

},

{

"id": "reasoning-34141",

"tag": "reasoning",

"weight": "bonus",

"description": "Mentions that `getTooltipContent` explicitly filters out null entries before returning the final payload array",

"rationale": "Data sanitization process within the tooltip generator is needed to filter any unwanted data",


```
"dependent_on": [],
"pass": true,
"evidence": "It finally filters out any `null` entries.",
"reasoning": "The golden answer explicitly states that `getTooltipContent` filters out
null entries before returning the payload array, demonstrating the required data
sanitization."
},
{
  "id": "reasoning-34050",
  "tag": "reasoning",
  "weight": "critical",
  "description": "Identifies `getAxisNameByLayout` as defining which axis is numeric
and which is categorical based on chart layout",
  "rationale": "This mapping is the foundational truth for layout, stack groups and
formatting across the file",
  "dependent_on": [],
  "pass": true,
  "evidence": "10. `getAxisNameByLayout` (L661) returns a `{numericAxisName,
cateAxisName}` pair telling the rest of the code, which axis role is which, based on chart
layout. The entire axis map construction, stack group computation and
`getFormatlItems` rely on knowing which axis carries numeric data and which carries
categories. This function is the single source of truth of that mapping, changing it would
flip axes for the whole chart.",
  "reasoning": "The golden answer explicitly states that `getAxisNameByLayout`
defines which axis is numeric versus categorical depending on the layout and highlights
its foundational role across the file."
},
{
  "id": "reasoning-34058",
  "tag": "reasoning",
```

```
"weight": "bonus",
  "description": "States that `tooltipTicksGenerator` produces arrays sorted by pixel
coordinate for efficient nearest tick scanning",
  "rationale": "Explains the underlying performance optimization used for the tooltip
hover mechanism",
  "dependent_on": [],
  "pass": true,
  "evidence": "`tooltipTicksGenerator` ... returns two arrays, `tooltipTicks` ... and
`orderedTooltipTicks` which is sorted by pixel coordinate, enabling
`calculateActiveTickIndex` to do an efficient left-right comparison scan to find the
nearest tick to the mouse.",
  "reasoning": "The answer explicitly states that `orderedTooltipTicks` is sorted by
pixel coordinate and explains that this sorting allows efficient nearest-tick scanning,
satisfying the criterion."
},
{
  "id": "reasoning-34143",
  "tag": "reasoning",
  "weight": "bonus",
  "description": "Notes that the wrapper pre-populates the default tooltip during the
mount phase if a `defaultIndex` is provided",
  "rationale": "Captures a specific initialization behavior of the interactive tooltip",
  "dependent_on": [],
  "pass": true,
  "evidence": "It is called by `getTooltipData` during mouse movements ... and
`displayDefaultTooltip` to pre populate the tooltip at mount with `defaultIndex`.",
  "reasoning": "The answer explicitly states that during mount, `displayDefaultTooltip`
uses `defaultIndex` to pre-populate the tooltip, matching the initialization behavior
described in the criterion."
},
```

```
{
  "id": "reasoning-34140",
  "tag": "reasoning",
  "weight": "bonus",
  "description": "Identifies that `getAxisMapByAxes` adds hidden sequence numbers to deduplicate specific data connections",
  "rationale": "These are the data transformation steps which are required for accurate axis plotting, without which duplicates will cause side effects",
  "dependent_on": [],
  "pass": true,
  "evidence": "It states, \"If it has a specific data connection: The function does some deduplication by adding hidden sequence numbers.\",",
  "reasoning": "The golden answer explicitly explains that getAxisMapByAxes adds hidden sequence numbers for deduplication when an axis is tied to a specific data connection, satisfying the criterion."
},
{
  "id": "reasoning-34007",
  "tag": "reasoning",
  "weight": "critical",
  "description": "Identifies `getAxisMapByAxes` as the primary function computing complete axis domain configurations",
  "rationale": "Axis computation is the most complex logic in the file and identifying the primary driver is essential",
  "dependent_on": [],
  "pass": true,
  "evidence": "\u201c`getAxisMapByAxes` (L275) is the primary axis configuration builder. Given a set of `` or `` React elements, it produces a map \u2026 This is the most complex function in the file.\u201d",
```

"reasoning": "The golden answer explicitly identifies `getAxisMapByAxes` as the primary function responsible for computing complete axis configurations and emphasizes its complexity, satisfying the criterion."

},

{

"id": "reasoning-33993",

"tag": "reasoning",

"weight": "critical",

"description": "Identifies `generateCategoricalChart` as a factory function",

"rationale": "Recognising the factory pattern is the absolute foundation for understanding the codebase and the library",

"dependent_on": [],

"pass": true,

"evidence": "`AreaChart`, `BarChart`, `LineChart` etc all depend on a factory function `generateCategoricalChart`.",

"reasoning": "The golden answer explicitly states that `generateCategoricalChart` is a factory function, which satisfies the criterion that it must identify the function as such."

},

{

"id": "reasoning-34070",

"tag": "reasoning",

"weight": "penalty",

"description": "Advises trusting the native browser X and Y mouse coordinates directly for tooltips",

"rationale": "Actively bypasses the vital custom `getMouseInfo` translation math, which would cause broken UI bounds",

"dependent_on": [],

"pass": true,

"evidence": "\u201cInstead of trusting the browser's raw X/Y coordinates, `getMouseInfo()` does its own math.\u201d",

"reasoning": "The golden answer explicitly rejects relying on native browser coordinates and explains that `getMouseInfo` performs its own coordinate translation, so it does not advise the harmful behavior."

},

{

"id": "completeness-34076",

"tag": "completeness",

"weight": "critical",

"description": "Explains the wrapper component's reliance on a dispatch table (`renderMap`) rather than rendering standard JSX children directly",

"rationale": "Explaining the custom render tree is vital for auditing how properties are injected into chart children",

"dependent_on": [],

"pass": true,

"evidence": "Rather than rendering `{children}` directly, the component uses a dispatch table (`renderMap`). `renderByOrder` walks the JSX children in order and applies the right handler from this map.",

"reasoning": "The answer explicitly explains that the wrapper component relies on a dispatch table (`renderMap`) and describes how `renderByOrder` processes the children via this table, satisfying the requirement to explain this custom rendering approach."

},

{

"id": "completeness-34075",

"tag": "completeness",

"weight": "critical",

"description": "Identifies `CategoricalChartWrapper` as the returned React class component",

"rationale": "The factory ultimately produces this class and ignoring it misses the entire React integration and lifecycle layer",

"dependent_on": [],

```

    "pass": true,
    "evidence": "\u201cCategoricalChartWrapper (L1020) is the React class component. It is the main chart component.\u201d",
    "reasoning": "The golden answer explicitly names CategoricalChartWrapper as the React class component produced by the factory, satisfying the requirement to identify it as the returned class."
  },
  {
    "id": "completeness-34142",
    "tag": "completeness",
    "weight": "bonus",
    "description": "States that `calculateActiveTickIndex` uses the sorted tick array to perform an efficient left-right comparison scan",
    "rationale": "Specific algorithmic approach used for the nearest neighbour hover detection",
    "dependent_on": [],
    "pass": true,
    "evidence": "\"`tooltipTicksGenerator` ... returns two arrays ... `orderedTooltipTicks` which is sorted by pixel coordinate, enabling `calculateActiveTickIndex` to do an efficient left-right comparison scan to find the nearest tick to the mouse.",
    "reasoning": "The answer explicitly states that `calculateActiveTickIndex` uses the sorted tick array (`orderedTooltipTicks`) to perform an efficient left-right comparison when determining the nearest tick, directly satisfying the criterion."
  },
  {
    "id": "completeness-35755",
    "tag": "completeness",
    "weight": "critical",
    "description": "Mentions that `renderByOrder` walks through the JSX children in order applying the right handler from `renderMap`",

```

"rationale": "This function is important to preserve the DOM order while optionally enforcing that certain component types only render once.",

"dependent_on": [],

"pass": true,

"evidence": "\u201cRather than rendering `{children}` directly, the component uses a dispatch table (`renderMap`). `renderByOrder` walks the JSX children in order and applies the right handler from this map.\u201d",

"reasoning": "The golden answer explicitly states that `renderByOrder` processes the JSX children sequentially and uses `renderMap` to apply the appropriate handler, directly satisfying the criterion."

},

{

"id": "completeness-34107",

"tag": "completeness",

"weight": "penalty",

"description": "Suggests removing or bypassing the Redux `RechartsStoreProvider` wrapper",

"rationale": "Actively breaks the state management context required by the library at the highest level of the component",

"dependent_on": [],

"pass": true,

"evidence": "\u201cThe factory ultimately returns this thin functional wrapper that: 1. Creates a Redux store options (`ChartOptions`) 2. Wraps `CategoricalChartWrapper` in `RechartsStoreProvider`.\u201d",

"reasoning": "The answer merely describes that the wrapper exists and does not suggest removing or bypassing it, so the harmful behavior is absent."

},

{

"id": "completeness-34079",

"tag": "completeness",

```
"weight": "bonus",
  "description": "Mentions that `CategoricalChartWrapper` establishes a mouse
movement speed limit in its constructor",
  "rationale": "Highlights a critical performance and anti crash safe guard built into the
UI layer",
  "dependent_on": [
    "completeness-34075"
  ],
  "pass": true,
  "evidence": "\u201c`constructor` and `componentWillUnmount`: When the chart is
first created, it sets up an ID for cropping the chart area and creates a speed-limit for
mouse movements so the browser doesn't crash when the cursor is moved
around.\u201d",
  "reasoning": "The golden answer explicitly states that the constructor creates a
speed limit for mouse movements, satisfying the requirement."
},
{
  "id": "completeness-34130",
  "tag": "completeness",
  "weight": "bonus",
  "description": "Mentions `calculateActiveTickIndex` as a dependent function called
within the `getTooltipData` execution flow",
  "rationale": "Shows the understanding of the internal dependency chain of the tooltip
coordinate resolution",
  "dependent_on": [],
  "pass": true,
  "evidence": "It first uses the `rangeObj {x, y}` ... passes them to `calculateTooltipPos
... The return value is a number which is passed to `calculateActiveTickIndex` that
calculates the activeIndex closest to the `pos`.",
```



```
    "reasoning": "The golden answer explicitly states that getTooltipData invokes  
calculateActiveTickIndex within its execution flow, satisfying the criterion."  
  },  
  {  
    "id": "completeness-34078",  
    "tag": "completeness",  
    "weight": "bonus",  
    "description": "Details the logic inside `getTooltipContent` for handling duplicate keys  
via `findEntryInArray`,  
    "rationale": "Demonstrates deep inspection of edge cases in the data lookup  
process",  
    "dependent_on": [],  
    "pass": true,  
    "evidence": "The answer states that `getTooltipContent` \"uses `findEntryInArray` to  
look up by `activeLabel` value is duplicates are not allowed or uses `activeIndex` if  
duplicates are allowed or there is no `dataKey`.\"",  
    "reasoning": "This explicitly explains how the function handles duplicate keys: it  
leverages `findEntryInArray` when duplicates are not allowed, and otherwise falls back  
to `activeIndex`, demonstrating the logic for duplicate handling."  
  },  
  {  
    "id": "completeness-34131",  
    "tag": "completeness",  
    "weight": "bonus",  
    "description": "Identifies the specific array method `data.slice()` utilized within  
`getDisplayedData`,  
    "rationale": "Highlights the specific and safe JavaScript operation used to isolate the  
visible data segment",  
    "dependent_on": [],  
    "pass": true,
```

```
"evidence": "This function uses `data.slice()` which is safe.",
"reasoning": "The golden answer explicitly states that `getDisplayedData` uses the
array method `data.slice()`, satisfying the criterion."
},
{
  "id": "completeness-35860",
  "tag": "completeness",
  "weight": "bonus",
  "description": "Mentions that `getMouseInfo` handovers to `handleMouseMove`
`handleMouseEnter` or `handleMouseLeave` handlers after getting the coordinates",
  "rationale": "Raw browser interactions and peripheral device interactions are natively
handled after coordinate computation",
  "dependent_on": [],
  "pass": true,
  "evidence": "\u201cWhen a click, touch or drag events \u2026 funnels through
\u2026 `getMouseInfo()` \u2026 It then hands it off to the right handler,
`handleMouseMove` `handleMouseEnter` or `handleMouseLeave`.\u201d",
  "reasoning": "The golden answer explicitly states that after `getMouseInfo`
computes the necessary coordinates, it passes control to `handleMouseMove`,
`handleMouseEnter`, or `handleMouseLeave`, satisfying the criterion."
},
{
  "id": "style-34112",
  "tag": "style",
  "weight": "bonus",
  "description": "Formats the explanation using structured hierarchical elements like
markdown headers or numbered lists",
  "rationale": "Given the sheer volume of functions in the target file, structural
formatting is essential for visual parsing",
  "dependent_on": [],
```

```
"pass": true,
  "evidence": "The response uses markdown headers such as `## Important Helper and Build Functions` and `## `generateCategoricalChart` Factory Function`, along with multiple ordered lists (e.g., `1. Imports and Constants...`, `2. Data Helpers...`).",
  "reasoning": "The explanation is structured with hierarchical markdown headers and numbered lists, fulfilling the requirement for structured formatting."
},
{
  "id": "style-34113",
  "tag": "style",
  "weight": "bonus",
  "description": "Presents inline code elements and function names using monospace markdown formatting",
  "rationale": "Maintains strict technical clarity by distinguishing variable/function names from explanation",
  "dependent_on": [],
  "pass": true,
  "evidence": "Examples include `AreaChart`, `generateCategoricalChart`, `getTooltipContent`, `getAxisMapByAxes`, `CategoricalChartWrapper`, `renderMap` and many others presented in backticks.",
  "reasoning": "Function names, files, and code elements are consistently formatted with inline monospace markdown, satisfying the criterion."
}
],
"passed": 34,
"failed": 0,
"total": 34,
"score_percent": 100.0,
"summary": "Passed 34/34 criteria (100.0%)"
}
```

Golden Answer Eval

```
{
  "pass": true,
  "score_percent": 100.0,
  "earned": 59.0,
  "max_possible": 59.0,
  "critical_fails": [],
  "penalties_triggered": [],
  "summary": "PASS \u2014 2014 golden answer follows rubric (100.0%)",
  "criteria": [
    {
      "id": "reasoning-34071",
      "weight": "penalty",
      "passed": true,
      "score": 0.0
    },
    {
      "id": "reasoning-34055",
      "weight": "bonus",
      "passed": true,
      "score": 1.0
    },
    {
      "id": "reasoning-34049",
      "weight": "critical",
      "passed": true,
      "score": 3.0
    },
    {
      "id": "reasoning-34008",
      "weight": "critical",
      "passed": true,
      "score": 3.0
    },
    {
      "id": "reasoning-34002",
      "weight": "critical",
      "passed": true,
      "score": 3.0
    },
    {
      "id": "reasoning-34059",
```

```
"weight": "bonus",
"passed": true,
"score": 1.0
},
{
  "id": "reasoning-34005",
  "weight": "critical",
  "passed": true,
  "score": 3.0
},
{
  "id": "reasoning-34057",
  "weight": "critical",
  "passed": true,
  "score": 3.0
},
{
  "id": "reasoning-34108",
  "weight": "penalty",
  "passed": true,
  "score": 0.0
},
{
  "id": "reasoning-34006",
  "weight": "critical",
  "passed": true,
  "score": 3.0
},
{
  "id": "reasoning-34051",
  "weight": "critical",
  "passed": true,
  "score": 3.0
},
{
  "id": "reasoning-34123",
  "weight": "penalty",
  "passed": true,
  "score": 0.0
},
{
  "id": "reasoning-34054",
  "weight": "critical",
  "passed": true,
```

```
"score": 3.0
},
{
  "id": "reasoning-34074",
  "weight": "critical",
  "passed": true,
  "score": 3.0
},
{
  "id": "reasoning-34141",
  "weight": "bonus",
  "passed": true,
  "score": 1.0
},
{
  "id": "reasoning-34050",
  "weight": "critical",
  "passed": true,
  "score": 3.0
},
{
  "id": "reasoning-34058",
  "weight": "bonus",
  "passed": true,
  "score": 1.0
},
{
  "id": "reasoning-34143",
  "weight": "bonus",
  "passed": true,
  "score": 1.0
},
{
  "id": "reasoning-34140",
  "weight": "bonus",
  "passed": true,
  "score": 1.0
},
{
  "id": "reasoning-34007",
  "weight": "critical",
  "passed": true,
  "score": 3.0
},
}
```

```
{
  "id": "reasoning-33993",
  "weight": "critical",
  "passed": true,
  "score": 3.0
},
{
  "id": "reasoning-34070",
  "weight": "penalty",
  "passed": true,
  "score": 0.0
},
{
  "id": "completeness-34076",
  "weight": "critical",
  "passed": true,
  "score": 3.0
},
{
  "id": "completeness-34075",
  "weight": "critical",
  "passed": true,
  "score": 3.0
},
{
  "id": "completeness-34142",
  "weight": "bonus",
  "passed": true,
  "score": 1.0
},
{
  "id": "completeness-35755",
  "weight": "critical",
  "passed": true,
  "score": 3.0
},
{
  "id": "completeness-34107",
  "weight": "penalty",
  "passed": true,
  "score": 0.0
},
{
  "id": "completeness-34079",
```

```
    "weight": "bonus",
    "passed": true,
    "score": 1.0
  },
  {
    "id": "completeness-34130",
    "weight": "bonus",
    "passed": true,
    "score": 1.0
  },
  {
    "id": "completeness-34078",
    "weight": "bonus",
    "passed": true,
    "score": 1.0
  },
  {
    "id": "completeness-34131",
    "weight": "bonus",
    "passed": true,
    "score": 1.0
  },
  {
    "id": "completeness-35860",
    "weight": "bonus",
    "passed": true,
    "score": 1.0
  },
  {
    "id": "style-34112",
    "weight": "bonus",
    "passed": true,
    "score": 1.0
  },
  {
    "id": "style-34113",
    "weight": "bonus",
    "passed": true,
    "score": 1.0
  }
]
}
```


Test Model

Score

33.9% (hard)

Model Response

=====

RUN 1/3

=====

You're seeing every chart component in `src/chart` "call the same function" because Recharts centralizes all of the cross-cutting chart behaviour inside the factory exported from `src/chart/generateCategoricalChart.tsx`. Instead of re-implementing axis parsing, tooltip state, syncing, accessibility, brushes, legends, portals, etc. for each chart, the library builds a single `CategoricalChartWrapper` class that all categorical/polar charts share, and then parameterizes it with just the pieces that differ per chart. Here's how it's wired together and why.

1. What each chart file actually does

Files such as `AreaChart.tsx`, `BarChart.tsx`, `LineChart.tsx`, `PieChart.tsx`, etc. are extremely thin:

```ts

```
export const BarChart = generateCategoricalChart({
 chartName: 'BarChart',
 GraphicalChild: Bar,
 defaultTooltipEventType: 'axis',
 validateTooltipEventTypes: ['axis', 'item'],
 axisComponents: [
 { axisType: 'xAxis', AxisComp: XAxis },
```

```

 { axisType: 'yAxis', AxisComp: YAxis },
],
 formatAxisMap,
 tooltipPayloadSearcher: arrayTooltipSearcher,
});
...

(src/chart/BarChart.tsx, lines 11-22)

```

Each one just calls `generateCategoricalChart`` with:

- \* `chartName`` – used for display names, Redux store keys, etc.
- \* `GraphicalChild`` – the primitive that actually renders the data (``Bar``, ``Area``, ``Line``, ``Pie``, ...).
- \* `axisComponents`` – which axis types this chart expects (Cartesian ``XAxis/YAxis``, polar ``angleAxis/radiusAxis``, etc.).
- \* `formatAxisMap`` – helper to massage computed axis maps for the particular coordinate system (``CartesianUtils.formatAxisMap`` vs ``PolarUtils.formatAxisMap``).
- \* Optional knobs like tooltip defaults or chart-specific default props (e.g. ``PieChart`` sets ``layout: 'centric', startAngle: 0, ...`` at lines 11-24 of ``src/chart/PieChart.tsx``).

Everything else—data slicing, state management, syncing, delegates, rendering order—is handled by the generated component.

---

### ### 2. What `generateCategoricalChart`` builds

`generateCategoricalChart`` (entire file ``src/chart/generateCategoricalChart.tsx``) returns a *function component* that wraps a ``CategoricalChartWrapper`` class component in a ``RechartsStoreProvider``. The wrapper:

1. **Derives state from props** (`createDefaultState`, `getDerivedStateFromProps``) — tracking mouse position, brush indexes, tooltip status, cached axis maps, formatted graphical items, etc. (see lines 268-458 and 615-918).
2. **Builds axis maps & offsets** (`getAxisMapByAxes`, `getAxisMapByItems`, `calculateOffset`, `formatAxisMap``) whenever data/layout/brush changes (lines 208-461, 542-693). This is where the `axisComponents`` you passed in matter: the factory knows which axis types to extract from children and how to interpret them.
3. **Formats child graphical elements** using the `GraphicalChild``'s static `getComposedData`` (lines 694-777). This step injects computed props (band positions, stacked data, animation ids, etc.) so `<Bar />`, <Line />`, etc.` can render correctly without duplicating logic per chart.
4. **Handles interaction plumbing** — tooltips (`getTooltipData`, `handleMouseEnter/Move/Leave`, `handleItemMouseEnter``), syncing multiple charts via `eventCenter`, throttling mouse move, click handlers, accessibility keyboard support, portals for cursors/tooltips, brush updates, legend bounding boxes, etc.` (lines 171-692, 779-1314).
5. **Renders** by wrapping children with many context providers and calling `renderByOrder(children, renderMap)`` so each supported child type gets either passed through untouched (`renderAsIs``) or cloned with the computed props (`renderGraphicChild``). See the render map around lines 1108-1138.
6. **Finally** the factory returns:

```
``tsx
return function CategoricalChart(props) {
```

```

const options: ChartOptions = { ... };
return (
 <RechartsStoreProvider preloadedState={{ options }} reduxStoreName={props.id ??
chartName}>
 <CategoricalChartWrapper {...props} />
 </RechartsStoreProvider>
);
};
...
(lines 1272-1289)

```

So every “chart component” you import is really “`generateCategoricalChart` with a specific configuration”.

---

### ### 3. Why Recharts does it this way

**\* \*\*Reduces duplication.\*\*** Axes, tooltips, syncing, animations, accessibility, portals, brushes, etc. are identical whether you’re plotting lines, areas, bars, or pies. The factory keeps that logic in one place.

**\* \*\*Keeps charts consistent.\*\*** When tooltip behaviour or axis formatting changes, fixing it in `generateCategoricalChart` automatically updates all categorical charts.

**\* \*\*Still allows specialization.\*\*** Each chart can override defaults (tooltip mode, layout, axis set, GraphicalChild) without rewriting the boilerplate.

**\* \*\*Future direction.\*\*** The file is full of comments about moving away from DOM-based child introspection (“@deprecated this relies on direct DOM access”). Centralizing the

logic makes it easier to migrate to Redux/context-driven state or plain JSX rendering later.

---

### ### 4. How to work with it as a contributor

1. **Understand `CategoricalChartState`` and contexts** (`src/chart/types.ts``). Most shared logic reads/writes `formattedGraphicalItems``, `xAxisMap``, `tooltipTicks``, etc.

2. **When creating a new chart**, add a new file that calls `generateCategoricalChart`` with:

- \* your `GraphicalChild`` component,
- \* the axis components it needs,
- \* a `formatAxisMap`` suited to its coordinate system,
- \* any default props or tooltip rules.

3. **If you need to change core behaviour** (new tooltip interaction, better stacking, improved accessibility), expect to update `generateCategoricalChart.tsx``. That change will automatically propagate to all charts that reference it.

4. **If something really must diverge**, you can extend the configuration options passed into `generateCategoricalChart``, or add chart-specific behaviour inside your `GraphicalChild`` (e.g. Scatter customizing tooltip positions, see lines 352-374 when `displayDefaultTooltip`` handles Scatter).

---

### ### Mental model

Think of each chart component as a *\*scenario\** fed into a common chart engine. The engine:

1. Collects children → builds axis maps → formats graphical items.
2. Manages interactions and shared state.
3. Renders children in the right order, injecting computed props where needed.

Your glass-box view of a “mini world per chart” still holds for the visual layer (each `<Bar />`, `<Line />` still computes its own shapes), but the scaffolding (axes, tooltips, layout, mouse handling) is centralized to avoid re-implementing the same infrastructure a dozen times.

=====

RUN 2/3

=====

In this repo the majority of “normal” Recharts components (AreaChart, BarChart, LineChart, etc.) aren’t hand-written as separate React classes. Instead, they are all thin configuration files that call the same factory, `generateCategoricalChart`, defined in `**src/chart/generateCategoricalChart.tsx**` (starts around line 802). The reason is that all of these charts share identical scaffolding: measuring the SVG container, parsing `<XAxis />` / `<YAxis />`, building brush ranges, synchronizing tooltips, hosting accessibility features, wiring Redux, etc. Rather than duplicating thousands of lines per chart, the project keeps a single “engine” and lets each chart describe only what’s different.

### How a chart file is structured

Take `**src/chart/BarChart.tsx**` (lines 11-22) as an example:

```
``ts
```

```

export const BarChart = generateCategoricalChart({
 chartName: 'BarChart',
 GraphicalChild: Bar,
 defaultTooltipEventType: 'axis',
 validateTooltipEventTypes: ['axis', 'item'],
 axisComponents: [
 { axisType: 'xAxis', AxisComp: XAxis },
 { axisType: 'yAxis', AxisComp: YAxis },
],
 formatAxisMap,
 tooltipPayloadSearcher: arrayTooltipSearcher,
});
...

```

Each chart passes a configuration object to the generator:

- \* ``chartName`` is only used for display/debugging and store keys.
- \* ``GraphicalChild`` tells the engine which graphical elements to treat as the “data series” it should render. It can be a single component (AreaChart uses ``Area``) or an array (ComposedChart passes ``[Line, Area, Bar, Scatter]``; see `**`src/chart/ComposedChart.tsx` lines 11-19**`).
- \* ``axisComponents`` describes which axis components the chart supports so the generator can discover ``<XAxis>`` / ``<YAxis>`` children and build axis maps.
- \* ``defaultTooltipEventType`` and ``validateTooltipEventTypes`` adjust tooltip UX (Scatter and Pie are ``item``-driven, Bar/Line/Area default to axis-aligned tooltips).
- \* ``formatAxisMap``, ``tooltipPayloadSearcher``, and optional ``defaultProps`` let each chart tweak formatting, tooltip lookup strategy, or default layout (e.g., PieChart/RadarChart set radial defaults; see `**`src/chart/PieChart.tsx` lines 11-27**`).

Because each chart is just “configuration”, they look minimal, yet the generator injects the massive shared implementation.

### What `generateCategoricalChart`` actually does

Inside `src/chart/generateCategoricalChart.tsx`` the generator returns a component called `CategoricalChartWrapper``. Highlights:

1. **Axis discovery & formatting** – Functions `getAxisMapByAxes``, `getAxisMapByItems``, and `getAxisMap`` (lines ~286-470) inspect the children using `findAllByType`` to collect each `<XAxis>`/`<YAxis>`` (or fall back to inferred axes if the user didn’t declare any). They build maps that include computed domains, ticks, scale info, etc. Config entries from `axisComponents`` drive which axis types are processed.
2. **Graphical item formatting** – `getFormatItems`` (lines ~877-1018) loops over every `GraphicalChild`` found in `props.children``, computes band sizes, stacking data, bar positions, etc., and calls each child’s static `getComposedData`` to produce ready-to-render points. That’s how a `<Bar>`` child gets all the coordinates it needs without the chart itself rendering the bars directly. Only children listed in `GraphicalChild`` are treated this way; everything else (grid, reference lines, etc.) is cloned “as-is”.
3. **Shared state machine** – The wrapper component (lines ~1099-1680) owns the full chart lifecycle:
  - \* default and derived state (`createDefaultState``, `getDerivedStateFromProps``) handle brush ranges, tooltip activity, layout changes, etc.;
  - \* event handlers (`handleMouseEnter``, `handleMouseMove``, sync events) keep tooltips, brushing and cross-chart synchronization consistent;



\* contexts (``ChartLayoutContextProvider``, ``TooltipPortalContext``, etc.) and ``RechartsStoreProvider`` wrap the entire surface so Redux slices and hooks can access chart metadata.

4. **Rendering** – ``render()`` (lines ~1578-1664) ultimately renders a ``Surface`` with ``renderByOrder(children, this.renderMap)``. ``renderMap`` dictates how each child type is treated—graphical children call ``renderGraphicChild``, which replaces them with the formatted props computed earlier.

5. **Why not per-chart classes?** Because all categorical charts need the same infrastructure (axes, tooltip orchestration, accessibility manager, syncing), maintaining separate copies would be error-prone. The generator centralizes that logic while still allowing per-chart behavior through its options and through the props of the graphical children themselves.

### ### When a chart really is its “own world”

Not every chart uses this generator. For instance, `**`src/chart/Sankey.tsx`**` is a standalone component with its own layout algorithm, mouse handling, and tooltip context because Sankey doesn’t fit the categorical axis model at all. You’ll recognize such charts because they export a class/function directly rather than calling ``generateCategoricalChart``.

### ### Recap

\* The files in ``src/chart/*.tsx`` that look like ``export const FooChart = generateCategoricalChart({...})`` are just configuration shims.

\* The heavy lifting lives in ``generateCategoricalChart``, which builds a reusable chart wrapper around whatever graphical children you declare.

\* Each chart customizes the shared engine via `GraphicalChild`, supported axes, default props, and tooltip behavior.

\* Truly different chart types (Sankey, Treemap, Sunburst, etc.) have their own implementations because they don't share the categorical axis workflow.

Once you internalize that pattern, contributing becomes easier: you rarely modify `AreaChart.tsx` itself—you tweak shared behavior in `generateCategoricalChart` or adjust the specific graphical child (e.g., `src/cartesian/Bar.tsx`).

=====

RUN 3/3

=====

### Why every chart “just calls a function”

If you open any categorical chart entry point under `src/chart` (e.g. `AreaChart.tsx` lines 4-20), you'll see the same pattern:

```
``ts
export const AreaChart = generateCategoricalChart({
 chartName: 'AreaChart',
 GraphicalChild: Area,
 axisComponents: [
 { axisType: 'xAxis', AxisComp: XAxis },
 { axisType: 'yAxis', AxisComp: YAxis },
],
 formatAxisMap,
 tooltipPayloadSearcher: arrayTooltipSearcher,
});
``
```

Each chart simply invokes `generateCategoricalChart`` with a configuration object. That helper (defined in `src/chart/generateCategoricalChart.tsx``) returns a fully functional React component class plus a Redux wrapper. The intent is to centralize the heavy, identical plumbing that every Cartesian/polar chart needs: axis resolution, tooltip handling, brushing, syncing, accessibility, portals, etc. Instead of duplicating hundreds of lines per chart, Recharts uses this factory to compose behavior.

### What the generator actually does

`generateCategoricalChart`` (lines 802–1120 and beyond):

1. **Normalizes axis and data state**

- It defines `getFormatItems`` (lines 812–915) which iterates over the “graphical children” (Bar, Line, Pie...) and asks each child to compose its data (`item.type.getComposedData``) with the correct axis info, stacked data, band sizes, etc.
- It builds axis maps via `getAxisMap`` / `getAxisMapByAxes`` / `getAxisMapByItems`` (lines 218–405, 474–532) and formats them with the `formatAxisMap`` function you pass from each chart file.

2. **Calculates layout offsets & tooltip ticks**

- `calculateOffset`` (lines 548–620) figures out how much room axes, legends, brushes take and stores the drawable view box in state.
- `tooltipTicksGenerator`` (lines 700–708) prepares ordered ticks for tooltip interaction.

3. **Provides the state machine**

- `CategoricalChartWrapper`` (lines 1019–2006) is a massive React `Component`` subclass that owns state such as `graphicalItems``, `xAxisMap``, `tooltipTicks``, `activeTooltipIndex``, etc.
- `static getDerivedStateFromProps`` (lines 1183–1333) is responsible for recomputing axis maps, formatted children, and tooltip data when props change.

- Mouse/touch/key handlers (lines 1400–1715) funnel all interaction through shared logic, including sync events (`eventCenter``) when `syncId`` is used.

#### 4. **Sets up React context + store**

- The returned function wraps the class in a `RechartsStoreProvider`` (lines 1993–2006), injecting chart-specific options (tooltip behavior, gaps, stack offset).
- It also wires `ChartLayoutContextProvider``, tooltip contexts, brush contexts, etc. inside `render`` (lines 1786–1990) so that descendants (axes, tooltips, custom components) can consume layout info without prop drilling.

### ### How charts customize the shared implementation

The options you pass to `generateCategoricalChart`` control the behavior:

- **`GraphicalChild``**: either a single component (AreaChart uses `Area`` at line 12) or an array (ComposedChart passes `[Line, Area, Bar, Scatter]`` at lines 11–18). The generator uses `findAllByType(children, GraphicalChild)`` to know which series are “native” to the chart when formatting data.
- **`axisComponents``**: tells the generator which axis React elements to look for. Area/Line charts give `{ axisType: 'xAxis', AxisComp: XAxis }`` etc., while Pie/Radar swap in `PolarAngleAxis`` / `PolarRadiusAxis`` (PieChart lines 9-27). FunnelChart passes an empty array because it doesn’t render axes (lines 1-11).
- **`Defaults``**: you can override layout, angles, centers, etc. via `defaultProps`` (e.g. PieChart sets `layout: 'centric', startAngle: 0, endAngle: 360`` on lines 18-26).
- **`Tooltip mode``**: `defaultTooltipEventType`` and `validateTooltipEventTypes`` define whether the chart responds to axis-based vs. item-based tooltips (see ScatterChart lines 11-20 for item-only mode).

So every chart isn't "its own world" because 90% of the lifecycle is identical: load data, build axes, format child props, manage tooltips/brush/legend/portals. The differentiators are captured in that config object.

### ### Exceptions

Not all charts fit this mold. `SunburstChart.tsx` is entirely custom (long file starting line 1) because a sunburst doesn't share Cartesian logic. It still reuses pieces like `RechartsWrapper`, context providers, and the Redux store, but rendering + interaction are bespoke.

### ### Mental model when contributing

When you add or modify a chart:

1. Decide whether it can reuse the categorical generator. If it uses X/Y (or polar) axes and the usual tooltip/brush stack, you probably just pass the right `GraphicalChild` and axis components.
2. If the chart needs unique layout defaults or tooltip behavior, set them in the config. That keeps the shared `CategoricalChartWrapper` logic untouched.
3. For brand-new graphical elements, implement `getComposedData` on the element (e.g. `cartesian/Area.tsx`) so `getFormatItems` can call it and inject the computed props back into the cloned element (lines 884-912).
4. Only build a bespoke chart component (like `SunburstChart`) if the rendering paradigm fundamentally differs.

By understanding `generateCategoricalChart`, you can trace how props flow: chart file → generator config → shared wrapper state → child formatting → cloneElement in `render`. That's why everything "just calls a function"—it's a deliberate abstraction to keep the library consistent and maintainable.

## Evaluation JSON

```
{
 "points": 20.0,
 "maxPoints": 59.0,
 "score_percent_weight": 34,
 "summary_weight": "20.0/59.0 points (34%)",
 "eval_runs": 3,
 "eval_runs_succeeded": 3,
 "per_run_points": [
 17.0,
 26.0,
 17.0
],
 "results": [
 {
 "id": "reasoning-34071",
 "weight": "penalty",
 "pass": false,
 "points": 0.0
 },
 {
 "id": "reasoning-34055",
 "weight": "bonus",
 "pass": false,
 "points": 0.0
 },
 {
 "id": "reasoning-34049",
 "weight": "critical",
 "pass": false,
 "points": 0.0
 },
 {
 "id": "reasoning-34008",
 "weight": "critical",
 "pass": false,
 "points": 0.0
 }
]
}
```

```
},
{
 "id": "reasoning-34002",
 "weight": "critical",
 "pass": true,
 "points": 3.0
},
{
 "id": "reasoning-34059",
 "weight": "bonus",
 "pass": false,
 "points": 0.0
},
{
 "id": "reasoning-34005",
 "weight": "critical",
 "pass": false,
 "points": 0.0
},
{
 "id": "reasoning-34057",
 "weight": "critical",
 "pass": false,
 "points": 0.0
},
{
 "id": "reasoning-34108",
 "weight": "penalty",
 "pass": false,
 "points": 0.0
},
{
 "id": "reasoning-34006",
 "weight": "critical",
 "pass": false,
 "points": 0.0
},
{
 "id": "reasoning-34051",
```

```
"weight": "critical",
"pass": false,
"points": 0.0
},
{
 "id": "reasoning-34123",
 "weight": "penalty",
 "pass": false,
 "points": 0.0
},
{
 "id": "reasoning-34054",
 "weight": "critical",
 "pass": false,
 "points": 0.0
},
{
 "id": "reasoning-34074",
 "weight": "critical",
 "pass": false,
 "points": 0.0
},
{
 "id": "reasoning-34141",
 "weight": "bonus",
 "pass": false,
 "points": 0.0
},
{
 "id": "reasoning-34050",
 "weight": "critical",
 "pass": false,
 "points": 0.0
},
{
 "id": "reasoning-34058",
 "weight": "bonus",
 "pass": false,
 "points": 0.0
}
```



```
},
{
 "id": "reasoning-34143",
 "weight": "bonus",
 "pass": false,
 "points": 0.0
},
{
 "id": "reasoning-34140",
 "weight": "bonus",
 "pass": false,
 "points": 0.0
},
{
 "id": "reasoning-34007",
 "weight": "critical",
 "pass": false,
 "points": 0.0
},
{
 "id": "reasoning-33993",
 "weight": "critical",
 "pass": true,
 "points": 3.0
},
{
 "id": "reasoning-34070",
 "weight": "penalty",
 "pass": false,
 "points": 0.0
},
{
 "id": "completeness-34076",
 "weight": "critical",
 "pass": true,
 "points": 3.0
},
{
 "id": "completeness-34075",
```

```
"weight": "critical",
"pass": true,
"points": 3.0
},
{
 "id": "completeness-34142",
 "weight": "bonus",
 "pass": false,
 "points": 0.0
},
{
 "id": "completeness-35755",
 "weight": "critical",
 "pass": true,
 "points": 3.0
},
{
 "id": "completeness-34107",
 "weight": "penalty",
 "pass": false,
 "points": 0.0
},
{
 "id": "completeness-34079",
 "weight": "bonus",
 "pass": false,
 "points": 0.0
},
{
 "id": "completeness-34130",
 "weight": "bonus",
 "pass": false,
 "points": 0.0
},
{
 "id": "completeness-34078",
 "weight": "bonus",
 "pass": false,
 "points": 0.0
}
```

```
},
{
 "id": "completeness-34131",
 "weight": "bonus",
 "pass": false,
 "points": 0.0
},
{
 "id": "completeness-35860",
 "weight": "bonus",
 "pass": false,
 "points": 0.0
},
{
 "id": "style-34112",
 "weight": "bonus",
 "pass": true,
 "points": 1.0
},
{
 "id": "style-34113",
 "weight": "bonus",
 "pass": true,
 "points": 1.0
}
]
```

**model eval score 33.9**

**Difficulty**    **hard**

## 4 Example Task 5137 Architecture & System Design

## Example Task 5137

**Name:** kubernetes-autoscaler-architecture  
**Intent:** Architecture & System Design  
**Difficulty:** Hard  
**Language:** Go

### Repo

**Repository:** <https://github.com/kubernetes/autoscaler>  
**Checkout (commit):** 975892c06115bee459e601a3c3d0e748d4eab88a

### Prompt

I am looking to add a new feature and would like architecture and design-level decisions made before we can proceed with actual implementation. I want to implement the ability to autoscale a cluster depending on a certain time of day. Can you help me with how to implement this from an architecture perspective? I am specifically looking at which places to add this to and how to define the config mechanisms.

### LLM Prompt Review

Prompt QC Feedback

**\*\*Grading Item 1 (1/1)\*\***

The request seeks architectural guidance for a time-based autoscaling feature, which is non-trivial and not derived from a single file.

---

**\*\*Grading Item 2 (1/1)\*\***

It reads like a natural developer inquiry without any role-play framing.

---

**\*\*Grading Item 3 (1/1)\*\***

It does not ask for basic definitions or obvious behavior.

---

**\*\*Grading Item 4 (1/1)\*\***

Addressing where to add the feature and define configuration requires broader repository understanding.

---

**\*\*Grading Item 5 (1/1)\*\***

The prompt contains only a question/description and no embedded repo context or code snippets.

---

**\*\*Grading Item 6 (1/1)\*\***

The question is specific about enabling time-based autoscaling and defining configuration points, leading to a concrete answer.

---

**\*\*Grading Item 7 (1/1)\*\***

The architectural guidance aligns with the likely intent of repository-context reasoning.

---

**\*\*Grading Item 8 (1/1)\*\***

It elicits architectural reasoning and design judgment, not a trivial lookup.

---

**\*\*Grading Item 9 (1/1)\*\***

There are no meta-instructions or rubric references.

---

**\*\*Grading Item 10 (1/1)\*\***

Only ASCII characters are used.

---

FINAL\_SCORE: 10

## Rubric

| ID    | tag          | description                                                                               | weight | rationale                                                                                                                                                     | dependent_on |
|-------|--------------|-------------------------------------------------------------------------------------------|--------|---------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------|
| 17714 | completeness | States that Cluster Autoscaler was originally not designed for time based autoscale usage | bonus  | Cluster Autoscaler was always designed to scale the cluster dynamically based on metrics, not to scale it based on specific set of time of day configurations |              |

|       |              |                                                                                                                                                               |          |                                                                                                                                                                                     |       |
|-------|--------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------|----------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------|
| 17715 | completeness | Gives an example of a use case in the real world from a tech company currently having the exact issue with Cluster Autoscaler                                 | bonus    | Examples help the developer better understand if the feature was discussed before on public forums.                                                                                 |       |
| 17733 | reasoning    | Identifies that normal cluster autoscaler logic cannot handle periodic bursts of traffic without eager configuration of thresholds or manual fallback         | critical | This is the original use case that the answer has to identify to get to a valid working solution and without this, the entire architecture falls apart                              |       |
| 17734 | reasoning    | Identifies that per-cloud provider architecture modification avoids touching the complex core logic and the code changes that would require extensive testing | critical | Complex core logic changes can cause a lot of side effects and burden to the maintainers because they require the critical component to remain stable and testing can become costly |       |
| 17737 | reasoning    | Identifies that core logic changes have a large blast radius with security risks that can spread across all cloud providers at once                           | bonus    | Security issues are a bonus to the existing difficulties that come with the implementation in the core logic.                                                                       | 17734 |



|       |              |                                                                                                                                                      |          |                                                                                                                              |       |
|-------|--------------|------------------------------------------------------------------------------------------------------------------------------------------------------|----------|------------------------------------------------------------------------------------------------------------------------------|-------|
| 17738 | reasoning    | Identifies that existing tags already being used by cloud provider node groups can be used to extend the configuration for this functionality        | critical | Configuring the schedule is important to be able to control which node group gets which schedule                             |       |
| 17742 | completeness | States the sample scale up and down schedule config that can be used with AWS ASGs                                                                   | bonus    | Example of how the config can be designed for use with AWS Auto Scaling Groups that will be used as tags                     | 17738 |
| 17745 | reasoning    | Identifies the requirement of having a `strict` and `relaxed` mode for better handling of defaults                                                   | bonus    | Extra feature to handle defaults better, making the auto scaler use defaults when needed.                                    | 17742 |
| 17748 | completeness | States that `utils/schedule_parser.go` can be used as a shared module to help parse the schedules from all Cloud Providers                           | critical | Standard way of parsing the schedules can help cloud provider implementations to focus specifically on how to read the tags. |       |
| 17754 | reasoning    | Identifies that `aws_manager.go` (or similar files in other cloud providers) is the file where the extract logic for the new tag must be implemented | critical | Initial read from ASG tags is required to load the configuration from AWS to further process it                              |       |

|       |              |                                                                                                                                                                                      |          |                                                                                                                                                                                                                                                                            |       |
|-------|--------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------|
| 17966 | completeness | States that <code>`extractScheduleFromAsg`</code> function can be implemented to handle the new tag                                                                                  | critical | New function is needed to keep the code modular and easy to replicate in other cloud providers                                                                                                                                                                             | 17754 |
| 17970 | completeness | States that <code>`buildAsgFromAWS`</code> needs to be wired up to read the new schedule tags                                                                                        | critical | New function needs to be called along with existing extract functions.                                                                                                                                                                                                     | 17966 |
| 17974 | reasoning    | Identifies that <code>`MinSize()`</code> and <code>`MaxSize()`</code> in <code>`aws_cloud_provider.go`</code> should expose scheduled bounds that cap scale-up and block scale-down. | critical | Without scheduled bounds from these accessors, the core autoscaler logic will not respect scheduled limits when reacting to pod demand.                                                                                                                                    |       |
| 17981 | completeness | Describes the explicit desired-capacity reconciliation path needed to raise an AWS ASG to the scheduled minimum at the target time.                                                  | critical | This prevents rewarding a plan that only changes <code>`MinSize()`</code> and <code>`MaxSize()`</code> ; in this repo, proactive AWS size changes must go through the provider size-changing path such as <code>`SetAsgSize`</code> or <code>`SetDesiredCapacity`</code> . | 17974 |
| 17986 | reasoning    | Identifies that the Refresh function runs at a one-minute interval, picking up schedule config changes when the refresh fires                                                        | bonus    | Refreshing of any new changes to the schedule config is important to capture new requirements for applications                                                                                                                                                             |       |

|       |              |                                                                                                                             |         |                                                                                                                                                           |       |
|-------|--------------|-----------------------------------------------------------------------------------------------------------------------------|---------|-----------------------------------------------------------------------------------------------------------------------------------------------------------|-------|
| 17988 | completeness | States to manually restart the CA pod if the changes are required instantly or near real time                               | bonus   | Near real time changes would prove costly if API calls made to AWS frequently and the best way is to restart the pod to force it into pulling ASG config. | 17986 |
| 17990 | style        | Separates the answer into distinct sections that describe one part of the architecture at a time                            | bonus   | Easier for the developer to follow the architecture and help in implementation planning                                                                   |       |
| 17991 | style        | Uses precise Go terminology to ground the architectural claims                                                              | bonus   | Grounding the architectural claims on the language of the repository helps developers to easily implement the required feature.                           |       |
| 17996 | reasoning    | Claims that the current architecture of Cluster Autoscaler does not allow the implementation of any new Scheduling features | penalty | This is wrong, the cluster autoscaler can be easily extended to have this feature                                                                         |       |
| 18004 | reasoning    | Claims that the Cluster Autoscaler does not read tags and depends on a YAML file that is passed as a ConfigMap              | penalty | The Cluster Autoscaler exclusively uses tags to identify how and why to scale a node group                                                                |       |

|       |              |                                                                                                                                                            |         |                                                                                                                                                                                                                                                          |  |
|-------|--------------|------------------------------------------------------------------------------------------------------------------------------------------------------------|---------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--|
| 18019 | completeness | States that the implementation must go into vertical pod autoscaler because nodes scale when pods increase their CPU                                       | penalty | Nodes do scale when VPA increases CPU of a pod if required, but that does not answer why the schedule has to go there.                                                                                                                                   |  |
| 18025 | completeness | Claims that modifying `MinSize()` and `MaxSize()` is sufficient on its own to automatically scale the cluster to the scheduled minimum at the target time. | penalty | `MinSize()` is an accessor that prevents scale-down; it does not proactively scale up the cluster. Claiming it does creates a false architectural model.                                                                                                 |  |
| 18032 | reasoning    | Claims that nodes are not needed and that clusters must never hit their defaults if schedule tags are present                                              | penalty | While this is possible, defaults are set on a cluster after a lot of internal decisions. Min levels are there to optimise costs when the applications see the least traffic (like night) and this would cause lot of cost overruns if implemented wrong. |  |

## LLM Rubric Review

### Rubric QC Feedback

**\*\*Grading Item 1:** The rubric is organized into exactly three top-level evaluation categories: `reasoning`, `completeness`, and `style`. All three categories are present. (2/2)\*\*

The rubric defines exactly the three required categories (`reasoning`, `completeness`, `style`) with no extras.

---

**\*\*Grading Item 2: The rubric contains a reasonable number of grading items for the task complexity and covers the task holistically. (2/2)\*\***

There are 23 total grading items—well within the typical range and appropriate for the task breadth.

---

**\*\*Grading Item 3: Every grading item is atomic and self-contained, evaluating exactly one distinct point. (2/2)\*\***

Each item focuses on a single requirement or prohibition without bundled checks.

---

**\*\*Grading Item 4: Grading item descriptions are clear, specific, and measurable. (2/2)\*\***

Descriptions mention concrete files, functions, or claims, making evaluation straightforward.

---

**\*\*Grading Item 5: Every grading item is derivable from the provided `golden\_answer`. (2/2)\*\***

All expectations stem directly from statements or implications in the golden answer.

---

**\*\*Grading Item 6: The rubric is comprehensive relative to the `golden\_answer`. (2/2)\*\***

All major points from the golden answer—per-provider approach, tag-based config, parser module, Min/Max overrides, refresh behavior, etc.—are captured.

---

**\*\*Grading Item 7: The rubric would award full marks to the provided `golden\_answer`. (2/2)\*\***

The golden answer satisfies every critical requirement and avoids all penalties defined.

---

**\*\*Grading Item 8: The rubric is fair to valid alternative reasoning paths. (1/2)\*\***

Because multiple critical items mandate specific mechanisms (e.g., AWS tags, strict/relaxed modes) and rule out alternative configurations that could also meet the prompt, the rubric is somewhat narrow even if aligned with the golden answer.

---

**\*\*Grading Item 9: The `reasoning` category evaluates correctness of reasoning, technical judgment, conceptual understanding, and explanations. (2/2)\*\***

Reasoning items focus on architectural choices, risk assessments, and justification of design elements.

---

**\*\*Grading Item 10: The `completeness` category evaluates whether the response addresses all major parts of the question with supporting details. (2/2)\*\***

Completeness items cover necessary follow-through—implementation touchpoints, parser wiring, configuration examples, and operational guidance.

---

**\*\*Grading Item 11: The `style` category evaluates clarity, organization, readability, and professionalism of the response. (2/2)\*\***

Style items emphasize structured sections and appropriate terminology, staying focused on presentation.

---

**\*\*Grading Item 12: Grading items are placed in appropriate categories. (2/2)\*\***

Each item aligns with its category's intent; no technical correctness checks are misplaced in style, and vice versa.

---

**\*\*Grading Item 13: The rubric includes the task-wide expectations that should always be present: reasoning quality, completeness, and code style. (2/2)\*\***

All three task-wide expectation areas are explicitly represented through multiple items.

---

**\*\*Grading Item 14: When the task intent requires it, the rubric includes appropriate intent-specific grading items. (2/2)\*\***

Architecture-specific expectations (per-provider vs. core logic decisions, file touchpoints, risk considerations) are included where needed.

---

**\*\*Grading Item 15: Weights use only valid values and are well calibrated. (2/2)\*\***

Weights are limited to `critical`, `bonus`, or `penalty`, and criticality assignments align with importance.

---

**\*\*Grading Item 16: Penalty items target genuinely harmful or misleading behavior. (2/2)\*\***

Penalties address incorrect claims or dangerous implementation advice rather than merely missing content.

---

**\*\*Grading Item 17: Dependency usage is logically sound. (2/2)\*\***

Dependencies reflect real prerequisite relationships and avoid illogical directionality.

---

**\*\*Grading Item 18: The rubric is not overprescriptive. (1/2)\*\***

Numerous critical checks demand very specific file names, tag formats, and mode semantics, leaving limited room for alternative but still valid repo-grounded approaches.

---



**\*\*Grading Item 19: Each grading item contributes meaningfully to evaluating answer quality. (2/2)\*\***

No items are redundant; each adds a distinct evaluative checkpoint.

---

FINAL\_SCORE: 36

## Golden Answer

The Kubernetes Cluster Autoscaler was originally not designed to handle scheduled tasks. But looking at the requirements, a lot of cluster managers have implemented their own model of doing this by using eviction pre warm pods or setting up CronJobs to scale the pods before a known time when they expect a burst of traffic hits. This is evident further from Tech Blogs of various high scale apps like Hotstar and Netflix where live shows or sporting events attract a huge number of floating crowd. One such example is Hotstar handling a burst of traffic when a match starts and no longer needing that capacity when a match ends.

Normal Cluster Autoscaling logic is not enough to handle the above scenarios without using some sort of manual fallback or highly eager configurations to reduce thresholds. The wait time for the loop to run and detect that the metrics are above the threshold is not enough in a case of a famous match broadcasting, especially when we know the timings of the event before the traffic even arrives.

With this in mind, here is the recommended architecture to achieve this functionality in a way that is easy to configure. This requires per-cloud provider modification, but once the cloud provider logic is in place, it requires no change to the core logic.

### ## Core Logic Issues:

While changing the core logic is a great approach where it avoids touching any Cloud Provider-specific code and delivers a single solution for all providers, it carries significant risk:

- Blast radius: A bug or misconfiguration propagates to across all Cloud providers simultaneously.

- Security exposure: Any vulnerability introduced into the core logic is immediately shared across every provider.
- Stability: The Cluster Autoscaler is a critical component. Changes to core logic require extensive testing across all providers before they can be considered safe.

For these reasons, a per-cloud-provider implementation is strongly recommended over modifying the core logic.

## ## Configuration Management:

The configuration is done at the cloud provider level. For AWS, this would be tags on the Autoscaling Group (ASG). This helps decentralise and configure per node group or autoscaling group.

We can have a tag specifically for schedule configuration:

- Scale Up and Maintain: ``k8s.io/cluster-autoscaler/schedule: "0 8 * * * min=10 max=40 strict"`
- Scale Down and Maintain: ``k8s.io/cluster-autoscaler/schedule: "0 22 * * * min=2 max=10 relaxed"`

``strict`` and ``relaxed`` are flags that can be used to indicate to the decision maker in the CA to either strictly follow the ``min`` and ``max`` or take a relaxed approach. In a ``relaxed`` mode, the cluster can scale up or down depending on the requirements of the pods and traffic, but in ``strict`` mode, the autoscaler will not touch or reduce to the default config. In ``relaxed`` mode, if the ``min=1`` on the default config and the CA sees no traffic or in CA's case metrics on CPU and Memory, then the CA can decide to fallback to the default value. The thresholds can be larger for such cases.

Adding these to ASG Tags can help in scaling up certain Node Groups, like GPU nodes for known times when researchers will be actively training or inferencing in large numbers or specific node groups that have special capabilities or are in a VPC that is required to access certain resources.

The configuration parser can be implemented as a new file in ``utils/schedule_parser.go`` that can handle all the parsing requirements. This can be a shared module for all cloud providers to use.

The final struct that can be used downstream after parsing can be:

```
```go
type scheduleConfig struct {
```

```

    Cron string // "0 8 * * *"
    Min int
    Max int
    Mode string // "strict" or "relaxed"
}
...

```

Tag Parsing:

The following shows an example of the tags can be parsed in AWS, but the mechanism remains same for other Cloud Providers including `vSphere VMware APIs`:

a. In `cluster-autoscaler/cloudprovider/aws/aws\_manager.go` add a new extract function `extractScheduleFromAsg()` and wire it up to extract the schedule format in `buildAsgFromAWS()` in `cluster-autoscaler/cloudprovider/aws/auto\_scaling\_groups.go`. This new function captures the value with this key (one or many) and passes it to the function in `utils/schedule\_parser.go`.

b. In `cluster-autoscaler/cloudprovider/aws/aws\_cloud\_provider.go` we need to override `MinSize()` and `MaxSize()`. These are the two methods that influence the scale up or down of any AWS EKS Cluster.

Inner Workings:

The inner workings are influenced exclusively by how the two modes will be working.

- `strict` mode (`k8s.io/cluster-autoscaler/schedule: "0 8 \* \* \* min=10 max=40 strict"`): `MinSize()` returns 10, `MaxSize()` returns 40 regardless of pod demand or utilization. Scale down is blocked below 10 and scale up is locked to a maximum of 40. No fallback is allowed by the CA even if the cluster is idle.

- `relaxed` mode (`k8s.io/cluster-autoscaler/schedule: "0 22 \* \* \* min=2 max=10 relaxed"`): `MinSize()` returns 2, `MaxSize()` returns 10 as soft suggestions. This logic can be added in the pre-check loop and determine based on thresholds like a preset time to understand if defaults take the place of the min or max configured.

Always use the current time to respect the schedule time. The loop can happen before or after the scheduled time passes and this is expected. We are not looking for an on-dot system. That would require a much more complex way to handle it.

Refresh of the ASG for changes of schedules on the fly can happen when the `Refresh` function is called in `cluster-autoscaler/cloudprovider/aws/aws\_manager.go`. This refreshes the ASG after a set interval, with the default being 1 minute. This calls the `regenerate()` which calls `buildAsgNames()` method and registers or unregisters the ASG Tags, while also caching them for future use.

Restarting the CA Pod can also force refresh the ASG tags if the changes are required near real time.

Conclusion

The above architecture provides a highly scalable plug and play per cloud provider solution to solve an issue that is common to the industry where traffic is predictable and only required for a certain period of time in any given day. Multiple schedules can be defined per day and all of them can be used to coordinate the traffic the cluster will see.

Golden Answer Review

Golden Answer QC Feedback

****Grading Item 1 (3/3)****

Internally consistent with coherent logic, no contradictions, and suggested structs/functions use defined data and realistic control flow.

****Grading Item 2 (3/3)****

Grounded in repo exploration by naming concrete files (`aws\_manager.go`, `aws\_cloud\_provider.go`, `auto\_scaling\_groups.go`, `utils/schedule\_parser.go`) and describing their roles.

****Grading Item 3 (2/2)****

Directly addresses how to architect time-based autoscaling and where to add configuration/logic.

****Grading Item 4 (3/3)****

Explains reasoning from problem (scheduled bursts) to per-provider tags, parsing, and Min/Max overrides.

****Grading Item 5 (2/2)****

Mechanistic detail is precise (e.g., how strict vs relaxed modes influence ``MinSize`/`MaxSize`` returns).

****Grading Item 6 (2/2)****

Technical statements align with Cluster Autoscaler behavior and AWS tagging without factual errors.

****Grading Item 7 (3/3)****

All critical rubric requirements met: uses existing tags, per-provider approach, ``extractScheduleFromAsg``, ``buildAsgFromAWS`` wiring, shared parser, and ``MinSize`/`MaxSize`` integration.

****Grading Item 8 (2/2)****

Avoids penalty behaviors; no incorrect claims about tags, defaults, or scheduling capability.

****Grading Item 9 (2/2)****

Comprehensive coverage of configuration, parsing, provider overrides, refresh behavior, and modes.

****Grading Item 10 (2/2)****

Clear, organized, and accessible to non-experts.

****Grading Item 11 (2/2)****

Includes concrete supporting details like tag keys, sample values, file paths, and struct definitions.

****Grading Item 12 (3/3)****

Follows architecture-task expectations: focuses on design decisions, module boundaries, and configuration strategy.

****Grading Item 13 (2/2)****

Not a PR triage task, so full credit.

****Grading Item 14 (1/1)****

Contains only ASCII characters.

FINAL_SCORE: {32}

Validate Golden Answer

Golden Answer Validation

```
{
  "results": [
    {
      "id": "reasoning-17738",
      "tag": "reasoning",
      "weight": "critical",
      "description": "Identifies that existing tags already being used by cloud provider node groups can be used to extend the configuration for this functionality",
      "rationale": "Configuring the schedule is important to be able to control which node group gets which schedule",
    }
  ]
}
```

```

    "dependent_on": [],
    "pass": true,
    "evidence": "\u201cThe configuration is done at the cloud provider level. For AWS,
this would be tags on the Autoscaling Group (ASG)\u2026. We can have a tag
specifically for schedule configuration.\u201d",
    "reasoning": "The answer explicitly states that existing cloud-provider constructs
(ASG tags) should be used to extend configuration for scheduling, meeting the
criterion."
  },
  {
    "id": "reasoning-17737",
    "tag": "reasoning",
    "weight": "bonus",
    "description": "Identifies that core logic changes has a large blast radius with security
risks that can spread across all cloud providers at once.",
    "rationale": "Security issues are a bonus to the existing difficulties that come with the
implementation in the core logic.",
    "dependent_on": [
      "reasoning-17734"
    ],
    "pass": true,
    "evidence": "\u201cBlast radius: A bug or misconfiguration propagates to across all
Cloud providers simultaneously. Security exposure: Any vulnerability introduced into the
core logic is immediately shared across every provider.\u201d",
    "reasoning": "The golden answer explicitly states that modifying the core logic has a
large blast radius and highlights the security risk that vulnerabilities would spread
across all cloud providers."
  },
  {
    "id": "reasoning-18004",

```



```
"tag": "reasoning",
"weight": "penalty",
"description": "Claims that the Cluster Autoscaler does not read tags and depends on
a YAML file that is passed as a ConfigMap",
"rationale": "The Cluster Autoscaler exclusively uses tags to identify how and why to
scale a node group",
"dependent_on": [],
"pass": true,
"evidence": "The answer repeatedly emphasizes using tags on the Autoscaling Group
for configuration (e.g., \u201cFor AWS, this would be tags on the Autoscaling
Group\u201d). It never claims the Cluster Autoscaler does not read tags or relies on a
YAML ConfigMap instead.",
"reasoning": "Because the golden answer explicitly describes tag-based configuration
and does not assert that the Cluster Autoscaler depends on a YAML ConfigMap, it
avoids the penalized behavior."
},
{
  "id": "reasoning-17974",
  "tag": "reasoning",
  "weight": "critical",
  "description": "Identifies that `MinSize()` and `MaxSize()` in `aws_cloud_provider.go`
must be used to override the required functionality",
  "rationale": "This is existing place where the cluster decisions are made to set the min
and max size of the cluster",
  "dependent_on": [],
  "pass": true,
  "evidence": "There is explicit mention: \"In
`cluster-autoscaler/cloudprovider/aws/aws_cloud_provider.go` we need to override
`MinSize()` and `MaxSize()`.\"",
```

"reasoning": "The golden answer directly identifies the need to override `MinSize()` and `MaxSize()` within `aws\_cloud\_provider.go`, satisfying the criterion."

},

{

"id": "reasoning-17734",

"tag": "reasoning",

"weight": "critical",

"description": "Identifies that per-cloud provider architecture modification is avoids touching the complex core logic avoiding code changes that require extensive testing",

"rationale": "Complex core logic changes can cause lot of side effects and burden to the maintainers maintaining the code because it requires the critical component to be stable and testing can become costly",

"dependent_on": [],

"pass": true,

"evidence": "For these reasons, a per-cloud-provider implementation is strongly recommended over modifying the core logic.",

"reasoning": "The answer explicitly highlights that modifying core logic introduces broad risk and testing burden, and therefore recommends per-cloud provider architecture changes to avoid those issues, satisfying the criterion."

},

{

"id": "reasoning-18032",

"tag": "reasoning",

"weight": "penalty",

"description": "Claims that modes are not needed and that clusters must never hit their defaults if schedule tags are present",

"rationale": "While this is possible, defaults are set on a cluster after a lot of internal decisions. Min levels are there to optimise costs when the applications see the least traffic (like night) and this would cause lot of cost overruns if implemented wrong.",

"dependent_on": [],

"pass": true,

"evidence": "The answer differentiates between a strict mode that enforces scheduled min/max values and a relaxed mode that allows fallback to defaults, e.g., \u201cIn a \u2018relaxed\u2019 mode \u2026 the CA can decide to fallback to the default value,\u201d and it never states that modes are unnecessary or that defaults must never be reached whenever schedule tags exist.",

"reasoning": "The golden answer explicitly uses modes and permits defaults in relaxed mode, so it does not make the penalized claim that modes are unnecessary or that defaults must never be hit when schedule tags are configured."

},

{

"id": "reasoning-17986",

"tag": "reasoning",

"weight": "bonus",

"description": "Identifies that `Refresh` is called once every one minute to refresh the tags making the config changes to schedule reflected when the refresh happens",

"rationale": "Refreshing of any new changes to the schedule config is important to capture new requirements for applications",

"dependent_on": [],

"pass": true,

"evidence": "Refresh of the ASG for changes of schedules on the fly can happen when the `Refresh` function is called in

`cluster-autoscaler/cloudprovider/aws/aws\_manager.go`. This refreshes the ASG after a set interval, with the default being 1 minute.",

"reasoning": "The golden answer explicitly states that the Refresh function runs at a set interval with a default of one minute, showing awareness that schedule tag changes are picked up roughly every minute."

},

{

"id": "reasoning-17733",

```
"tag": "reasoning",
"weight": "critical",
"description": "Identifies that normal cluster autoscaler logic cannot handle periodic
bursts of traffic without eager configuration of thresholds or manual fallback",
"rationale": "This is the original use case that the answer has to identify to get to a
valid working solution and without this, the entire architecture falls apart",
"dependent_on": [],
"pass": true,
"evidence": "Normal Cluster Autoscaling logic is not enough to handle the above
scenarios without using some sort of manual fallback or highly eager configurations to
reduce thresholds. The wait time for the loop to run and detect that the metrics are
above the threshold is not enough...",
"reasoning": "The answer explicitly states that default cluster autoscaler logic cannot
handle predictable bursts without manual fallback or proactive threshold tuning,
satisfying the criterion."
},
{
  "id": "reasoning-17754",
  "tag": "reasoning",
  "weight": "critical",
  "description": "Identifies that `aws_manager.go` (or similar files in other cloud
providers) is the file where the extract logic for the new tag must be implemented",
  "rationale": "Initial read from ASG tags is required to load the configuration from AWS
to further process it",
  "dependent_on": [],
  "pass": true,
  "evidence": "In `cluster-autoscaler/cloudprovider/aws/aws_manager.go` add a new
extract function `extractScheduleFromAsg()` and wire it up to extract the schedule
format in `buildAsgFromAWS()` in
`cluster-autoscaler/cloudprovider/aws/auto_scaling_groups.go`.",
```

"reasoning": "The response explicitly identifies `aws\_manager.go` as the location to implement the extraction logic for the new tag, satisfying the criterion."

},

{

"id": "reasoning-17996",

"tag": "reasoning",

"weight": "penalty",

"description": "Claims that the current architecture of Cluster Autoscaler does not allow the implementation of any new Scheduling features",

"rationale": "This is wrong, the cluster autoscaler can be easily extended to have this feature",

"dependent_on": [],

"pass": true,

"evidence": "The response explains that the Cluster Autoscaler wasn\u2019t originally designed for scheduled tasks but describes how to extend it via per-cloud-provider implementations, showing it can support new scheduling functionality.",

"reasoning": "The golden answer does not claim that the Cluster Autoscaler architecture prevents implementing new scheduling features; rather, it details how to extend it, so it avoids the prohibited claim."

},

{

"id": "reasoning-17745",

"tag": "reasoning",

"weight": "bonus",

"description": "Identifies the requirement of having a `strict` and `relaxed` mode for better handling of defaults",

"rationale": "Extra feature to handle defaults better, making the auto scaler use defaults when needed.",

"dependent_on": [

```
"completeness-17742"
],
"pass": true,
"evidence": "The answer explicitly defines configuration options including `strict` and `relaxed` flags: \u201c`strict` and `relaxed` are flags that can be used\u2026 In `relaxed` mode\u2026 but in `strict` mode\u2026 the autoscaler will not touch or reduce to the default config.\u201d It further details behavior of each mode under \u201cInner Workings.\u201d",
"reasoning": "The response identifies and explains both `strict` and `relaxed` modes, showing how they manage defaults differently, satisfying the criterion."
},
{
  "id": "completeness-17981",
  "tag": "completeness",
  "weight": "critical",
  "description": "Gives a description of how `MinSize()` and `MaxSize()` can be implemented to support the cron and modes required.",
  "rationale": "Required to help the developer further investigate and implement the feature",
  "dependent_on": [
    "reasoning-17974"
  ],
  "pass": true,
  "evidence": "In the AWS-specific section, the answer states \"we need to override `MinSize()` and `MaxSize()`\" and later explains that in strict mode \"`MinSize()` returns 10, `MaxSize()` returns 40 regardless of pod demand\" while in relaxed mode \"`MinSize()` returns 2, `MaxSize()` returns 10 as soft suggestions.\",
  "reasoning": "The response explicitly describes how `MinSize()` and `MaxSize()` should be overridden and how they behave in both strict and relaxed cron-scheduled modes, satisfying the requirement."
}
```

```
},  
{  
  "id": "completeness-17714",  
  "tag": "completeness",  
  "weight": "bonus",  
  "description": "States that Cluster Autoscaler was originally not designed for time  
based autoscale usage",  
  "rationale": "Cluster Autoscaler was always designed to scale the cluster dynamically  
based on metrics, not to scale it based on specific set of time of day configurations",  
  "dependent_on": [],  
  "pass": true,  
  "evidence": "\u201cThe Kubernetes Cluster Autoscaler was originally not designed to  
handle scheduled tasks.\u201d",  
  "reasoning": "The golden answer explicitly states that the Cluster Autoscaler was not  
originally designed for scheduled tasks/time-based autoscaling, directly satisfying the  
criterion."  
},  
{  
  "id": "completeness-17966",  
  "tag": "completeness",  
  "weight": "critical",  
  "description": "States that `extractScheduleFromAsg` function can be implemented to  
handle the new tag",  
  "rationale": "New function is needed to keep the code modular and easy to replicate  
in other cloud providers",  
  "dependent_on": [  
    "reasoning-17754"  
  ],  
  "pass": true,
```

"evidence": "In `cluster-autoscaler/cloudprovider/aws/aws\_manager.go` add a new extract function `extractScheduleFromAsg()` and wire it up to extract the schedule format in `buildAsgFromAWS()` in `cluster-autoscaler/cloudprovider/aws/auto\_scaling\_groups.go`. This new function captures the value with this key (one or many) and passes it to the function in `utils/schedule\_parser.go`.",

"reasoning": "The answer explicitly instructs to add the `extractScheduleFromAsg` function to handle the new tag by extracting schedule information and feeding it into the parser, satisfying the criterion."

},

{

"id": "completeness-17742",

"tag": "completeness",

"weight": "bonus",

"description": "States the sample scale up and down schedule config that can be used with AWS ASGs",

"rationale": "Example of how the config can be designed for use with AWS Auto Scaling Groups that will be used as tags",

"dependent_on": [

"reasoning-17738"

],

"pass": true,

"evidence": "a. Scale Up and Maintain: k8s.io/cluster-autoscaler/schedule: `0 8 \* \* \* min=10 max=40 strict`\n\n. Scale Down and Maintain: k8s.io/cluster-autoscaler/schedule: `0 22 \* \* \* min=2 max=10 relaxed`",

"reasoning": "The response explicitly presents sample scale-up and scale-down schedule configurations as AWS Auto Scaling Group tags, satisfying the criterion."

},

{

"id": "completeness-17970",


```
"tag": "completeness",
"weight": "critical",
"description": "States that `buildAsgFromAWS` needs to be wired up to read the new
schedule tags",
"rationale": "New function needs to be called along with existing extract functions.",
"dependent_on": [
  "completeness-17966"
],
"pass": true,
"evidence": "\u201cIn `cluster-autoscaler/cloudprovider/aws/aws_manager.go` add a
new extract function `extractScheduleFromAsg()` and wire it up to extract the schedule
format in `buildAsgFromAWS()` in
`cluster-autoscaler/cloudprovider/aws/auto_scaling_groups.go`. \u201d",
"reasoning": "The golden answer explicitly instructs wiring the new
schedule-extraction function into `buildAsgFromAWS`, satisfying the criterion."
},
{
  "id": "completeness-17748",
  "tag": "completeness",
  "weight": "critical",
  "description": "States that `utils/schedule_parser.go` can be used as a shared module
to help parse the schedules from all Cloud Providers",
  "rationale": "Standard way of parsing the schedules can help cloud provider
implementations to focus specifically on how to read the tags.",
  "dependent_on": [],
  "pass": true,
  "evidence": "\u201cThe configuration parser can be implemented as a new file in
`utils/schedule_parser.go` that can handle all the parsing requirements. This can be a
shared module for all cloud providers to use. \u201d",
```

"reasoning": "The golden answer explicitly states that `utils/schedule\_parser.go` should be implemented as a shared module for all cloud providers, satisfying the criterion."

},

{

"id": "completeness-17715",

"tag": "completeness",

"weight": "bonus",

"description": "Gives an example of a use case in the real world from a tech company currently having the exact issue with Cluster Autoscaler",

"rationale": "Examples help the developer better understand if the feature was discussed before on public forums.",

"dependent_on": [],

"pass": true,

"evidence": "\u201cThis is evident further from Tech Blogs of various high scale apps like Hotstar and Netflix where live shows or sporting events attract a huge number of floating crowd. One such example is Hotstar handling a burst of traffic when a match starts and no longer needing that capacity when a match ends.\u201d",

"reasoning": "The golden answer cites real-world examples of Hotstar and Netflix facing the autoscaling timing issue, fulfilling the requirement to provide a concrete tech-company use case."

},

{

"id": "completeness-18019",

"tag": "completeness",

"weight": "penalty",

"description": "States that the implementation must go into vertical pod autoscaler because nodes scale when pods increase their CPU",

"rationale": "Nodes do scale when VPA increases CPU of a pod if required, but that does not answer why the schedule has to go there.",

```
"dependent_on": [],
"pass": true,
"evidence": "The golden answer discusses modifying the Kubernetes Cluster
Autoscaler with provider-specific schedules; it does not mention the Vertical Pod
Autoscaler or claim the implementation must go there.",
"reasoning": "Since the answer never states that the implementation must go into the
VPA, it avoids the specified penalty behavior."
},
{
  "id": "completeness-18025",
  "tag": "completeness",
  "weight": "penalty",
  "description": "Suggests to skip the existing `MinSize()` and `MaxSize()` functionality
by implementing new functions to handle this feature alone",
  "rationale": "Using existing functionality helps in reusing components like actually
calling AWS APIs to push the requirements to ASG controllers.",
  "dependent_on": [],
  "pass": true,
  "evidence": "\u201ccluster-autoscaler/cloudprovider/aws/aws_cloud_provider.go`
we need to override `MinSize()` and `MaxSize()`. \u201d",
  "reasoning": "The golden answer proposes using (overriding) the existing `MinSize()`
and `MaxSize()` methods rather than skipping them and creating entirely new
functionality, thus avoiding the penalized behavior."
},
{
  "id": "completeness-17988",
  "tag": "completeness",
  "weight": "bonus",
  "description": "States to manually restart the CA pod if the changes are required
instantly or near real time",
```

```

    "rationale": "Near real time changes would prove costly if API calls made to AWS frequently and the best way is to restart the pod to force it into pulling ASG config.",
    "dependent_on": [
        "reasoning-17986"
    ],
    "pass": true,
    "evidence": "Restarting the CA Pod can also force refresh the ASG tags if the changes are required near real time.",
    "reasoning": "The golden answer explicitly instructs to manually restart the Cluster Autoscaler pod to force near real-time configuration refresh, satisfying the criterion."
},
{
    "id": "style-17990",
    "tag": "style",
    "weight": "bonus",
    "description": "Separates the answer into distinct sections that describe one part of the architecture at a time",
    "rationale": "Easier for the developer to follow the architecture and help in implementation planning",
    "dependent_on": [],
    "pass": true,
    "evidence": "The response is split into sections such as \"Core Logic Issues,\" \"Configuration Management,\" \"Tag Parsing,\" \"Inner Workings,\" and \"Conclusion,\" each addressing different parts of the architecture.",
    "reasoning": "These titled sections clearly separate the architectural aspects, making the structure easy to follow and satisfying the criterion."
},
{
    "id": "style-17991",
    "tag": "style",

```

```

    "weight": "bonus",
    "description": "Uses precise Go terminology to ground the architectural claims",
    "rationale": "Grounding the architectural claims on the language of the repository
helps developers to easily implement the required feature.",
    "dependent_on": [],
    "pass": true,
    "evidence": "\u201cThe final struct\u2026`type scheduleConfig struct { \u2026 }`
\u2026 \u201cadd a new extract function `extractScheduleFromAsg()` \u2026 wire it up
\u2026 in `buildAsgFromAWS()` \u2026 override `MinSize()` and `MaxSize()`.\u201d",
    "reasoning": "The answer explicitly references Go constructs\u2014struct definitions
and function names\u2014and ties them to specific files in the repository, grounding the
architecture in precise Go terminology."
}
],
"passed": 23,
"failed": 0,
"total": 23,
"score_percent": 100.0,
"summary": "Passed 23/23 criteria (100.0%)"
}

```

Test Model

```
=====
```

RUN 1/3

```
=====
```

Below is an architecture-level plan for adding “time-of-day autoscaling” to Cluster Autoscaler (CA). It focuses on where to extend the codebase, which new abstractions to introduce, and how to expose/configure the feature so that we can move into implementation with minimal churn later.

1. Desired behaviour

- * Allow operators to define time-based rules (e.g., weekdays 08:00-18:00) that change the expected capacity of specific node groups without waiting for workload-driven scale-up.
- * Rules should raise the *minimum* capacity (pre-warm for predictable peaks) and optionally lower the *maximum* capacity (to force scale-down after the window).
- * Rules must be cluster-safe: they should honour regular CA safeguards (max total nodes, resource limits, etc.) and integrate with existing events/metrics.

2. Configuration surface

2.1 CLI / flags (`cluster-autoscaler/main.go`)

Add gated support so the feature is opt-in:

```
``go
var (
    scheduledScalingEnabled = flag.Bool("scheduled-scaling-enabled", false, "Enable
time-of-day scaling rules")
    scheduledScalingConfigMap = flag.String("scheduled-scaling-configmap",
"cluster-autoscaler-schedule", "ConfigMap containing schedule definitions")
    scheduledScalingNamespace = flag.String("scheduled-scaling-namespace", "",
"Namespace for the schedule ConfigMap (defaults to --namespace)")
    scheduledScalingDefaultTZ = flag.String("scheduled-scaling-timezone", "UTC",
"Default timezone for schedule windows")

```

```
    scheduledScalingLookahead = flag.Duration("scheduled-scaling-lookahead",
30*time.Minute, "How far ahead CA should protect capacity before a window begins")
)
``
```

Propagate these knobs into `AutoscalingOptions` (`config/autoscaling\_options.go`) and `AutoscalingContext` (`context/autoscaling\_context.go`) so downstream processors can consume them.

2.2 ConfigMap data model

Store schedules in a dedicated ConfigMap (similar to the priority expander config in `expander/priority`). Example:

```
``yaml
apiVersion: v1
kind: ConfigMap
metadata:
  name: cluster-autoscaler-schedule
  namespace: kube-system
data:
  schedule.yaml: |
    timezone: America/Los_Angeles # optional default
    rules:
      - name: business-hours
        days: [Mon, Tue, Wed, Thu, Fri]
        start: "08:00"
        end: "18:00"
        nodeGroups:
          - id: "ng-us-west-1"
```

```
    min: 20
    max: 100
- name: nightly-batch
  days: [Mon, Tue, Wed, Thu, Fri]
  start: "21:00"
  end: "23:00"
  nodeGroups:
  - id: "ng-batch"
    min: 40
...

```

* `days` is optional; absence $\Rightarrow$ 7 days.

* `start`/`end` are local times in HH:MM (24h).

* `min`/`max` overrides are independent (max optional).

We can extend this later with cron expressions or per-rule timezones if needed.

3. New internal components

3.1 Schedule loader & cache (new package `cluster-autoscaler/scheduledscaling`)

Responsibilities:

1. Watch/read the ConfigMap via the existing lister framework (`utils/kubernetes/listers.go`) to avoid bespoke clients.
2. Parse YAML into canonical structs, validate overlaps/conflicts, convert to UTC windows.
3. Expose a pure function:


```

``go
type Overrides struct {
    NodeGroup string
    MinOverride *int
    MaxOverride *int
    Active bool      // true if current time within window
    NextWindow time.Time // start of next window (for lookahead)
}

type Planner interface {
    CurrentOverrides(now time.Time) []Overrides
    Refresh(ctx context.Context) // triggered when config map changes
}
...

```

Implementation details:

- * On ConfigMap changes, rebuild an in-memory schedule map keyed by node group.
- * Accept a `Clock` interface to keep tests deterministic.

3.2 Processor hook

Introduce a new processor (e.g., `ScheduledScalingProcessor`) so we stay within the existing extension framework (`cluster-autoscaler/processors`). Add it to `AutoscalingProcessors` in `processors/processors.go` and gate its creation in `core/autoscaler.go` when the feature flag is set.

Responsibilities per RunOnce:

1. Inspect `clusterstate.ClusterStateRegistry` (`ClusterStateRegistry.GetStatus`) for current/target sizes.

2. Ask the `Planner` for overrides at `currentTime`.

3. Enforce **min overrides**:

- * If `currentTarget < minOverride`, calculate `delta`.

- * Use the same path as scale-up (`core/scale\_up.go`) by preparing a fake “demand” object rather than bypassing CA safeguards.

- * Implementation sketch:

- Add helper `ScheduleScaleUp(context, registry, nodeInfos, nodeGroupID, delta)` that wraps `ScaleUp` internal primitives (choose node group, call `nodeGroup.IncreaseSize`, register requests, emit events).

- Because there are no backing pods, log events with a dedicated reason (`ScheduledScaleUp`) and skip predicate simulations (the node template is already known).

4. Respect **max overrides** / future windows:

- * Before scale-down, reduce the “acceptable range” for a node group so CA will hurry scale-down when `currentTarget > maxOverride`.

- * Implementation: extend `ClusterStateRegistry.updateAcceptableRanges` to subtract scheduled deltas. This ensures scale-down logic sees those nodes as removable and still honours graceful policies.

5. “Lookahead” protection:

- * If a window starts within `--scheduled-scaling-lookahead`, treat `minOverride` as active already to give CA time to spin nodes up.

Integration point in `core/static\_autoscaler.go`:

```
...
```

```
stateUpdateStart := time.Now()
```

```
...
```

```
if a.scheduledScalingProcessor != nil {
```

```
    if err := a.scheduledScalingProcessor.Apply(autoscalingContext,  
a.clusterStateRegistry, nodeInfosForGroups, currentTime); err != nil {
```

```

    klog.Errorf("Scheduled scaling failed: %v", err)
    return errors.ToAutoscalerError(errors.InternalError, err)
}
}
...

```

Place this **after** `updateClusterState` (so we have fresh readiness info) and **before** evaluating unschedulable pods.

3.3 Cluster state awareness

`ClusterStateRegistry` already tracks readiness/target sizes per node group (`perNodeGroupReadiness`, `acceptableRanges`). We need two small extensions:

1. Provide a helper `GetNodeGroupSizes(id string) (ready, target int)` so the processor can decide how many nodes to add/remove.
2. Allow temporary min/max overrides when computing acceptable ranges. Add two optional maps (`scheduledMin`, `scheduledMax`) injected each loop by the processor, and apply them inside `updateAcceptableRanges`. This prevents scale-down from going below scheduled mins even if there is no scheduled scale-up activity that iteration.

All new data should live only in memory (no API changes) and reset each loop to avoid stale overrides if the feature is disabled at runtime.

4. Control flow summary

1. **Startup**:
 - * Flags parsed in `main.go`.
 - * `createAutoscalingOptions` populates new fields.

- * `core.NewAutoscaler`` wires a `ScheduledScalingProcessor`` only when enabled.

2. **RunOnce** (after nodes/pods fetched):

- 1. `ScheduledScalingProcessor.Apply()` obtains overrides from the planner.

- 2. For each node group:

- * If `CurrentTarget < ScheduledMin``, register a scale-up (calls into the same code path used for pod-driven scale-ups).

- * If `CurrentTarget > ScheduledMax``, push info into `ClusterStateRegistry`` so the forthcoming scale-down run immediately considers those nodes as removable.

- 3. Downstream logic (filter pods, ScaleUp, ScaleDown) proceeds unchanged but sees the cluster already converging to the scheduled bounds.

3. **Config updates**: the planner listens for ConfigMap changes; when they happen, it rebuilds rules and toggles a `generation`` counter. The processor checks the counter each loop to decide whether to log events (`"Reloaded scheduled scaling config rev=123"`).

5. Validation & safety

- * **Conflict handling**: When overlapping windows specify different mins for the same node group, resolve deterministically (e.g., "highest min wins"; log a warning).

- * **Timezone correctness**: convert rules to UTC at load time; support per-rule `timezone`` overrides (optional field) falling back to `spec.timezone`` or flag default.

- * **Back-pressure**: honour `MaxNodesTotal`` / resource limits by verifying deltas against `config.AutoscalingOptions`` before calling `IncreaseSize``. If a scheduled request would violate limits, emit a warning event and skip the action.

- * **Observability**:

- * Emit events (`ScheduledScaleUp``, `ScheduledScaleDown``) on the ConfigMap owner reference (or the CA Deployment) so operators can audit behaviour.

* Add metrics: ``scheduled_scale_up_requested_total``, ``scheduled_override_active{node_group}`` gauge, etc., alongside the existing metrics pipeline (``metrics`` package).

6. Testing strategy

- * Unit tests for the planner (parse schedules, timezone math, overlap resolution).
- * Unit tests for processor logic using fake node groups (``utils/test``) to verify scale-up/down decisions.
- * Integration-style test around ``StaticAutoscaler.RunOnce`` that injects a fake planner and asserts CA registers the correct scale-up requests without actual unschedulable pods.

7. File touchpoints recap

| Area | Files |

|-----|-----|

| Flags & options | ``cluster-autoscaler/main.go``, ``config/autoscaling_options.go``,
``context/autoscaling_context.go`` |

| Processor wiring | ``core/autoscaler.go``, ``processors/processors.go``, new
``processors/scheduledscaling/...`` |

| Core loop integration | ``core/static_autoscaler.go`` |

| Cluster state support | ``clusterstate/clusterstate.go`` (helpers & acceptable-range
overrides) |

| Planner package | new ``cluster-autoscaler/scheduledscaling/{loader.go, planner.go}``
plus tests |

| Docs | `cluster-autoscaler/README.md`, `cluster-autoscaler/FAQ.md` (new section describing scheduled scaling) |

This architecture keeps the feature isolated (new planner + processor), reuses existing CA mechanics for actual scaling, and leans on ConfigMaps for operator ergonomics. Once you're comfortable with the design, we can start slicing the work into implementation PRs (flags/options, planner, processor, tests, docs).

=====

RUN 2/3

=====

1. Executive summary

We can add "time of day" autoscaling without refactoring the entire Cluster Autoscaler (CA) loop by introducing a **scheduled-scaling engine** that (a) periodically computes time-based capacity overrides and (b) feeds those overrides into existing scale-up and scale-down paths. The integration hinges on a few well-scoped entry points:

| Area | File (line range) | Why it matters |

|----|-----|-----|

| CLI / options | `cluster-autoscaler/main.go:L82-L240` | All new flags and initial parsing happen here. |

| Config object | `cluster-autoscaler/config/autoscaling\_options.go` |

`AutoscalingOptions` must carry schedule data so that it reaches the core logic. |

| Context | `cluster-autoscaler/context/autoscaling\_context.go` | Every processor/scale component reads configuration from here. |

| Main loop | `cluster-autoscaler/core/static\_autoscaler.go` | Best place to inject schedule enforcement before regular scale-up/down logic. |

| Scale-down candidate filtering |

`cluster-autoscaler/processors/nodes/pre\_filtering\_processor.go:L31-L74` | Today it only respects static min-size. We can reuse this choke point to honor time-based minimums. |

2. Configuration & API design

1. **New CLI flags (parse in `main.go`)**

- `--scheduled-scaling-configmap=<ns/name>` (mutually exclusive with `--scheduled-scaling-config-file`).
- `--scheduled-scaling-default-timezone=<tz>` (fallback when a window does not specify one).
- Optional `--scheduled-scaling-dry-run` to emit metrics/events without actually touching node groups.

2. **File / ConfigMap schema (YAML)**

```yaml

policies:

- name: "weekday-daytime"  
nodeGroupSelector: # match either explicit nodegroup IDs or labels  
ids: ["gke-primary", "gke-batch"]  
windows:
  - days: ["Mon","Tue","Wed","Thu","Fri"]  
start: "08:30"  
end: "18:00"  
timezone: "America/Los\_Angeles"  
minNodes: 10  
maxNodes: 40 # optional upper guard  
targetCapacity: 25 # optional proactive scale-up  
scaleDownDelay: "30m" # don't allow scale-down for 30m after window closes

...

Parsing/validation lives in a new package (e.g. ``cluster-autoscaler/scheduled/config.go``). It should:

- \* Compile windows into absolute ranges (``time.Time`` with week offsets).
- \* Resolve timezones once (use ``time.LoadLocation``).
- \* Support overlapping policies by taking the **max** of all requested mins for a node group.

### 3. **AutoscalingOptions additions**

In ``config/autoscaling_options.go``, add:

```
``go
ScheduledScalingEnabled bool
ScheduledScalingPolicies []scheduled.Policy
ScheduledScalingDefaultTZ string
...
```

Update ``createAutoscalingOptions()`` (``main.go:L176+``) to populate these fields after reading the flag(s). When the source is a `ConfigMap`, reuse existing client-go config reading (``config/kubernetes.go``) machinery.

### 4. **Context plumbing**

Add ``ScheduleEngine scheduled.Engine`` to ``context.AutoscalingContext``. The engine is constructed inside ``core.NewAutoscaler()`` once options are validated. Every subsystem can then call ``ctx.ScheduleEngine`` instead of re-reading files.

### ### 3. Scheduled scaling engine (new pkg ``cluster-autoscaler/scheduled``)

Responsibilities:

1. Hold compiled policies.
2. Expose pure functions:
  - ``DesiredMin(nodeGroupID string, when time.Time) (min int, ok bool)``



- `DesiredTarget(nodeGroupID string, when time.Time) (target int, reason string, ok bool)`

- `ScaleDownCooldown(nodeGroupID string, when time.Time) time.Time`

3. Optionally watch the ConfigMap (using the same client set created in `context.NewAutoscalingKubeClients`). When data changes, atomically swap the compiled schedule.

Implementation sketch:

```
```go
type Engine interface {
    Evaluate(now time.Time) map[string]Evaluation // keyed by nodeGroup ID
}
```

```
type Evaluation struct {
    MinNodes      *int
    TargetNodes    *int
    MaxNodes      *int
    ScaleDownBlockedUntil *time.Time
}
...

```

4. Control-loop integration (core/static_autoscaler.go)

1. **Early enforcement (pre scale-up)**

At the beginning of `RunOnce` (~ line 215), after `nodeInfosForGroups` is built but before we look at unschedulable pods, invoke:

```
```go
a.applyScheduledScaleUp(currentTime, nodeInfosForGroups)
...

```

Algorithm:

- For each node group with a `TargetNodes`` override greater than the current target (obtain current target via `clusterStateRegistry``'s acceptable range or `NodeGroup.TargetSize()``):
  - \* Call `NodeGroup.IncreaseSize(delta)`` (same path as regular scale-up).
  - \* Mirror what `ScaleUp()`` does: `clusterStateRegistry.RegisterOrUpdateScaleUp()`` so health/backoff logic stays consistent.
  - \* Emit metric `scheduled_scale_up_forced`` with labels (policy, nodegroup).
- For groups below their **minimum** override, treat `TargetNodes = MinNodes`` so they get bumped immediately even if no pods are pending.

## 2. **Scale-down guard**

When `ScaleDown`` asks the node processor for candidates (`processors/nodes/pre_filtering_processor.go:L31-L74``), compute an **effective min**:

```
``go
scheduledMin := ctx.ScheduleEngine.MinNodes(nodeGroup.Id(), now)
enforcedMin := max(nodeGroup.MinSize(), scheduledMin)
if size <= enforcedMin { skip node }
``
```

This single change prevents draining nodes that would violate the next window. Because this file already talks to `AutoscalingContext``, no extra plumbing is required beyond adding the engine to the context.

## 3. **Cooldown handling**

Before calling `scaleDown.TryToScaleDown`` (~line 500), check whether any node group has `ScaleDownBlockedUntil > now`` and, if so, set `scaleDownStatus.Result = status.ScaleDownInCooldown``. You can reuse the existing callback mechanism (`processorCallbacks``) by setting `SetExtraValue("scheduled-scale-down-blocked", true)`` so downstream processors can emit events.

## 4. **Cluster-state awareness**

To keep `clusterstate.ClusterStateRegistry`` (see `cluster-autoscaler/clusterstate/clusterstate.go``) honest, feed scheduled minimums into `updateAcceptableRanges()``:

- After computing `MinNodes := size - readiness.LongUnregistered``, wrap it with `max(MinNodes, scheduledMin)``.

This ensures health checks and status ConfigMaps reflect the enforced minimum.

### ### 5. Metrics, events, and observability

Add gauges/counters in `metrics/metrics.go``:

- `scheduled_scale_forced_up_total`` (labels: policy, nodegroup, reason=min|target).
- `scheduled_scale_blocked_down`` gauge (per nodegroup) to show cooldowns.

Emit EVENTS (via `ctx.LogRecorder``) when:

- A schedule bumps capacity even though no pods were pending (“ScheduledScalingUp”).
- Scale-down is skipped due to upcoming window (“ScheduledScalingGuard”).

### ### 6. Config reload & validation

- When using ConfigMap, reuse the same `ListerRegistry`` informer factory (`context/autoscaling_context.go``). Add a lightweight controller under `scheduled/loader.go`` that watches that ConfigMap, parses YAML, and updates the engine through an atomic pointer.
- Reject invalid configs during boot (fail fast in `main.go``).
- Surface validation errors through `/health-check`` by wiring them into `metrics.HealthCheck`` (already instantiated in `main.go:L356``).

### ### 7. Failure / edge cases

- **Overlapping policies**: define deterministic merge order (e.g., `max(minNodes)`, `min(maxNodes)`, highest target wins). Document this in the YAML schema.
- **Timezones & DST**: compile windows using `time.Location``, and store next-trigger timestamps to avoid recomputing every loop.
- **CloudProvider constraints**: if a scheduled target exceeds `NodeGroup.MaxSize()``, emit warning and cap at max. If it is lower than `MinSize()``, still allow `MinSize()`` (the provider already enforces that).
- **Dry run**: honor `--scheduled-scaling-dry-run`` by skipping the `IncreaseSize`` call but still logging what would happen.

### ### 8. Summary of code touch points

| Change | Files |

|-----|-----|

| Flags + option wiring | ``cluster-autoscaler/main.go`` |

| Options struct & serialization | ``cluster-autoscaler/config/autoscaling_options.go`` |

| New scheduled package | ``cluster-autoscaler/scheduled/*`` |

| Context carries engine | ``cluster-autoscaler/context/autoscaling_context.go`` |

| Static autoscaler invokes engine | ``cluster-autoscaler/core/static_autoscaler.go`` |

| Scale-down candidate filter respects schedule |

``cluster-autoscaler/processors/nodes/pre_filtering_processor.go`` |

| Cluster state min-range incorporates schedule |

``cluster-autoscaler/clusterstate/clusterstate.go`` |

| Metrics | ``cluster-autoscaler/metrics/metrics.go`` |

With this architecture, the existing CA behavior remains unchanged when no schedule is configured, but operators gain precise, time-of-day control over minimum/target capacity via declarative policies.

=====

RUN 3/3

=====

### ### 1. Current control loop and places we must touch

| Responsibility | Where it lives today | Why it matters for time-of-day scaling |

| -- | -- | -- |

| CLI / flag parsing → `AutoscalingOptions` | `cluster-autoscaler/main.go` (flag definitions around lines 70-250) feeds into `createAutoscalingOptions()` which populates `config.AutoscalingOptions` (`config/autoscaling\_options.go`) | Any new feature must be surfaced here (flags, config files, ConfigMaps) so that the options are visible everywhere through `AutoscalingContext`. |

| Main reconciliation loop | `core/static\_autoscaler.go`, especially  
`(\*StaticAutoscaler).RunOnce()` (roughly lines 120-520) | This loop decides whether to scale up/down. Time-based logic needs to run inside this loop so it can influence both scale-up triggers and how scale-down treats nodes. |

| Scale-down protection | Also in `core/static\_autoscaler.go` (lines 530-880) and  
`core/scale\_down.go` | If a schedule requires “keep X nodes from Y:00 to Z:00”, scale-down must respect that floor (e.g., mark nodes as unremovable or adjust min-size logic). |

| Cloud provider interaction | NodeGroup abstractions in `cloudprovider` packages |

Enforcing scheduled min/max per node group eventually calls

`IncreaseSize/DecreaseTargetSize`, so any direct manipulations must go through these interfaces. |

### ### 2. Proposed architecture for time-of-day autoscaling

#### #### 2.1 Configuration model

Add a structured policy to `AutoscalingOptions`:

```go

```
// config/timebased.go (new file)
type TimeWindow struct {
    Start string `json:"start" // "22:00"
    End   string `json:"end"  // "06:00"
    Min   int    `json:"min"`
    Max   *int    `json:"max,omitempty"`
    Days  []time.Weekday `json:"days,omitempty" // optional
    Timezone string `json:"timezone,omitempty"`
}

type TimeBasedPolicy struct {
    NodeGroupID string    `json:"nodeGroupId,omitempty"`
    NodeGroupSelector map[string]string `json:"nodeGroupSelector,omitempty" // match
labels/tags
    Windows []TimeWindow `json:"windows"`
}

type TimeBasedAutoscalingConfig struct {
    Policies []TimeBasedPolicy `json:"policies"`
}
...

```

Then extend `config.AutoscalingOptions` (`config/autoscaling\_options.go`) with:

```
```go
 TimeBasedConfigPath string
 TimeBasedConfig *TimeBasedAutoscalingConfig
...

```

New CLI flags in `main.go` (near other flag declarations):

```

...

timeBasedConfigFile = flag.String("time-based-config", "",
 "Path to JSON/YAML describing time-of-day scaling policies")
timeBasedConfigMap = flag.String("time-based-configmap", "",
 "ConfigMap (namespace/name) to watch for time-of-day policies")
timeBasedDefaultTZ = flag.String("time-based-default-timezone", "UTC",
 "Default IANA timezone for time-of-day rules")
...

```

During `createAutoscalingOptions()` parse the file/ConfigMap (re-using the existing kube client, similar to how dynamic configs are handled under `config/dynamic`). We already write/read ConfigMaps in `cluster-autoscaler/config/dynamic`, so we can mirror that pattern.

**\*Why both file and ConfigMap?\*** – File works for static deployments; ConfigMap enables live reconfiguration without restart.

## #### 2.2 Runtime wiring

1. **\*\*AutoscalingContext exposure\*\*** – add a helper in `context/autoscaling_context.go` (not shown above but fed by options) to cache the parsed `TimeBasedAutoscalingConfig` and to expose a method:

```

```go
func (c *AutoscalingContext) ActiveTimePolicies(now time.Time) []ResolvedPolicy
...

```

`ResolvedPolicy` would be the concrete “for nodeGroup <id>, enforce min=m, max=n right now”.

2. **New processor** – mimic existing processor slots (see `processors`` package).
Create `processors/timebased/time_based_processor.go`` with interface:

```
``go
type TimeBasedProcessor interface {
    EnsureScheduledCapacity(ctx *context.AutoscalingContext, now time.Time,
        allNodes []*apiv1.Node, nodeInfos map[string]*scheduling.NodeInfo)
(actions ActionsTaken, err error)
}
``
```

Default implementation:

- Iterate active policies.
- Resolve target node groups (by ID or via selector using `cloudProvider.NodeGroups()`` metadata).
- Compare desired min/max with:
 - Actual size (`nodeGroup.TargetSize()``).
 - Kubernetes view (length of nodes belonging to that group).
- If current < scheduled min:
 - Call `nodeGroup.IncreaseSize(delta)`` and record the action into `clusterStateRegistry.RegisterOrUpdateScaleUp(...)`` so the registry is aware (see `core/static_autoscaler.go`` lines 299-333 for similar flows).
 - Optionally annotate nodes to prevent scale-down until the scheduled period ends.
- If current > scheduled max (if defined):
 - Mark extra nodes as “scheduled for removal” by injecting an artificial unremovable reason except for the nodes above max, and let scale-down handle them in the next phase (or call `nodeGroup.DecreaseTargetSize`` directly if allowed).

Hook this processor from `core.NewAutoscaler()` (right after existing processors are wired in `main.go`, lines ~280-307). If no config is provided, use a no-op implementation.

3. **RunOnce integration** – early in `StaticAutoscaler.RunOnce()` after we've collected node lists and before we branch into scale-up/down logic (roughly after we call `a.updateClusterState(...)` around line 274), invoke:

```
``go
if a.processors != nil && a.processors.TimeBasedProcessor != nil {
    actions, err := a.processors.TimeBasedProcessor.EnsureScheduledCapacity(
        a.AutoscalingContext, currentTime, allNodes, nodeInfosForGroups)
    ...
}
``
```

`actions` can annotate the `autoscaler` status ConfigMap (already written in the `defer` block later) and record metrics.

4. **Scale-down awareness**

- For nodes that are needed only to satisfy a schedule, we must not let regular utilization-based logic evict them. We can achieve this by extending `TimeBasedProcessor` to mark nodes with the existing annotation cluster-autoscaler.kubernetes.io/scale-down-disabled (see `ScaleDownDisabledKey` in `core/scale_down.go` line 35). A helper can:
 - Choose `min` nodes in the group (e.g., oldest first or ones already annotated) and ensure the annotation is set.
 - When the window ends, remove the annotation so normal scale-down resumes.

- Because ``scale_down.go`` already checks for that annotation (``hasNoScaleDownAnnotation``, lines 615-626), no further changes are required there besides ensuring annotations are managed.

5. ****Max enforcement**** (optional): to treat “scheduled max”, feed a synthetic resource limit into ``scaleDownResourcesLeft`` so that the node removal side prefers those nodes even if utilization is low. This could be as simple as storing “policyMax[nodeGroupID]” inside ``staticAutoscalerProcessorCallbacks`` (``core/static_autoscaler.go``, lines 90-115) and reading it in ``ScaleDown.chooseCandidates``. For a first iteration, it’s usually enough to request ``DecreaseTargetSize`` via cloud provider if current > scheduled max.

2.3 Data flow summary

...

Flags/ConfigMap -> AutoscalingOptions.TimeBasedConfig

↓

context.AutoscalingContext (new helper)

↓

processors.TimeBasedProcessor.EnsureScheduledCapacity()

↓

- Direct NodeGroup Increase/Decrease calls
- Node annotations for scale-down protection

↓

ScaleUp/ScaleDown logic sees updated cluster state (some nodes already pending, some protected)

...

3. Configuration mechanisms

1. ****Static file**** – mount JSON/YAML via ConfigMap/Secret, pass ``--time-based-config=/etc/ca/schedule.yaml``.

2. **Live ConfigMap** – set `--time-based-configmap=kube-system/ca-time-schedule`.
Use an informer (similar to `utils/kubernetes/listers.go`) to watch for updates; when `resourceVersion` changes, re-parse and swap the in-memory policy (guard with `RWMutex`). Because `AutoscalingContext` already has kube clients, the watcher can be started alongside other background components in `StaticAutoscaler.Start()`.

Example YAML:

```
``yaml
policies:
- nodeGroupSelector:
  cloud.google.com/node-group: spot-ng
  windows:
  - start: "22:00"
    end: "06:00"
    min: 10
    timezone: "America/Los_Angeles"
  - start: "06:00"
    end: "22:00"
    min: 3
- nodeId: "prod-ng-a"
  windows:
  - start: "00:00"
    end: "24:00"
    min: 5
    max: 15
...

```

Validation should happen on load (reject overlapping windows per policy, ensure `min <= max`, parse timezone).

4. Edge cases & considerations

- **Time zones / DST** – use `time.Location` when parsing windows. Decide how to handle overnight spans (e.g., start 22:00, end 06:00) by normalizing to “clock range possibly wrapping past midnight.”
- **Overlapping policies** – define deterministic precedence (e.g., most specific selector > ID > default). Document this.
- **Interaction with HPA / regular CA logic** – scheduled min guarantees capacity but does not suppress reactive scale-up; unschedulable pods still trigger scale-up beyond scheduled min. Conversely, scale-down never goes below scheduled min even if utilization is low.
- **Metrics & status** – extend `metrics` to record when schedule-driven scale-ups/downs happen and surface the active window in the status ConfigMap (written near the `defer` block in `RunOnce`).
- **Testing** – add unit tests for policy parsing, matching, and a fake clock integration test that runs `RunOnce` at different times to ensure the processor requests the right actions. Use the existing fake cloud provider in `cluster-autoscaler/core/scale_test_common.go`.

5. Implementation roadmap

1. **Config parsing**

- Add new struct definitions + parser (`config/timebased.go`).
- Extend CLI flags and `createAutoscalingOptions()` (`main.go` and `config/autoscaling_options.go`).

2. **Context plumbing**

- Store parsed config in `AutoscalingContext`; expose helper methods.

3. **Processor**

- Implement `TimeBasedProcessor` (new package under `processors/timebased`).
- Wire into `ca_processors.DefaultProcessors()` (probably in `processors/processors.go`) and `core.NewAutoscaler()` so it's available via `a.processors.TimeBasedProcessor`.

4. **Run loop integration**

- Call processor early in `RunOnce`.
- Manage annotations and NodeGroup size changes inside the processor.

5. **Optional config reloader**

- If ConfigMap-based config is desired, create a lightweight watcher using existing informer utilities in `utils/kubernetes/listers.go`.

With this layout the feature stays orthogonal to existing scale-up/down logic, leverages the Processor abstraction, and keeps configuration centralized in `AutoscalingOptions`, making it easy to reason about and test.

model evaluation

```
{
  "points": 7.0,
  "maxPoints": 36.0,
  "score_percent_weight": 19,
  "summary_weight": "7.0/36.0 points (19%)",
  "eval_runs": 3,
  "eval_runs_succeeded": 3,
  "per_run_points": [
    7.0,
    7.0,
    7.0
  ],
}
```

```
"results": [  
  {  
    "id": "reasoning-17738",  
    "weight": "critical",  
    "pass": false,  
    "points": 0.0  
  },  
  {  
    "id": "reasoning-17737",  
    "weight": "bonus",  
    "pass": false,  
    "points": 0.0  
  },  
  {  
    "id": "reasoning-18004",  
    "weight": "penalty",  
    "pass": true,  
    "points": -1.0  
  },  
  {  
    "id": "reasoning-17974",  
    "weight": "critical",  
    "pass": false,  
    "points": 0.0  
  },  
  {  
    "id": "reasoning-17734",  
    "weight": "critical",  
    "pass": false,  
    "points": 0.0  
  }  
]
```

```
},  
{  
  "id": "reasoning-18032",  
  "weight": "penalty",  
  "pass": false,  
  "points": 0.0  
},  
{  
  "id": "reasoning-17986",  
  "weight": "bonus",  
  "pass": false,  
  "points": 0.0  
},  
{  
  "id": "reasoning-17733",  
  "weight": "critical",  
  "pass": true,  
  "points": 3.0  
},  
{  
  "id": "reasoning-17754",  
  "weight": "critical",  
  "pass": false,  
  "points": 0.0  
},  
{  
  "id": "reasoning-17996",  
  "weight": "penalty",  
  "pass": false,  
  "points": 0.0
```

```
},  
{  
  "id": "reasoning-17745",  
  "weight": "bonus",  
  "pass": false,  
  "points": 0.0  
},  
{  
  "id": "completeness-17981",  
  "weight": "critical",  
  "pass": true,  
  "points": 3.0  
},  
{  
  "id": "completeness-17714",  
  "weight": "bonus",  
  "pass": false,  
  "points": 0.0  
},  
{  
  "id": "completeness-17966",  
  "weight": "critical",  
  "pass": false,  
  "points": 0.0  
},  
{  
  "id": "completeness-17742",  
  "weight": "bonus",  
  "pass": false,  
  "points": 0.0
```



```
},  
{  
  "id": "completeness-17970",  
  "weight": "critical",  
  "pass": false,  
  "points": 0.0  
},  
{  
  "id": "completeness-17748",  
  "weight": "critical",  
  "pass": false,  
  "points": 0.0  
},  
{  
  "id": "completeness-17715",  
  "weight": "bonus",  
  "pass": false,  
  "points": 0.0  
},  
{  
  "id": "completeness-18019",  
  "weight": "penalty",  
  "pass": false,  
  "points": 0.0  
},  
{  
  "id": "completeness-18025",  
  "weight": "penalty",  
  "pass": false,  
  "points": 0.0
```

```
},  
{  
  "id": "completeness-17988",  
  "weight": "bonus",  
  "pass": false,  
  "points": 0.0  
},  
{  
  "id": "style-17990",  
  "weight": "bonus",  
  "pass": true,  
  "points": 1.0  
},  
{  
  "id": "style-17991",  
  "weight": "bonus",  
  "pass": true,  
  "points": 1.0  
}  
]  
}
```

model eval score 19.4

difficulty **hard**



Reviewer Guidelines

Reviewer Guidelines

This section outlines responsibilities and guidelines for reviewers. Writers may also read it to understand what reviewers are checking before approval.

New Reviewer Onboarding

Reviewers should be familiar with the full writer workflow, especially Task Claiming & Setup, Repo Exploration & Task Scope, Prompt & Golden Answer Writing, Rubric Writing & Difficulty Calibration, Rubric Design 101, and the Task Examples.

Before reviewing independently, read the examples and ask in the reviewer Slack channel when a task raises a guideline question or ambiguous edge case.

Understanding the Job

Reviewers are guardians of quality. Your primary job as a reviewer is to maintain quality standards while keeping the task pipeline moving. At all times you should operate with professional courtesy, and expect the same from all colleagues. Secondary priorities include:

- Providing education for new writers
- Flagging problems in documentation, workflows, etc.
- Improving your own efficiency (and sharing these improvements)

Reviewer performance is measured by AHT (Average Handling Time) per review, as well as quality (typically measured by overturn rate by Super Reviewer). Work with Team Leads to understand your performance and ensure you meet targets.

Reviewer Guidelines

1. How to Review

Reviewers are responsible for checking every part of the task before approval: prompt, golden answer, rubric, repository evidence, metadata, QC outputs, and difficulty calibration. Use the checklist below to keep reviews consistent.

Review Area	What To Check	Send Back / Escalate If
Assignment &	Repo, commit, language, intent,	Wrong repo/commit/language,

repo setup	difficulty, Dockerfile, and task metadata all match the assignment.	inaccessible repo, broken Dockerfile, or metadata that prevents a fair review.
Prompt quality	Prompt is developer-style, repo-specific, fair, concise, and aligned to the assigned intent/difficulty. It gives starting context without leaking the answer. It reads as human.	Prompt is generic, too broad, a local lookup, answer-leaking, mismatched to intent, non-human sounding (“robotic”), or impossible to verify from the repo.
Golden answer	Answer is correct, complete, grounded in repository evidence, and explains the reasoning chain with relevant files/functions/behaviors. Answer scores 100% on QC.	Unsupported claims, missed causal steps, wrong file/function references, incomplete PR impact, or answer content that the repo does not support.
Rubric quality	Criteria are atomic, measurable, tagged/weighted correctly, include logical dependencies (if any), include fair alternatives, and let the golden answer earn full credit.	Vague criteria, overlapping criteria, placeholder text, direct reversal penalties, style as critical/penalty, or missing important failure modes.
QC & validation	Writer used Prompt QC, Rubric QC, Golden Answer QC, Validate Golden, and Step 5 eval as appropriate, then applied human judgment.	Golden validation fails, QC flags valid unresolved issues, or the task was submitted without the required calibration evidence.
Difficulty & policy	Step 5 score and trajectory fit the assigned band; work follows LLM usage policy and is meaningfully distinct from prior tasks.	Difficulty is out of band, model succeeds with shallow exploration, suspected LLM-authored prompt/golden answer/rubric (non-ASCII characters are a red flag), duplicate task pattern, or policy concern.

2. Reviewer Decision Rules

Use the decision rules below to decide whether to approve, make a small direct edit, send the task back, or escalate it.

Decision	Use When	Reviewer Action
Approve	The task has no substantive issues, or only tiny issues that do not affect task meaning or grading.	Approve. Leave concise feedback if there is a useful note for the writer.
Direct edit + approve	The only issue is low-risk: typo, formatting, obvious label cleanup, or a minor rubric wording fix that does not change task substance.	Make the edit directly, re-run eval, then approve and mention what you changed.
Send back or escalate	Prompt, golden answer, rubric, repo evidence, policy, or difficulty calibration needs substantive writer judgment; or the task is on its 3rd+ revision cycle with unresolved issues.	Send back with specific artifact-level feedback, or escalate to a Team Lead when access/metadata/policy/repeat ed-cycle issues block a normal review.

3. Priorities

Prioritize review work before writing tasks. Clear reviews from oldest to newest, give priority to writers who may be blocked (new/at claim cap), then review newly submitted tasks. Only after these steps, write new tasks if you are also contributing as a writer. (Do not run your reviewer timer when writing your own tasks.)

4. Communication

Leave feedback that tells the writer exactly what to change and why. Reference the relevant artifact, such as prompt, golden answer, specific rubric criteria, Step 5 eval output, or the task metadata.

Use the reviewer Slack channel for guideline questions, suspected policy violations, repeated writer issues, duplicate task patterns, or project-wide clarifications. Reviewers should proactively answer writer questions when the instruction doc already resolves the issue.

If the documentation is unclear, flag the issue for a Team Lead in Slack. Drive all decisions into documentation, avoiding “lore” stored in chat.

5. Time Expectations

Most reviews should take 30-45 minutes. Spend longer only when the task is hard, repo evidence is subtle, the Step 5 calibration needs careful interpretation, or you are a new reviewer. If a review is blocked by missing access, wrong metadata, or an unusable repo, escalate instead of spending review time reconstructing the task. Rely on Team Leads to flag tasks or handle special cases.

6. Maximum Review Cycles

If a task has already been through 3 or more revision cycles, avoid another broad send-back. Make low-risk direct edits if the task is otherwise approvable, or escalate to a Team Lead when the task still has substantive prompt, answer, rubric, or calibration problems.

7. Exceptions

When a delivery is imminent, sometimes reviewers enter “fix-it mode.” When instructed by a Team Lead, reviewers may prioritize fixing a task (even if handling time increases) in order to meet delivery deadlines.